

STAT 8670 – Computational Methods in Statistics

2026 Fall Course Notes

Chi-Kuang Yeh

2026-09-22

Table of contents

Preface	7
Description	7
Prerequisites	7
Instructor	7
Office Hour	7
Grade Distribution	7
Assignment	8
Midterm	8
Project	8
Topics and Corresponding Lectures	8
Recommended Textbooks	8
Side Readings	9
 1 Introduction to R	 10
1.1 Why R?	10
1.2 The working environment	11
1.2.1 RStudio	12
1.2.2 R scripts and Quarto documents	13
1.2.3 A project-oriented workflow	14
1.3 R as a computing environment	15
1.4 Objects, data, and functions	16
1.5 Expressions, objects, and names	16
1.6 Data types	17
1.6.1 Changing data types	18
1.7 Operators	19
1.8 Vectorization	20
1.9 Recycling	21
1.10 Coercion	22
1.11 Missing and special values	23
1.12 Indexing and subsetting	24
1.13 Naming elements	26
1.14 Arrays and matrices	27
1.15 Lists and key-value pairs	29
1.16 Factors	31

1.17	Data frames	32
1.17.1	Inspecting a data frame	33
1.17.2	Subsetting a data frame	34
1.17.3	Adding and modifying variables	36
1.17.4	Importing rectangular data	36
1.18	Functions	37
1.18.1	Argument matching	38
1.18.2	Lazy evaluation	38
1.19	Environments and lexical scoping	39
1.20	Memory and copy-on-modify	40
1.21	Iteration: vectorization, loops, and the apply family	41
1.21.1	<code>apply()</code>	42
1.21.2	Why <code>apply()</code> can be dangerous for data frames	44
1.21.3	<code>lapply()</code>	45
1.21.4	<code>sapply()</code> and <code>vapply()</code>	46
1.21.5	<code>tapply()</code>	46
1.21.6	<code>Map()</code> and <code>mapply()</code>	46
1.21.7	Choosing an iteration tool	47
1.22	Packages and help	47
1.23	The tidyverse	48
1.23.1	Data frames and tibbles	48
1.23.2	Pipe operators	49
1.23.3	Core <code>dplyr</code> verbs	50
1.23.4	Applying functions across several columns	52
1.24	Base R and tidyverse are complementary	52
1.25	Numerical computation and finite precision	53
1.26	Randomness and reproducibility	54
1.27	Debugging and error messages	55
1.28	A computational workflow for this course	56
1.29	In-class exercises	57
1.30	Takeaways	59
1.31	Optional challenge	59
2	Intro to Computational Statistics	61
2.1	Theory versus Computation	62
2.2	Matrix Inversion	63
2.2.1	Multi-collinearity	65
2.2.2	Trade-off	67
3	Optimization	72
3.1	Why Numerical Optimization?	72
3.2	When There Is No Closed-Form Solution	75

3.3	Two Families of Optimization Methods	78
3.3.1	Deterministic Methods	78
3.3.2	Metaheuristic Methods	79
3.4	Local and Global Optima	79
3.5	Deterministic Numerical Methods	83
3.5.1	Root Finding	83
3.5.2	Bisection Method	86
3.5.3	Newton-Raphson Method	89
3.5.4	Visualizing Newton's Method	92
3.5.5	Secant Method	94
3.6	Convergence	96
3.6.1	Comparing Convergence in R	97
3.7	From One Dimension to Multiple Dimensions	99
3.8	Gradient Descent	101
3.8.1	Visualizing the Optimization Path	103
3.8.2	Step Size Matters	105
3.9	Multivariate Newton's Method	106
3.10	Quasi-Newton Methods	107
3.11	Scaling and Conditioning	110
3.12	Metaheuristic Methods	112
3.13	Simulated Annealing	113
3.13.1	Simulated Annealing in R	115
3.14	Genetic Algorithm	118
3.14.1	Selection	119
3.14.2	Crossover	119
3.14.3	Mutation	119
3.15	Particle Swarm Optimization	119
3.15.1	Advantages	120
3.15.2	Limitations	120
3.16	Deterministic versus Metaheuristic Methods	120
3.17	Computational Perspective	122
3.18	Questions to Think at Home	122
3.19	Takeaways	123
3.20	Optimal topic: Real-World Applications	124
4	Generating Random Variable	135
4.1	Overview	135
4.2	Inverse Transformation Method	138
4.2.1	Continuous Distributions	139
4.2.2	Discrete Distributions	145
4.3	Acceptance-Rejection Method	148
4.4	Using Probability Distribution Theory	152

4.5	Sums and Mixtures	158
4.5.1	Sums and Convolution	158
4.5.2	Mixture Distributions	159
4.6	Generating Multivariate Random Variables	163
4.6.1	Multivariate Normal Distribution	163
4.6.2	Multivariate Normal Mixtures	168
4.6.3	Uniform Directions and Points in a Ball	170
4.7	Checking a Generator	172
4.8	Optional: Real-World Applications	177
	Exercises	185
	References and Further Reading	186

I Assignment 187

Assignment 1 188

Question 1: R Programming and Statistical Computing [10]	188
1a)	188
1b)	189
1c)	189
1d)	189
1e)	189
1f)	190
1g)	190
1h)	190
1i)	190
1j)	191
Question 2: Computational Results Using the Iris Data [10]	191
2a) [2]	191
2b) [2]	191
2c) [2]	191
2d) [2]	192
2e) [2]	192
Question 3: R Coding and Visualization [6]	192
3a) [3]	192
3b) [3]	193
Question 4: Numerical Linear Algebra [4]	193
4a) [2]	193
4b) [2]	194

Assignment 2 195

Question 1: Computational Approaches [8]	196
1a)	196

1b)	196
1c)	196
1d)	196
1e)	197
1f)	197
1g)	197
1h)	197
Question 2: Optimization [8]	198
2a)	198
2b)	198
2c)	198
2d)	198
2e)	199
2f)	199
2g)	199
2h)	199
Question 3: True or False [6]	200
3a)	200
3b)	200
3c)	200
3d)	200
3e)	200
3f)	200
Question 4: Explain What the Results Mean [8]	201
4a) Rank deficiency is not missing data [4]	201
4b) A converged result still needs interpretation [4]	202

Preface

Description

Topics included are optimization, numerical integration, bootstrapping, cross-validation and Jackknife, density estimation, smoothing, and use of the statistical computer package of S-plus/R.

Prerequisites

MATH 4752/6752 – Mathematical Statistics II, and the ability to program in a high-level language.

Instructor

[Chi-Kuang Yeh](#), I am an Assistant Professor in the Department of Mathematics and Statistics, [Georgia State University](#).

- Office: Suite 1407, 25 Park Place.
- Email: cych@gsu.edu.

Office Hour

Wednesday 9:00 - 12:00 or by appointment

Grade Distribution

- Assignments: 40%
- Exam: 30%
- Project: 30%
- Attendance: *7%

Assignment

- ☒ [Assignment 1](#): September 11, 2026
- ☐ [Assignment 2](#): October 2, 2026

Midterm

- ☐ Exam 1: October 7, 2026
- ☐ Exam 2: November 16, 2026

Project

- ☐ Report: TBA
- ☐ Presentation: TBA

Topics and Corresponding Lectures

Those chapters are based on the lecture notes. This part will be updated frequently.

Status	Chapter	Topic	Lecture
	Ch. 1	R Programming	1–3
	Ch. 2	Intro to Computational Statistics	4–5
	Ch. 3	Optimization	6–
	Ch. 4	Generating Random Variable	TBA

Recommended Textbooks

- Givens, G.H. and Hoeting, J.A. (2012). [Computational Statistics](#). Wiley, New York.
- Rizzo, M.L. (2007) [Statistical Computing with R](#). CRC Press, Boca Raton.
- Hothorn, T. and Everitt, B.S. (2006). [A Handbook of Statistical Analyses Using R](#). CRC Press, Boca Raton.

Side Readings

- Wickham, H., Çetinkaya-Rundel, M. and Grolemund, G. (2023). *R for Data Science*. O'Reilly.

1 Introduction to R

Learning objectives

By the end of this chapter, you should be able to:

1. Describe how R reads, evaluates, and prints an expression.
2. Distinguish an object from the name bound to that object.
3. Identify the major data types and data structures in R.
4. Predict how vectorization, coercion, recycling, and missing values affect a computation.
5. Create, inspect, index, and modify vectors, matrices, arrays, lists, and data frames.
6. Write and use simple R functions.
7. Choose appropriately among vectorization, loops, and the `apply()` family.
8. Use the tidyverse pipe and core `dplyr` verbs to express a data-analysis pipeline.
9. Write computations that are reproducible, readable, and numerically responsible.

1.1 Why R?

Modern statistical analyses routinely involve hundreds, thousands, or millions of observations. The calculations are too numerous, and often too complex, to perform reliably by hand. We therefore need software that can manage data, implement statistical methods, automate repeated computations, and document the complete analysis.

R is both a high-level programming language and an environment for statistical computing and graphics. It is especially useful for statistics because it is:

- **free and open source**, so students and researchers can use the same tools;
- **designed for data analysis**, with vectors, missing values, statistical models, and graphics built into the language;
- **extensible**, with a large ecosystem of packages developed by researchers and software developers;
- **reproducible**, because an analysis can be saved as code and rerun from beginning to end;

- **interoperable**, with interfaces to databases and languages such as C++, Python, and Julia.

R is not a menu-driven statistics program. Learning R means learning to express an analysis as a sequence of precise and reproducible computations.

i R, RStudio, and Quarto are different tools

R performs the computation. **RStudio** is an integrated development environment that helps us write and run code. **Quarto** combines text, code, results, figures, equations, and references into reproducible HTML, PDF, Word, or presentation output.

1.2 The working environment

There are different integrated development environment to use R, some of them are Rstudio, VS Code and Positron.

Rstudio

RStudio is an integrated development environment (IDE) designed specifically for working with the R programming language. It provides a user-friendly interface that includes a source editor, console, environment pane, and tools for plotting, debugging, version control, and package management. RStudio supports both R and Python and is widely used for data analysis, statistical modeling, and reproducible research. It also integrates seamlessly with tools like R Markdown, Shiny, and Quarto, making it popular among data scientists, statisticians, and educators.

Visual Studio Code (VS Code)

VS Code is a versatile code editor that supports multiple programming languages, including R. With the R extension for VS Code, users can write and execute R code, access R's console, and utilize features like syntax highlighting, code completion, and debugging. While not as specialized as RStudio for R development, VS Code offers a lightweight alternative with extensive customization options and support for various programming tasks.

Positron

Positron IDE is the next-generation integrated development environment developed by Posit, the company behind RStudio. Designed to be a modern, extensible, and language-agnostic IDE, Positron builds on the strengths of RStudio while supporting a broader range of languages and workflows, including R, Python, and Quarto.

1.2.1 RStudio

RStudio is an integrated development environment (IDE) for R, Python, and reproducible publishing. Its default layout contains four panes:

Pane	Main purpose
Source Console	Write and edit scripts and Quarto documents. Send expressions directly to R and view immediate results.
Environment/History	Inspect objects and review previous commands.
Files/Plots/Packages/Help/Viewer	Navigate files, inspect graphics, manage packages, read documentation, and preview output.

Use the console for short experiments. Any computation that matters should be placed in an R script or Quarto document so that it can be reproduced.

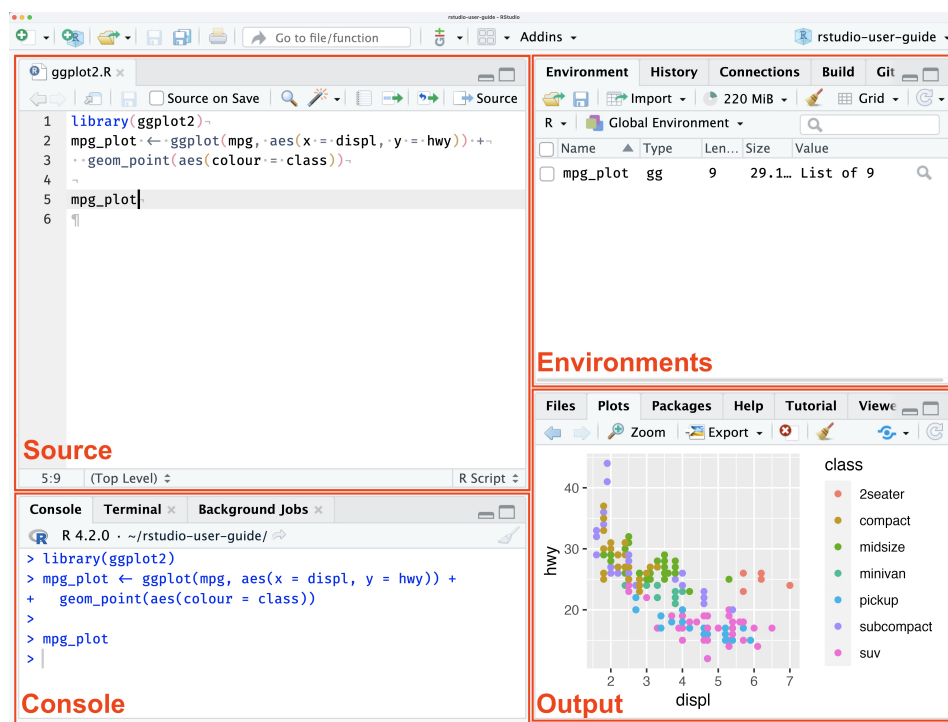


Figure 1.1: RStudio layout borrow from the Posit website.

1.2.2 R scripts and Quarto documents

An R script is a plain-text file with the extension `.R`. It contains R expressions that can be executed one line, one selection, or one file at a time.

A Quarto document has the extension `.qmd`. It combines:

- a YAML header describing the document;
- prose written in Markdown;
- executable code chunks;
- tables, equations, figures, citations, and cross-references.

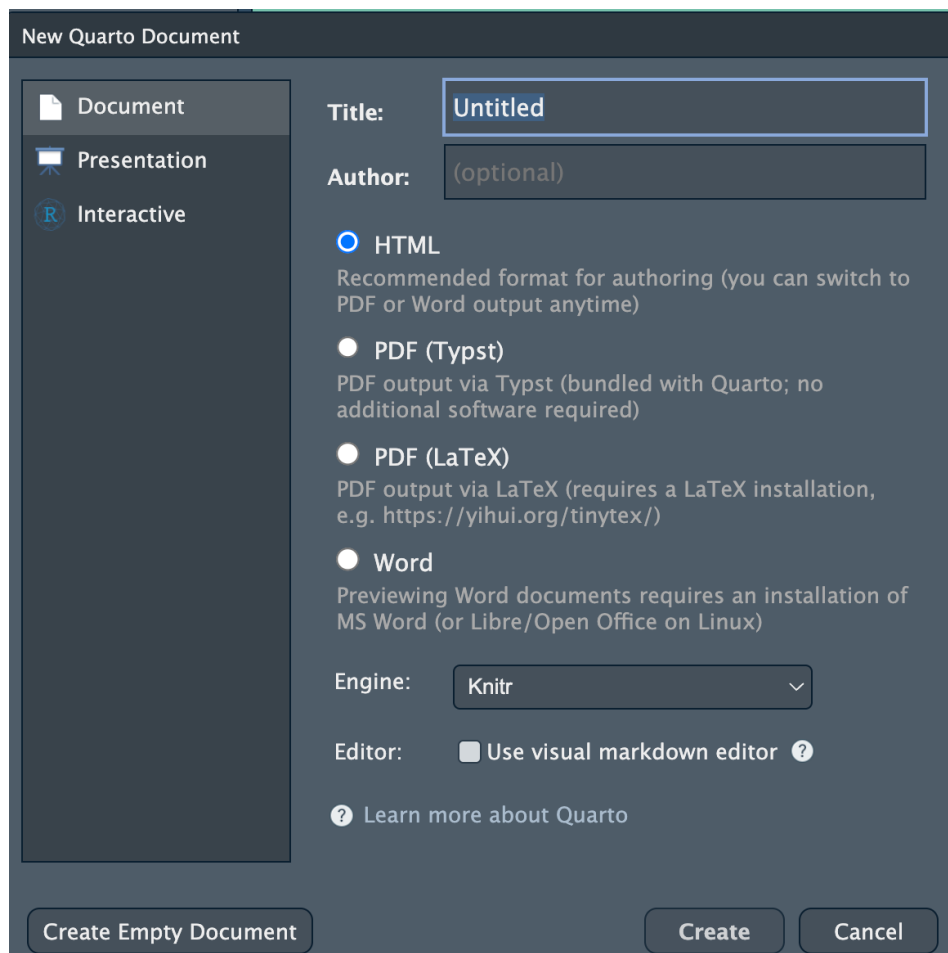


Figure 1.2: Quarto.

In Figure 1.3, the red box is the yaml, the yellow box is the executable code chunk and the green box is a figure.

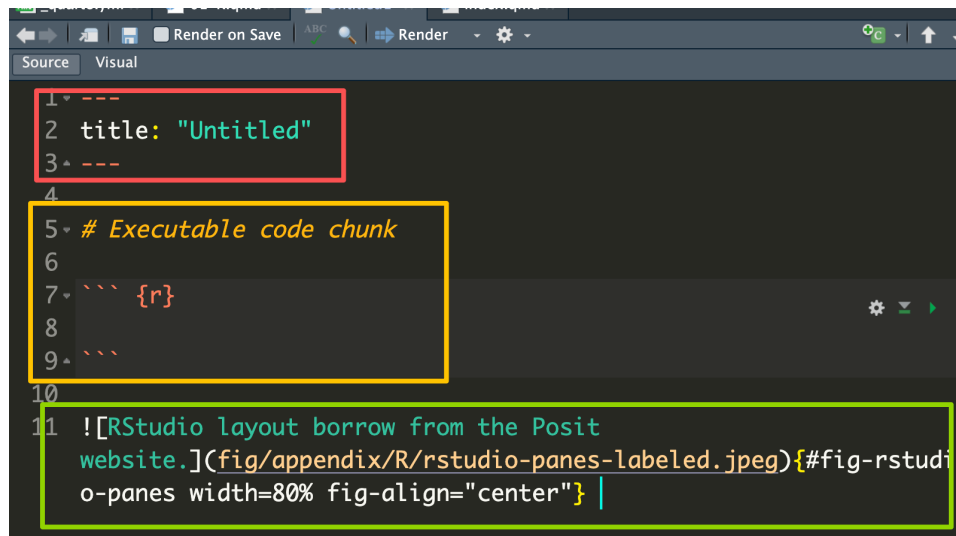


Figure 1.3: Quarto 2.

For example, an executable R chunk is written as

```
```{r}
x <- rnorm(100)
mean(x)
```
```

```
[1] 0.1211347
```

Rendering the document executes the enabled chunks in order and places their results into the final output.

1.2.3 A project-oriented workflow

Create one RStudio Project for the course or for each substantial analysis. Use project-relative paths such as `data/survey.csv` rather than hard-coded paths tied to one computer.

A simple project might contain:

```
stat8670-project/
  data/           # original and processed data
  R/              # reusable functions
  figures/        # saved figures
```

```
notes/          # Quarto lecture or analysis files
stat8670-project.Rproj
```

Avoid using `setwd()` inside a reproducible document. When a project is opened, its root provides a stable reference point for relative paths.

1.3 R as a computing environment

At the console, R follows a **read, evaluate, print loop** (REPL):

1. **Read:** parse the text entered by the user.
2. **Evaluate:** compute the value of the expression.
3. **Print:** display the result when appropriate.
4. **Loop:** wait for the next expression.

```
2 + 3 * 4
```

```
[1] 14
```

R respects operator precedence, so multiplication is performed before addition.

```
(2 + 3) * 4
```

```
[1] 20
```

Assignment usually returns its value invisibly:

```
x <- 2 + 3 * 4
x
```

```
[1] 14
```

A useful mental model

Most R code can be understood as **functions operating on objects**. Even operators such as `+` are functions.

```
`+`(2, 3)
```

```
[1] 5
```

1.4 Objects, data, and functions

Two fundamental kinds of objects that we will work with are:

1. **Data objects**
2. **Functions**

Data objects include numbers, vectors, matrices, arrays, lists, data frames, and fitted models.

A **function** is a set of instructions that receives input, performs a computation, and returns an output. Functions can be built into R, provided by a package, or written by the user.

```
mean(c(1, 2, 3, 4))
```

```
[1] 2.5
```

Here, `c(1, 2, 3, 4)` is a data object and `mean()` is a function.

1.5 Expressions, objects, and names

R computes with **objects**. A **name** is a symbol bound to an object.

```
sample_size <- 100
another_name <- sample_size

sample_size
```

```
[1] 100
```

```
another_name
```

```
[1] 100
```


The assignment operator `<-` creates or changes a binding.

Names are case-sensitive:

```
n <- 10
N <- 25

c(n = n, N = N)
```

```
  n  N
10 25
```

A useful naming convention is to separate words using `_`:

```
sample_mean <- 10
number_students <- 30
```

Use `ls()` to list names in the current environment and `rm()` to remove a binding.

```
temporary_result <- 42
"temporary_result" %in% ls()
```

```
[1] TRUE
```

```
rm(temporary_result)

"temporary_result" %in% ls()
```

```
[1] FALSE
```

1.6 Data types

The fundamental data structure in R is the **vector**. An atomic vector contains elements of one common type.

Common atomic types include:

| Type | Example | Typical use |
|-----------|----------------|---------------------------|
| logical | TRUE, FALSE | conditions and indicators |
| integer | 2L | counts and indices |
| double | 2, 3.14 | numerical computation |
| character | "R" | text and labels |
| complex | 1 + 2i | complex arithmetic |
| raw | charToRaw("R") | bytes |

```
logical_vector <- c(TRUE, FALSE, TRUE)
integer_vector <- c(1L, 2L, 3L)
double_vector <- c(1, 2, 3)
character_vector <- c("one", "two", "three")

typeof(logical_vector)
```

```
[1] "logical"
```

```
typeof(integer_vector)
```

```
[1] "integer"
```

```
typeof(double_vector)
```

```
[1] "double"
```

```
typeof(character_vector)
```

```
[1] "character"
```

1.6.1 Changing data types

Data types can be explicitly converted, but conversion may result in information loss.

```
x <- pi
x
```

```
[1] 3.141593
```

```
x_int <- as.integer(x)
x_int
```

```
[1] 3
```

Here, converting π to an integer removes its decimal component.

Common conversion functions include:

- `as.integer()`
- `as.numeric()`
- `as.character()`
- `as.logical()`
- `as.Date()`
- `as.factor()`
- `as.list()`
- `as.matrix()`
- `as.data.frame()`
- `as.vector()`
- `as.complex()`

 Type conversion can lose information

Always inspect the result after converting an object from one type to another.

1.7 Operators

A **unary operator** operates on one object:

```
-x
!x
```

A **binary operator** operates on two objects:

```
x + y
x - y
x * y
x / y
```

Comparison operators

Comparison operators return logical values:

```
x == y
x != y
x < y
x > y
x <= y
x >= y
```

Logical operators

Logical operators combine logical values:

```
x & y
x | y
!x
```

The operators `&` and `|` operate element by element.

```
x <- c(TRUE, FALSE, FALSE)
y <- c(TRUE, TRUE, FALSE)

x | y
```

```
[1] TRUE TRUE FALSE
```

```
x & y
```

```
[1] TRUE FALSE FALSE
```

By contrast, `&&` and `||` are intended for a single logical condition and are commonly used inside `if` statements.

1.8 Vectorization

Many R functions and operators act on entire vectors.

```
x <- c(1, 2, 3, 4)
```

```
x^2
```

```
[1] 1 4 9 16
```

```
sqrt(x)
```

```
[1] 1.000000 1.414214 1.732051 2.000000
```

```
sum(x)
```

```
[1] 10
```

```
mean(x)
```

```
[1] 2.5
```

Vectorized code often expresses statistical calculations more directly than element-by-element code.

1.9 Recycling

Arithmetic between vectors is usually element-wise.

```
x <- c(10, 20, 30)
```

```
y <- c(1, 2, 3)
```

```
x + y
```

```
[1] 11 22 33
```

If one vector is shorter, R may **recycle** its values.

```
c(10, 20, 30, 40) + c(1, 2)
```

```
[1] 11 22 31 42
```

When the shorter length does not divide the longer length exactly, R produces a warning.

```
c(10, 20, 30, 40) + c(1, 2, 3)
```

Warning in `c(10, 20, 30, 40) + c(1, 2, 3)`: longer object length is not a multiple of shorter object length

```
[1] 11 22 33 41
```

1.10 Coercion

An atomic vector must have one type. Combining different types may cause R to convert values to a common type.

```
mixed_numeric <- c(TRUE, 2L, 3.5)
mixed_character <- c(TRUE, 2L, 3.5, "four")

mixed_numeric
```

```
[1] 1.0 2.0 3.5
```

```
typeof(mixed_numeric)
```

```
[1] "double"
```

```
mixed_character
```

```
[1] "TRUE" "2"    "3.5"  "four"
```

```
typeof(mixed_character)
```

```
[1] "character"
```

A simplified coercion hierarchy is:

```
logical → integer → double → complex → character
```

Inspect unfamiliar objects using

```
typeof()  
class()  
length()  
str()
```

1.11 Missing and special values

R uses NA to represent a missing value.

```
day <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")  
weather <- c("Raining", "Sunny", NA, "Windy", "Snowing")  
  
weather_data <- data.frame(day, weather)  
weather_data
```

```
      day weather  
1  Monday Raining  
2  Tuesday  Sunny  
3 Wednesday  <NA>  
4  Thursday  Windy  
5   Friday Snowing
```

Missing values usually propagate through calculations:

```
scores <- c(88, NA, 91, 84)  
  
mean(scores)
```

```
[1] NA
```

```
mean(scores, na.rm = TRUE)
```

```
[1] 87.66667
```

```
is.na(scores)
```

```
[1] FALSE TRUE FALSE FALSE
```

Do not use

```
x == NA
```

to detect missing values. Use

```
is.na(x)
```

instead.

Other special numerical values include

```
c(1 / 0, -1 / 0, 0 / 0)
```

```
[1] Inf -Inf NaN
```

which produce `Inf`, `-Inf`, and `NaN`.

1.12 Indexing and subsetting

Indexing allows us to access or modify parts of an object.

! R indexing begins at 1

Unlike languages such as Python and C++, R uses **1-based indexing**. The first element is indexed by 1, not 0.

For a vector:

```
x <- c(first = 10, second = 20, third = 30, fourth = 40)
```

```
x[1]
```

```
first  
10
```



```
x[c(1, 3)]
```

```
first third  
  10    30
```

```
x[-2]
```

```
first  third fourth  
   10    30    40
```

```
x[x >= 30]
```

```
third fourth  
   30    40
```

```
x[c("second", "fourth")]
```

```
second fourth  
   20    40
```

Positive integers select positions, negative integers exclude positions, logical values filter positions, and character values select named elements.

 A common surprise

Prefer `seq_along(x)` to `1:length(x)` when iterating over the positions of an object.

```
empty <- numeric(0)
```

```
1:length(empty)
```

```
[1] 1 0
```

```
seq_along(empty)
```

```
integer(0)
```

1.13 Naming elements

Names can be attached to vector elements:

```
temp <- c(20, 30, 27, 31, 45)

names(temp) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")

temp
```

| Mon | Tues | Wed | Thurs | Fri |
|-----|------|-----|-------|-----|
| 20 | 30 | 27 | 31 | 45 |

```
temp["Wed"]
```

```
Wed
27
```

A vector does not have rows, so this is inappropriate:

```
rownames(temp) <- "Day1"
```

A matrix can have row and column names:

```
temp_mat <- matrix(c(20, 30, 27, 31, 45),
                    nrow = 1,
                    ncol = 5)

colnames(temp_mat) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")
rownames(temp_mat) <- "Day1"

temp_mat
```

| | Mon | Tues | Wed | Thurs | Fri |
|------|-----|------|-----|-------|-----|
| Day1 | 20 | 30 | 27 | 31 | 45 |

1.14 Arrays and matrices

An **array** is a multidimensional data structure.

```
array_1d <- array(1:10, dim = 10)
array_1d
```

```
[1]  1  2  3  4  5  6  7  8  9 10
```

```
array_2d <- array(1:12, dim = c(4, 3))
array_2d
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
array_3d <- array(1:24, dim = c(4, 3, 2))
array_3d
```

```
, , 1
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
, , 2
```

```
      [,1] [,2] [,3]
[1,]    13    17    21
[2,]    14    18    22
[3,]    15    19    23
[4,]    16    20    24
```

A **matrix** is a two-dimensional array:

```
my_matrix <- matrix(1:12, nrow = 4, ncol = 3)
my_matrix
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
is.matrix(array_2d)
```

```
[1] TRUE
```

```
is.matrix(my_matrix)
```

```
[1] TRUE
```

```
is.array(array_2d)
```

```
[1] TRUE
```

```
is.array(my_matrix)
```

```
[1] TRUE
```

A matrix is an atomic vector with a `dim` attribute, so all entries must share one common type.

```
A <- matrix(1:6, nrow = 2, ncol = 3)
attributes(A)
```

```
$dim
[1] 2 3
```

```
A[2, 3]
```

```
[1] 6
```

Matrix multiplication uses `%*%`:

```
B <- matrix(c(1, 0, 0, 1, 1, 1), nrow = 3)
```

```
A %*% B
```

```
      [,1] [,2]  
[1,]     1     9  
[2,]     2    12
```

while `*` performs element-wise multiplication.

1.15 Lists and key-value pairs

A list can contain objects of different types and lengths.

```
fit_summary <- list(  
  method = "least squares",  
  coefficients = c(intercept = 1.2, slope = 0.8),  
  converged = TRUE  
)  
  
str(fit_summary)
```

```
List of 3  
 $ method      : chr "least squares"  
 $ coefficients: Named num [1:2] 1.2 0.8  
 ..- attr(*, "names")= chr [1:2] "intercept" "slope"  
 $ converged    : logi TRUE
```

Named list elements naturally form **key-value pairs**.

```
key1 <- "Tues"
value1 <- 32

key2 <- "Wed"
value2 <- 28

list_temp <- list()

list_temp[[key1]] <- value1
list_temp[[key2]] <- value2

list_temp
```

```
$Tues
[1] 32
```

```
$Wed
[1] 28
```

Values can then be obtained from their keys:

```
list_temp[["Tues"]]
```

```
[1] 32
```

```
list_temp$Tues
```

```
[1] 32
```

For lists, note the important distinction:

```
fit_summary["coefficients"]
```

```
$coefficients
intercept      slope
      1.2        0.8
```

```
fit_summary[["coefficients"]]
```

```
intercept      slope  
      1.2      0.8
```

[returns a sublist, whereas [[extracts the element itself.

1.16 Factors

A factor represents categorical data using integer codes and a set of levels.

```
group <- factor(c("control", "treatment", "control"))  
group
```

```
[1] control  treatment control  
Levels: control treatment
```

```
unclass(group)
```

```
[1] 1 2 1  
attr("levels")  
[1] "control" "treatment"
```

```
levels(group)
```

```
[1] "control" "treatment"
```

The order of factor levels matters in statistical models, so define it deliberately when necessary.

1.17 Data frames

A **data frame** is a two-dimensional rectangular data structure.

Conceptually, a data frame is a named list of equal-length vectors:

- each row represents an observational unit;
- each column represents a variable;
- columns may have different data types.

```
students <- data.frame(  
  id = 1:6,  
  name = c("Alice", "Bob", "Chen", "Divya", "Elena", "Farah"),  
  program = c("MS", "PhD", "MS", "PhD", "MS", "PhD"),  
  score = c(90, 85, NA, 94, 88, 91),  
  completed = c(TRUE, TRUE, FALSE, TRUE, TRUE, TRUE)  
)  
  
students
```

| | id | name | program | score | completed |
|---|----|-------|---------|-------|-----------|
| 1 | 1 | Alice | MS | 90 | TRUE |
| 2 | 2 | Bob | PhD | 85 | TRUE |
| 3 | 3 | Chen | MS | NA | FALSE |
| 4 | 4 | Divya | PhD | 94 | TRUE |
| 5 | 5 | Elena | MS | 88 | TRUE |
| 6 | 6 | Farah | PhD | 91 | TRUE |

Another familiar built-in data frame is:

```
iris <- datasets::iris  
head(iris)
```

| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 3 | 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 4 | 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 5 | 5.0 | 3.6 | 1.4 | 0.2 | setosa |
| 6 | 5.4 | 3.9 | 1.7 | 0.4 | setosa |

1.17.1 Inspecting a data frame

Always inspect newly imported data before analyzing them.

```
class(students)
```

```
[1] "data.frame"
```

```
dim(students)
```

```
[1] 6 5
```

```
nrow(students)
```

```
[1] 6
```

```
ncol(students)
```

```
[1] 5
```

```
names(students)
```

```
[1] "id"      "name"    "program" "score"   "completed"
```

```
str(students)
```

```
'data.frame':  6 obs. of  5 variables:
 $ id      : int  1 2 3 4 5 6
 $ name    : chr  "Alice" "Bob" "Chen" "Divya" ...
 $ program : chr  "MS" "PhD" "MS" "PhD" ...
 $ score   : num  90 85 NA 94 88 91
 $ completed: logi  TRUE TRUE FALSE TRUE TRUE TRUE
```

```
summary(students)
```

| | id | | name | | program | | score | | completed |
|---------|-------|-----------|------|-----------|---------|---------|-------|-------|-----------|
| Min. | :1.00 | Length | :6 | Length | :6 | Min. | :85.0 | Mode | :logical |
| 1st Qu. | :2.25 | N.unique | :6 | N.unique | :2 | 1st Qu. | :88.0 | FALSE | :1 |
| Median | :3.50 | N.blank | :0 | N.blank | :0 | Median | :90.0 | TRUE | :5 |
| Mean | :3.50 | Min.nchar | :3 | Min.nchar | :2 | Mean | :89.6 | | |
| 3rd Qu. | :4.75 | Max.nchar | :5 | Max.nchar | :3 | 3rd Qu. | :91.0 | | |
| Max. | :6.00 | | | | | Max. | :94.0 | | |
| | | | | | | NAs | :1 | | |

```
head(students, 3)
```

```

  id name program score completed
1  1 Alice      MS    90        TRUE
2  2  Bob      PhD    85        TRUE
3  3  Chen      MS    NA        FALSE

```

`str()` is especially useful because it gives a compact summary of the object's structure and types.

1.17.2 Subsetting a data frame

Use

```
data[rows, columns]
```

for two-dimensional indexing:

```
students[1, ]
```

```

  id name program score completed
1  1 Alice      MS    90        TRUE

```

```
students[, c("name", "score")]
```

```

  name score
1 Alice    90
2  Bob    85
3  Chen    NA
4 Divya    94
5 Elena    88
6 Farah    91

```

```
students[students$program == "PhD", ]
```

| | id | name | program | score | completed |
|---|----|-------|---------|-------|-----------|
| 2 | 2 | Bob | PhD | 85 | TRUE |
| 4 | 4 | Divya | PhD | 94 | TRUE |
| 6 | 6 | Farah | PhD | 91 | TRUE |

```
students[1:3, c("name", "score")]
```

| | name | score |
|---|-------|-------|
| 1 | Alice | 90 |
| 2 | Bob | 85 |
| 3 | Chen | NA |

```
students[2, "score"]
```

```
[1] 85
```

Three commonly used ways to extract a column are:

```
students["score"]
```

| | score |
|---|-------|
| 1 | 90 |
| 2 | 85 |
| 3 | NA |
| 4 | 94 |
| 5 | 88 |
| 6 | 91 |

```
students[["score"]]
```

```
[1] 90 85 NA 94 88 91
```

```
students$score
```

```
[1] 90 85 NA 94 88 91
```

The first returns a one-column data frame. The latter two return the underlying vector.

1.17.3 Adding and modifying variables

```
students$passed <- students$score >= 70

students$centered_score <-
  students$score - mean(students$score, na.rm = TRUE)

students
```

| | id | name | program | score | completed | passed | centered_score |
|---|----|-------|---------|-------|-----------|--------|----------------|
| 1 | 1 | Alice | MS | 90 | TRUE | TRUE | 0.4 |
| 2 | 2 | Bob | PhD | 85 | TRUE | TRUE | -4.6 |
| 3 | 3 | Chen | MS | NA | FALSE | NA | NA |
| 4 | 4 | Divya | PhD | 94 | TRUE | TRUE | 4.4 |
| 5 | 5 | Elena | MS | 88 | TRUE | TRUE | -1.6 |
| 6 | 6 | Farah | PhD | 91 | TRUE | TRUE | 1.4 |

Missingness propagates naturally. If a score is unknown, whether the student passed is also unknown.

1.17.4 Importing rectangular data

```
survey <- read.csv(
  "data/survey.csv",
  na.strings = c("", "NA", ".")
)

measurements <- read.table(
  "data/measurements.txt",
  header = TRUE,
  sep = "\t"
)
```

After importing, check:

```
dim(survey)
names(survey)
str(survey)
head(survey)
colSums(is.na(survey))
```

Data import is part of the analysis

Import settings such as delimiters, missing-value codes, decimal marks, encodings, and inferred variable types can change the data that R sees and therefore affect the final analysis.

Do not automatically treat a data frame as a matrix

`apply()` first converts a data frame to a matrix. A mixed-type data frame can therefore be converted entirely to character values. Use `lapply()`, `vapply()`, or `dplyr::across()` for column-wise operations when types differ.

1.18 Functions

A function packages a computation so that it can be reused.

```
standardize <- function(x, remove_na = FALSE) {  
  center <- mean(x, na.rm = remove_na)  
  spread <- sd(x, na.rm = remove_na)  
  
  (x - center) / spread  
}  
  
standardize(c(2, 4, 6, 8))
```

```
[1] -1.1618950 -0.3872983  0.3872983  1.1618950
```

A function has three main components:

- **formals:** the arguments;
- **body:** the expressions evaluated;
- **environment:** where the function was created.

```
formals(standardize)
```

```
$x
```

```
$remove_na  
[1] FALSE
```

```
body(standardize)
```

```
{  
  center <- mean(x, na.rm = remove_na)  
  spread <- sd(x, na.rm = remove_na)  
  (x - center)/spread  
}
```

```
environment(standardize)
```

```
<environment: R_GlobalEnv>
```

When a function is called, R creates a temporary environment, matches the arguments, evaluates the body, and returns the value of the final expression unless `return()` is called earlier.

1.18.1 Argument matching

Arguments can be matched by name or position.

```
round(x = 3.14159, digits = 3)
```

```
[1] 3.142
```

```
round(3.14159, 3)
```

```
[1] 3.142
```

Named arguments are often clearer in code intended to be read by others.

1.18.2 Lazy evaluation

R evaluates function arguments only when they are needed.

```
choose_one <- function(x, y) {  
  x  
}  
  
choose_one(  
  10,  
  stop("This argument is never evaluated")  
)
```

```
[1] 10
```

This behavior is called **lazy evaluation**.

1.19 Environments and lexical scoping

An environment stores name-object bindings and has a parent environment.

R uses **lexical scoping**: functions look for nonlocal names in the environment where they were defined.

```
make_power <- function(power) {  
  function(x) {  
    x^power  
  }  
}  
  
square <- make_power(2)  
cube <- make_power(3)  
  
square(5)
```

```
[1] 25
```

```
cube(5)
```

```
[1] 125
```

A function together with its defining environment is called a **closure**.

Local assignment affects the current function environment:

```
x <- 100

change_locally <- function() {
  x <- 5
  x
}

change_locally()
```

```
[1] 5
```

```
x
```

```
[1] 100
```

1.20 Memory and copy-on-modify

R behaves as if objects are copied when modified.

```
x <- c(10, 20, 30)
y <- x

y[1] <- 999

x
```

```
[1] 10 20 30
```

```
y
```

```
[1] 999 20 30
```

Changing `y` does not change `x`.

Repeatedly expanding an object can require many allocations:


```
result <- numeric(0)

for (i in seq_len(100000)) {
  result <- c(result, i^2)
}
```

Instead, preallocate the required memory:

```
n <- 100000

result <- numeric(n)

for (i in seq_len(n)) {
  result[i] <- i^2
}

head(result)
```

```
[1] 1 4 9 16 25 36
```

1.21 Iteration: vectorization, loops, and the apply family

An iterative computation can usually be expressed in several ways.

```
x <- c(2, 4, 6, 8)

# Vectorized
x^2
```

```
[1] 4 16 36 64
```

```
# Preallocated loop
squares <- numeric(length(x))

for (i in seq_along(x)) {
  squares[i] <- x[i]^2
}

squares
```

```
[1]  4 16 36 64
```

```
# Apply a function
vapply(
  x,
  function(value) value^2,
  numeric(1)
)
```

```
[1]  4 16 36 64
```

Use vectorization when an existing vectorized function directly expresses the computation. Use a loop when the calculation has evolving state or complex indexing. Use an apply-family function when the same operation is independently applied to several pieces of an object.

! The apply family is not magic vectorization

`apply()` and `lapply()` still perform iteration internally. They are not automatically faster than a well-written preallocated `for` loop. Their main advantage is often clarity and conciseness.

1.21.1 `apply()`

`apply(X, MARGIN, FUN, ...)` applies a function over dimensions of a matrix or array.

- `MARGIN = 1`: rows
- `MARGIN = 2`: columns

```
X <- matrix(1:12, nrow = 3, byrow = TRUE)

colnames(X) <- c("x1", "x2", "x3", "x4")

X
```

```
      x1 x2 x3 x4
[1,]  1  2  3  4
[2,]  5  6  7  8
[3,]  9 10 11 12
```

```
apply(X, MARGIN = 1, FUN = mean)
```

```
[1] 2.5 6.5 10.5
```

```
apply(X, MARGIN = 2, FUN = sd)
```

```
x1 x2 x3 x4  
4 4 4 4
```

Additional arguments are passed through ...:

```
X_missing <- X  
X_missing[2, 3] <- NA  
  
apply(X_missing, 2, mean, na.rm = TRUE)
```

```
x1 x2 x3 x4  
5 6 7 8
```

For common operations, specialized functions are preferable:

```
rowMeans(X)
```

```
[1] 2.5 6.5 10.5
```

```
colMeans(X)
```

```
x1 x2 x3 x4  
5 6 7 8
```

```
rowSums(X)
```

```
[1] 10 26 42
```

```
colSums(X)
```

```
x1 x2 x3 x4  
15 18 21 24
```

1.21.2 Why apply() can be dangerous for data frames

```
mixed_df <- data.frame(  
  id = c("A", "B", "C"),  
  x = c(10, 20, 30),  
  y = c(2, 4, 6)  
)  
  
as.matrix(mixed_df)
```

```
      id  x  y  
[1,] "A" 10 2  
[2,] "B" 20 4  
[3,] "C" 30 6
```

```
typeof(as.matrix(mixed_df))
```

```
[1] "character"
```

Because one column is character, the entire matrix becomes character.

A safer approach is:

```
numeric_columns <- vapply(  
  mixed_df,  
  is.numeric,  
  logical(1)  
)  
  
vapply(  
  mixed_df[numeric_columns],  
  mean,  
  numeric(1)  
)
```

```
  x  y  
20 4
```

1.21.3 lapply()

`lapply()` applies a function to each element and always returns a list.

```
lapply(students, class)
```

```
$id
[1] "integer"

$name
[1] "character"

$program
[1] "character"

$score
[1] "numeric"

$completed
[1] "logical"

$passed
[1] "logical"

$centered_score
[1] "numeric"
```

```
lapply(
  students[c("score", "centered_score")],
  mean,
  na.rm = TRUE
)
```

```
$score
[1] 89.6

$centered_score
[1] 5.861978e-15
```

1.21.4 `sapply()` and `vapply()`

`sapply()` attempts to simplify the output.

```
sapply(students, class)
```

| | | | | |
|-----------|----------------|-------------|-----------|-----------|
| id | name | program | score | completed |
| "integer" | "character" | "character" | "numeric" | "logical" |
| passed | centered_score | | | |
| "logical" | "numeric" | | | |

`vapply()` requires the expected output type:

```
vapply(students, typeof, character(1))
```

| | | | | |
|-----------|----------------|-------------|----------|-----------|
| id | name | program | score | completed |
| "integer" | "character" | "character" | "double" | "logical" |
| passed | centered_score | | | |
| "logical" | "double" | | | |

This makes `vapply()` safer for reusable code.

1.21.5 `tapply()`

`tapply()` splits a vector into groups and applies a function within each group.

```
with(
  students,
  tapply(score, program, mean, na.rm = TRUE)
)
```

```
MS PhD
89  90
```

1.21.6 `Map()` and `mapply()`

These functions apply a function to corresponding elements of multiple inputs.

```
Map(
  function(center, spread) rnorm(3, center, spread),
  center = c(0, 10),
  spread = c(1, 2)
)
```

```
[[1]]
[1] 0.9444969 -0.3233628 -1.3212824
```

```
[[2]]
[1] 7.857089 11.570026 8.406937
```

1.21.7 Choosing an iteration tool

| Goal | Recommended starting point |
|---|--|
| Element-wise arithmetic | Vectorized function or operator |
| Standard matrix row/column summaries | <code>rowSums()</code> , <code>colSums()</code> , <code>rowMeans()</code> ,
<code>colMeans()</code> |
| General matrix row/column operation | <code>apply()</code> |
| Operation on list or data-frame columns | <code>lapply()</code> |
| Known scalar output type | <code>vapply()</code> |
| Grouped operation on one vector | <code>tapply()</code> |
| Data-frame columns in tidyverse | <code>dplyr::across()</code> |
| Complex state-dependent computation | Preallocated for loop |

1.22 Packages and help

Packages extend R with functions, data, documentation, and statistical methods. The historical package count can be found <https://www.datasciencemeta.com/rpackages>.

```
# CRAN
install.packages("packageName")

# Bioconductor
BiocManager::install("packageName")

# GitHub
remotes::install_github("username/repository")
```

Load an installed package:

```
library(packageName)
```

or call one function explicitly:

```
packageName::function_name()
```

Useful help tools include:

```
?mean  
help("mean")  
example(mean)  
args(mean)  
methods(mean)
```

1.23 The tidyverse

The **tidyverse** is a coordinated collection of R packages for data manipulation, visualization, importing, tidying, and programming.

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Important packages include:

```
library(dplyr)    # data manipulation  
library(ggplot2)  # data visualization  
library(readr)    # data import  
library(tibble)   # tidy data frames  
library(tidyr)    # data tidying  
library(purrr)    # functional programming
```

Base R and tidyverse tools are complementary rather than competing systems.

1.23.1 Data frames and tibbles

A **tibble** is a modern data frame.


```
students_tbl <- as_tibble(students)
```

```
students_tbl
```

```
# A tibble: 6 x 7
```

```
      id name  program score completed passed centered_score
  <int> <chr> <chr>    <dbl> <lgl>    <lgl>         <dbl>
1     1  Alice  MS        90  TRUE     TRUE         0.400
2     2   Bob  PhD        85  TRUE     TRUE        -4.60
3     3  Chen  MS         NA FALSE     NA           NA
4     4 Divya  PhD        94  TRUE     TRUE         4.40
5     5  Elena  MS        88  TRUE     TRUE        -1.60
6     6  Farah  PhD        91  TRUE     TRUE         1.40
```

```
class(students_tbl)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

Some important differences are:

| Operation | Base data frame | Tibble |
|---------------------------|----------------------------|------------------|
| Printing | May print many rows | Compact preview |
| <code>x[, "score"]</code> | May simplify | Remains a tibble |
| Partial matching | May occur in some contexts | More strict |
| Row names | Supported | Discouraged |

Both are still data frames:

```
is.data.frame(students_tbl)
```

```
[1] TRUE
```

1.23.2 Pipe operators

The native R pipe is

```
|>
```

and the commonly used magrittr/tidyverse pipe is

```
%>%
```

The pipe passes the result of one operation to the next function.

```
set.seed(777)

x <- rnorm(5)

# Without pipe
print(round(mean(x), 2))
```

```
[1] 0.37
```

```
# Native pipe
x |>
  mean() |>
  round(2) |>
  print()
```

```
[1] 0.37
```

The pipe makes the sequence of operations explicit.

A useful formatting rule is that `|>` should have spaces around it and normally appear at the end of a line.

1.23.3 Core dplyr verbs

| Verb | Purpose |
|--------------------------|-----------------------------------|
| <code>select()</code> | Choose or reorder columns |
| <code>filter()</code> | Retain rows satisfying conditions |
| <code>arrange()</code> | Sort observations |
| <code>mutate()</code> | Create or modify variables |
| <code>summarise()</code> | Compute summaries |

| Verb | Purpose |
|-------------------------|---------------|
| <code>group_by()</code> | Define groups |

Example:

```
students_tbl |>
  filter(completed, !is.na(score)) |>
  select(name, program, score) |>
  arrange(desc(score))
```

```
# A tibble: 5 x 3
  name  program score
<chr> <chr>   <dbl>
1 Divya PhD      94
2 Farah PhD      91
3 Alice MS       90
4 Elena MS       88
5 Bob   PhD       85
```

Create variables and summarize by group:

```
students_tbl |>
  mutate(
    passed = score >= 70,
    score_z =
      (score - mean(score, na.rm = TRUE)) /
      sd(score, na.rm = TRUE)
  ) |>
  group_by(program) |>
  summarise(
    n = n(),
    n_observed = sum(!is.na(score)),
    mean_score = mean(score, na.rm = TRUE),
    sd_score = sd(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 5
  program      n n_observed mean_score sd_score
<chr>   <int>     <int>     <dbl>   <dbl>
1 MS         3         2         89     1.41
2 PhD        3         3         90     4.58
```

1.23.4 Applying functions across several columns

Inside `mutate()` or `summarise()`, `across()` applies functions to selected columns.

```
students_tbl |>
  summarise(
    across(
      where(is.numeric),
      list(
        mean = ~ mean(.x, na.rm = TRUE),
        sd = ~ sd(.x, na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
    )
  )
```

A tibble: 1 x 6

| | id_mean | id_sd | score_mean | score_sd | centered_score_mean | centered_score_sd |
|---|---------|-------|------------|----------|---------------------|-------------------|
| | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| 1 | 3.5 | 1.87 | 89.6 | 3.36 | 5.86e-15 | 3.36 |

This provides a tidyverse alternative to column-wise `lapply()` or `vapply()`.

1.24 Base R and tidyverse are complementary

For example, the same grouped mean can be obtained using either style.

```
# Base R
aggregate(
  score ~ program,
  data = students,
  FUN = mean,
  na.rm = TRUE
)
```

| | program | score |
|---|---------|-------|
| 1 | MS | 89 |
| 2 | PhD | 90 |

```
# tidyverse
students_tbl |>
  group_by(program) |>
  summarise(
    mean_score = mean(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 2
  program mean_score
  <chr>      <dbl>
1 MS          89
2 PhD          90
```

The important questions are the same regardless of syntax:

- What object enters the function?
- What object does the function return?
- How are missing values handled?
- Are types and groups preserved?

1.25 Numerical computation and finite precision

Most real numbers cannot be represented exactly using binary floating-point arithmetic.

```
0.1 + 0.2
```

```
[1] 0.3
```

```
(0.1 + 0.2) == 0.3
```

```
[1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

For computed floating-point quantities, tolerance-based comparisons such as `all.equal()` are usually more appropriate than exact equality.

Numerical algorithms must also consider overflow, underflow, cancellation, conditioning, and convergence.

```
log(dbinom(500, size = 1000, prob = 0.01))
```

```
[1] -Inf
```

```
dbinom(  
  500,  
  size = 1000,  
  prob = 0.01,  
  log = TRUE  
)
```

```
[1] -1618.143
```

The second computation is numerically safer.

1.26 Randomness and reproducibility

Pseudo-random numbers are generated deterministically from an internal state.

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

The two results are identical.

Set the seed at a meaningful boundary, usually once near the beginning of a simulation, rather than repeatedly inside the simulation loop.

Record the information necessary to reproduce the analysis:

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.6.2
```

```
Matrix products: default
```

```
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/New_York
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
[5] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
[9] ggplot2_4.0.3   tidyverse_2.0.0
```

```
loaded via a namespace (and not attached):
```

```
[1] gtable_0.3.6      jsonlite_2.0.0    compiler_4.6.0    tidyselect_1.2.1
[5] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0     R6_2.6.1
[9] generics_0.1.4    knitr_1.51        pillar_1.11.1     RColorBrewer_1.1-
3
[13] tzdb_0.5.0        rlang_1.3.0       utf8_1.2.6        stringi_1.8.9
[17] xfun_0.60         S7_0.2.2          otel_0.2.0        timechange_0.4.0
[21] cli_3.6.6         withr_3.0.3       magrittr_2.0.5    digest_0.6.39
[25] grid_4.6.0        rstudioapi_0.19.0 hms_1.1.4         lifecycle_1.0.5
[29] vctrs_0.7.3       evaluate_1.0.5    glue_1.8.1        farver_2.1.2
[33] codetools_0.2-20  rmarkdown_2.31    tools_4.6.0       pkgconfig_2.0.3
[37] htmltools_0.5.9
```

1.27 Debugging and error messages

When an error occurs:

1. Read the entire error message.
2. Reduce the problem to the smallest example that still fails.
3. Inspect objects with `str()`, `typeof()`, `class()`, and `dim()`.
4. Use `traceback()` after an error when necessary.
5. Verify assumptions with functions such as `stopifnot()`.

```
safe_mean <- function(x) {  
  stopifnot(  
    is.numeric(x),  
    length(x) > 0  
  )  
  
  mean(x)  
}  
  
safe_mean(c(2, 4, 6))
```

```
[1] 4
```

1.28 A computational workflow for this course

A reliable statistical computation separates:

1. **Inputs:** data, constants, configuration, and random seeds.
2. **Transformation:** cleaning and constructing analysis variables.
3. **Computation:** algorithms, models, optimization, or simulation.
4. **Validation:** diagnostics, tests, and sensitivity analysis.
5. **Communication:** figures, tables, interpretation, and limitations.

For example:

```
set.seed(8670)  
  
n <- 200  
  
x <- rnorm(n)  
y <- 1 + 2 * x + rnorm(n, sd = 0.75)  
  
simulated_data <- data.frame(  
  x = x,  
  y = y
```



```
)

fit <- lm(y ~ x, data = simulated_data)

coef(fit)
```

```
(Intercept)          x
    1.020794    1.991843
```

This script records where the data came from, fixes the random seed, creates an explicit data object, and passes that object to the statistical model.

1.29 In-class exercises

Predict the value and type of each expression before running the code.

```
c(TRUE, 2L, 3.5)

c(1, 2, 3, 4) + c(10, 20)

c(1, 2, NA) > 1

TRUE | NA
```

Using

```
x <- c(4, NA, 10, 15, 3, NA, 21)
```

write expressions that return:

1. all nonmissing values;
2. values greater than or equal to 10;
3. the positions of values greater than or equal to 10;
4. the mean of the observed values.

Without running the code, determine the result of `f(5)` and identify where each occurrence of `a` is found.

```

a <- 10

f <- function(x) {
  a <- 2

  g <- function(y) {
    a * y
  }

  g(x)
}

f(5)

```

Write a function that takes a positive integer n and returns a vector whose i th element is

$$\frac{(-1)^{i+1}}{i}.$$

Preallocate the result and use `seq_len(n)`.

Using `students`, complete each task once with base R and once with `dplyr`:

1. retain students with observed scores;
2. select `name`, `program`, and `score`;
3. create a variable equal to `score - 90`;
4. sort from highest to lowest score;
5. compute the number of observed scores and mean score within each program.

Before running the code, predict whether each result is a vector, data frame, tibble, or grouped tibble.

For each task, choose among a vectorized function, `apply()`, `lapply()`, `vapply()`, `tapply()`, `dplyr::across()`, or a loop. Justify your choice and then write the code.

1. Compute the mean of every numeric column in `students`.
2. Compute the row sums of a $10,000 \times 50$ numerical matrix.
3. Record the class of every column in a data frame.
4. Compute the mean score within each program.
5. Fit the same model to each of 20 independently generated data sets and retain the fitted-model objects.

Predict the type and contents of `result`. Explain why `apply()` is inappropriate and rewrite the computation using `vapply()` or `dplyr::across()`.

```
dat <- data.frame(
  subject = c("A", "B", "C"),
  pre = c(10, 12, 9),
  post = c(14, 13, 15)
)

result <- apply(dat, 2, mean)
```

1.30 Takeaways

- R is both a programming language and an environment for statistical computing.
- Objects and functions are the basic building blocks of R programs.
- Atomic vectors are the foundation of R computation.
- R uses **1-based indexing**.
- Vectorization, recycling, coercion, and missing-value propagation are central features of R.
- Matrices and arrays require homogeneous element types, whereas lists can contain heterogeneous objects.
- A data frame is a named list of equal-length columns.
- Tibbles are modern data frames with stricter and more predictable behavior.
- Functions have arguments, a body, and an environment.
- R uses lexical scoping and lazy evaluation.
- The apply family provides convenient abstractions for repeated computation but is not automatically faster than loops.
- `apply()` is designed primarily for matrices and arrays and can cause unwanted coercion when applied to mixed-type data frames.
- Base R and tidyverse tools are complementary.
- The pipe (`%>%` or `|>`) can make a sequence of data transformations easier to read.
- R's copy-on-modify behavior gives objects intuitive semantics, although memory allocation still matters.
- Statistical computing requires attention to floating-point arithmetic, missing values, reproducibility, and numerical stability.
- Understanding R's computational model makes code easier to predict, debug, optimize, and trust.

1.31 Optional challenge

Explain why these two functions may have very different memory and timing behavior even though they return the same values.

```

grow <- function(n) {
  out <- numeric(0)

  for (i in seq_len(n)) {
    out <- c(out, i^2)
  }

  out
}

preallocate <- function(n) {
  out <- numeric(n)

  for (i in seq_len(n)) {
    out[i] <- i^2
  }

  out
}

```

Design a timing experiment for increasing values of `n`.

Some of the materials are adapted from [CMU Stat36-350](#).

A comprehensive reference for the *tidyverse* tools is [R for Data Science](#).

A comprehensive reference for *ggplot2* is [ggplot2: Elegant Graphics for Data Analysis](#).

Additional introductory material is adapted from [Why R?](#).

2 Intro to Computational Statistics

“Statistical computing is more than programming. It is the process of using computation to connect statistical theory with practical data analysis. Programming provides the tools, but the main goal is to understand how a statistical method can be implemented, approximated, validated, and interpreted. A good computational solution should therefore be not only correct in code, but also statistically meaningful, numerically stable, and reproducible.”

A computational approach is a systematic way to solve a statistical problem when an exact analytical solution is difficult, unavailable, or inefficient. Examples include using Monte Carlo methods to approximate expectations, numerical optimization to estimate parameters, bootstrap methods to approximate sampling distributions, and cross-validation to select tuning parameters. In this course, the emphasis is on understanding the connection

Theory $\iff$ Computation $\iff$ Practice.

There is a general principle that will be repeated in this chapter that Kenneth Lange calls *optimization transfer* in his 1999 paper. The basic idea applies to the problem of maximizing a function f .

1. Direct optimize the function f .
 - It can be difficult
2. Optimize a surrogate function g that is easier to optimize than f .
3. So here, instead of optimize f , we optimize g .

Note 1: steps 2&3 are repeated until convergence.

Note 2: maximizing f is equivalent to minimizing $-f$.

Note 3: the surrogate function g should be chosen such that it is easier to optimize than f .

For instance, for a linear regression

$$y = X\beta + \varepsilon. \tag{2.1}$$

From regression class, we know that the (ordinary) least-squares estimation (OLE) for β is given by $\hat{\beta} = (X^\top X)^{-1} X^\top y$. It is convenient as the solution is in the **closed-form**! However, in the most case, the closed-form solutions will not be available.

For GLMs or non-linear regression, we need to do this **iteratively**!

2.1 Theory versus Computation

One confusing aspect of statistical computing is that often there is a disconnect between what is printed in a statistical computing textbook and what should be implemented on the computer.

- In textbooks, simpler to **present solutions as convenient mathematical formulas whenever possible**, in order to communicate basic ideas and to provide some insight.
 - However, directly translating these formulas into computer code is usually not advisable because there are many problematic aspects of computers that are simply not relevant when writing things down on paper.

Some potential issues include:

1. Memory overflow: The computer has a limited amount of memory, and it is possible to run out of memory when working with large datasets or complex models.
2. Numerical Precision: Sometimes, due to the cut precision of floating-point arithmetic, calculations that are mathematically equivalent can yield different results on a computer.
 - Example 1: round $1/3$ to two decimal places, we get 0.33. Then, $3 \cdot (1/3)$ is exactly 1, but $3 \cdot 0.33$ is 0.99.
 - Example 2: $1 - 0.99999999$ is 0.00000001 ($=1\text{E-}8$), but if we round 0.99999999 to two decimal places, we get 1.00, and then $1 - 1.00$ is 0. If we round 0.00000001 to two decimal places, we get 0.00.
 - Example 3: π
3. (Lienar) Dependence: The detection of linear dependence in matrix computations is influenced by machine precision. Since computers operate with finite precision, situations often arise where true linear dependence exists, but the computer cannot distinguish it from independence.
 - Example: Consider the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

The 3rd column is a linear combination of the first two columns (i.e., $\text{col3} = \text{col1} + \text{col2}$). However, due to machine precision limitations, the computer might not recognize this exact linear dependence, leading to numerical instability in computations involving this matrix. With a small distortion, we have

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 + 10^{-5} \end{pmatrix}$$

```
A <- matrix(
  c(1, 2, 3,
    4, 5, 6,
    7, 8, 9),
  nrow = 3, ncol = 3, byrow = TRUE)
B <- A
B[3, 3] <- B[3, 3] + 1E-5

qr(A)$rank
```

```
[1] 2
```

```
qr(B)$rank
```

```
[1] 3
```

2.2 Matrix Inversion

In many statistical analyses, such as linear regression and specify the distribution (such as normal distribution), matrix inversion plays a central role.

Example 2.1 (Normal distribution). We know that, a normal density with the parameters mean μ and standard deviation σ is

$$f(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{1}{2\sigma^2} (x - \mu)^2 \right\}$$

or we may work on the multivariate normal distribution case which is a bit more involved.

$X = (X_1, \dots, X_d)$ is said to be a multivariate normal distribution if and only if it is a linear combination of independent and identically distributed standard normals:

$$X = CZ + \mu, \quad Z = (Z_1, \dots, Z_d), \quad Z_i \stackrel{iid}{\sim} N(0, 1).$$

The property of the multivariate normal are:

- mean vector: $E(X) = \mu$
- variance: $Var(X) = CZC^\top = Cvar(Z)C^\top := \Sigma$

Notation: $X \sim N_d(\mu, \Sigma)$.

PDF:

$$f(x \mid \mu, \Sigma) = (2\pi)^{-d/2} \cdot \exp \left\{ -\frac{1}{2}(x - \mu)' \Sigma^{-1}(x - \mu) - \frac{1}{2} \log |\Sigma| \right\}.$$

Some of the potential ways to do this is to take logarithm of the PDF (Think about why).

Recall the linear regression model . The OLE for β is given by $\hat{\beta} = (X^\top X)^{-1} X^\top y$.

We can solve this using the R command

```
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
```

where `solve()` is the R function for matrix inversion. However, it is not a desired way (think about why).

A better way is to go back to the formula, and look at

$$X^\top X \beta = X^\top y,$$

and solve this using the R command

```
solve( crossprod(X), crossprod(X, y) )
# this is the same as
# solve(t(X) %*% X, t(X) %*% y)
```

Here, we avoid explicitly calculating the inverse of $X^\top X$. Instead, we use gaussian elimination to solve the system of equations, which is generally more numerically stable and efficient.

2.2.0.1 Speed comparison

```
set.seed(2025-09-03)
X <- matrix(rnorm(5000 * 100), 5000, 100)
y <- rnorm(5000)
library(microbenchmark)
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y)
```


Unit: milliseconds

| | | | expr | min | lq |
|----------|----------|----------|---|----------|----------|
| | | | <code>solve(t(X) %*% X) %*% t(X) %*% y</code> | 28.83505 | 30.16593 |
| mean | median | uq | max | neval | |
| 31.96782 | 30.79489 | 32.63315 | 111.0151 | 100 | |

Warning message:

In `microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y) :`
less accurate nanosecond times to avoid potential integer overflows

```
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,  
               solve(crossprod(X), crossprod(X, y)))
```

Unit: milliseconds

| | | | expr | min | lq |
|----------|----------|----------|---|----------|----------|
| | | | <code>solve(t(X) %*% X) %*% t(X) %*% y</code> | 28.90135 | 30.11608 |
| | | | <code>solve(crossprod(X), crossprod(X, y))</code> | 25.05859 | 25.27480 |
| mean | median | uq | max | neval | |
| 31.78686 | 31.38513 | 32.66482 | 53.03354 | 100 | |
| 26.15771 | 25.81678 | 26.89188 | 29.12045 | 100 | |

Take home message 1

The take home here is that the issues arise from the finite precision of computer arithmetic and the limited memory available on computers. When implementing statistical methods on a computer, it is crucial to consider these limitations and choose algorithms and implementations that are robust to numerical issues.

2.2.1 Multi-collinearity

The above approach may break down when there is any multi-collinearity in the X matrix. For example, we can tack on a column to X that is very similar (but not identical) to the first column of X .

```
set.seed(7777)  
N <- 3000  
K <- 100  
y <- rnorm(N)  
X <- matrix(rnorm(N * K), N, K)  
W <- cbind(X, X[, 1] + rnorm(N, sd = 1E-15))
```

```
solve(crossprod(W), crossprod(W, y))

Error in `solve.default()`:
! system is computationally singular: reciprocal condition number = 1.36748e-32
```

The algorithm does not work because the cross product matrix $W^\top W$ is **singular**. In practice, matrices like these can come up a lot in data analysis and it would be useful to have a way to deal with it automatically.

R takes a different approach to solving for the unknown coefficients in a linear model. R uses the QR decomposition, which is not as fast, but has the added benefit of being able to automatically detect and handle colinear columns in the matrix.

Here, we use the fact that X can be decomposed as $X = QR$, where Q is an orthonormal matrix and R is an upper triangular matrix. Given that, we can rewrite $X^\top X\beta = X^\top y$ as

$$\begin{aligned} R^\top Q^\top QR\beta &= R^\top Q^\top y \\ R^\top IR\beta &= R^\top Q^\top y \\ R^\top R\beta &= R^\top Q^\top y, \end{aligned}$$

this leads to $R\beta = Q^\top y$. Now we can perform the Gaussian elimination to do it. Because R is an upper triangular matrix, the computational speed is much faster. Here, we **avoid to compute the cross product** $X^\top X$, which is numerical unstable if it is not *standardized* properly

We can see in R code that even with our singular matrix W above, the QR decomposition continues without error.

```
Qw <- qr(W)
str(Qw)
```

```
List of 4
 $ qr    : num [1:3000, 1:101] 54.43933 0.00123 -0.02004 -0.00671 -0.00178 ...
 $ rank  : int 100
 $ graux : num [1:101] 1.01 1.01 1.01 1 1 ...
 $ pivot : int [1:101] 1 2 3 4 5 6 7 8 9 10 ...
 - attr(*, "class")= chr "qr"
```

Note that the output of `qr()` computes the rank of W to be 100, not 101 as the last column is collinear to the 1st column. From there, we can get $\hat{\beta}$ if we want using `qr.coef()`,

```
betahat <- qr.coef(Qw, y)
head(betahat, 3)
```

```
[1] 0.024314718 0.000916951 -0.005980588
```

```
tail(betahat, 3)
```

```
[1] 0.01545039 -0.01010440 NA
```

Q: Why there is an NA?

2.2.2 Trade-off

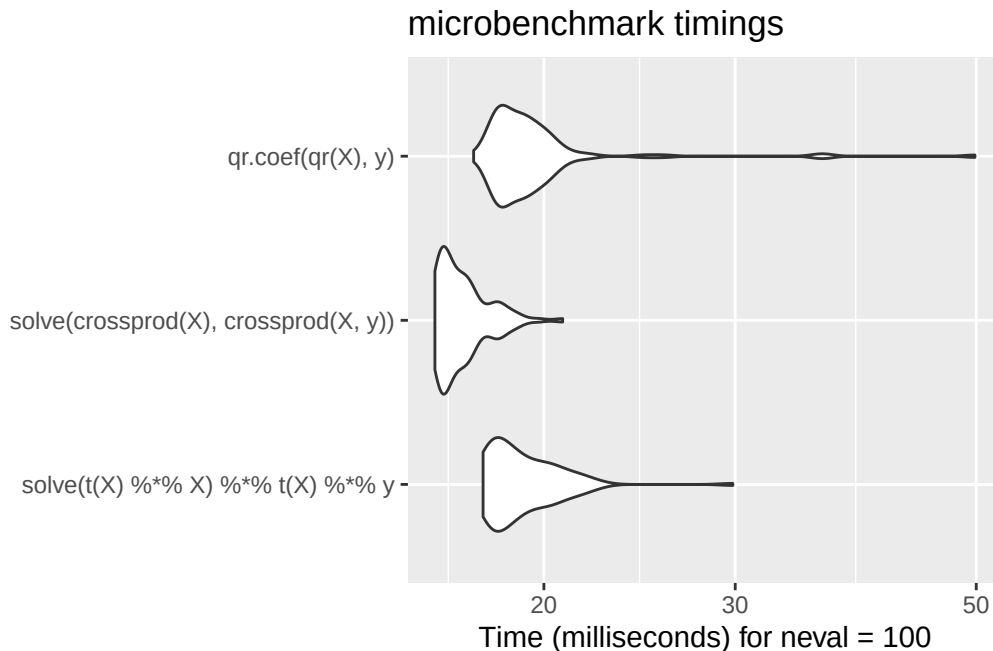
There isn't always elegance and flourish. When we take the robust approach, we accept that it comes at a cost.

```
library(ggplot2)
library(microbenchmark)
m <- microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,
                    solve(crossprod(X), crossprod(X, y)),
                    qr.coef(qr(X), y))
```

```
Warning in microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y, solve(crossprod(X),
: less accurate nanosecond times to avoid potential integer overflows
```

```
autoplot(m)
```

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the microbenchmark package.
  Please report the issue at
  <https://github.com/joshuaulrich/microbenchmark/issues/>.
```



Compared with the approaches discussed above, this method performs similarly to the naive approach but is much more stable and reliable.

In practice, we rarely call functions such as `qr()` or `qr.coef()` directly, since higher-level functions like `lm()` handle these computations automatically. However, in certain specialized and performance-critical settings, it can be advantageous to use alternative matrix decompositions to compute regression coefficients, especially when the computation must be repeated many times in a loop (i.e., *Vectorization*)

Example 2.2 (Multivariate Normal revisit). Computing the multivariate normal (MVN) density is a common task, for example, when fitting spatial models or Gaussian process models. Because maximum likelihood estimation (MLE) and likelihood ratio tests (LRT) often require evaluating the likelihood many times, efficiency is crucial.

After taking the log of the MVN density, we have

$$\ell(x \mid \mu, \Sigma) := \log \{f(x \mid \mu, \Sigma)\} = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu).$$

On the right hand side, the first term is a constant, the second term is linear, and the last term is quadratic, which requires much more computational power.

A Naive Implementation

We first center the data $z := x - \mu$. Then we have $z^\top \Sigma^{-1} z$. This simplified the question for a bit.

Here, much like the linear regression example above, the key bottleneck is the inversion of the p -dimensional covariance matrix Σ . If we take z to be a $p \times 1$ column vector, then a literal translation of the mathematics into R code might look something like this,

```
t(z) %*% solve(Sigma) %*% z
```

To illustrate, let's simulate some data and compute the quadratic form the naive way:

```
set.seed(2025-09-03)

# Generate data
z <- matrix(rnorm(200 * 100), 200, 100)
S <- cov(z)

# Naive quadratic form
quad.naive <- function(z, S) {
  Sinv <- solve(S)
  rowSums((z %*% Sinv) * z)
}

library(dplyr)
quad.naive(z, S) %>% summary()
```

| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|-------|---------|--------|--------|---------|--------|
| 70.67 | 93.61 | 99.94 | 100.54 | 107.31 | 126.73 |

A Better Way: Cholesky Decomposition

Because the covariance matrix is symmetric and positive definite, we can exploit its **Cholesky decomposition**. That is, we write $\Sigma = R^\top R$, where R is a upper triangular matrix. Then,

$$z^\top \Sigma^{-1} z = z^\top (R^\top R)^{-1} z = z^\top R^{-1} R^{-\top} z = (R^{-\top} z)^\top (R^{-\top} z) := v^\top v.$$

Note that $v \in \mathbb{R}^p$ is the solution to the linear system $R^\top v = z$. Because R is upper triangular, we can solve this system efficiently using back substitution. Also, we can solve this without doing the inversion.

Once we have v we can compute its quadratic form $v^\top v$ by the `crossprod()` function.

```
quad.chol <- function(z, S) {
  R <- chol(S)
  v <- backsolve(R, t(z), transpose = TRUE)
```

```
colSums(v * v)
}

quad.chol(z, S) %>% summary()
```

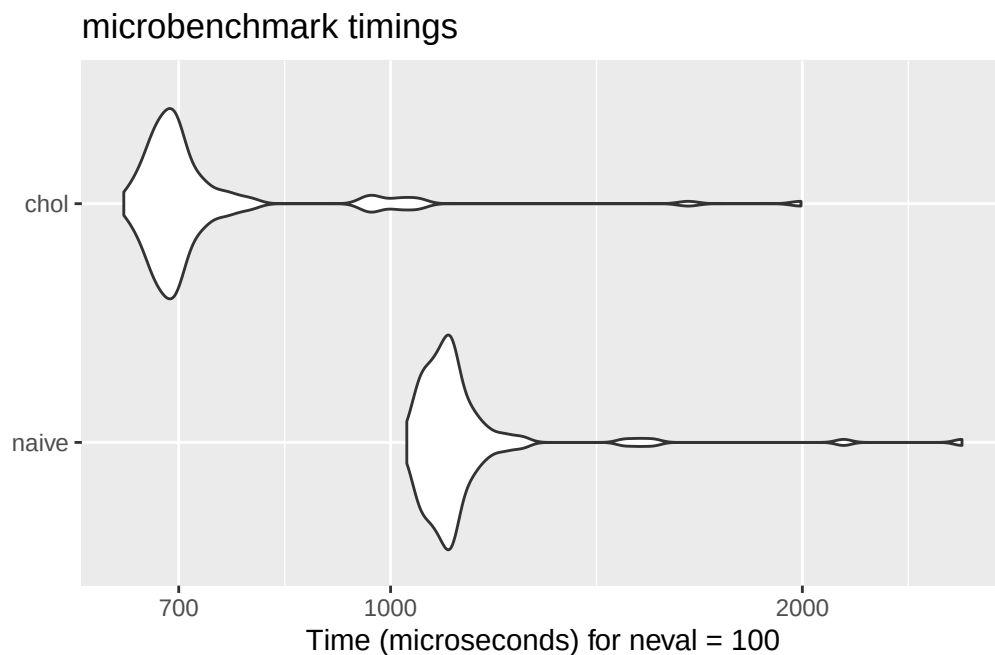
| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|-------|---------|--------|--------|---------|--------|
| 70.67 | 93.61 | 99.94 | 100.54 | 107.31 | 126.73 |

By product

Another benefit of the Cholesky decomposition is that it gives us a simple way to compute the log-determinant of Σ . The log-determinant of Σ is simply two times the sum of the log of the diagonal elements of R . (Why?)

Performance comparison

```
library(microbenchmark)
library(ggplot2)
m2 <- microbenchmark(
  naive = quad.naive(z, S),
  chol  = quad.chol(z, S)
)
autoplot(m2)
```



Q: Why one is faster than the other?

 Take home message 2

The naive algorithm simply inverts the covariance matrix. The Cholesky-based approach, on the other hand, exploits the fact that covariance matrices are symmetric and positive definite. This results in an implementation that is both faster and numerically more stable—exactly the kind of optimization that makes a difference in real-world statistical computing.

Thus, a knowledge of statistics and numerical analysis can often lead to better algorithms, often invaluable!

Examples are borrowed from the following sources:

- Peng, R.D. [Advanced Statistical Computing](#).

3 Optimization

Optimization plays an important role in statistical computing. Many statistical estimators can be written as

$$\hat{\theta} = \arg \min_{\theta} Q(\theta) \quad \text{or} \quad \hat{\theta} = \arg \max_{\theta} \ell(\theta),$$

where $Q(\theta)$ is an objective function and $\ell(\theta)$ may be a likelihood.

For simple problems, a closed-form solution may exist. For many statistical models, however, the solution must be obtained numerically.

Examples include maximum likelihood estimation, nonlinear regression, mixture models, machine learning, smoothing, and regularization.

The central idea is

Statistical Problem $\iff$ Optimization Problem $\iff$ Numerical Algorithm $\iff$ R Implementation.

3.1 Why Numerical Optimization?

Consider

$$X_1, \dots, X_n \stackrel{iid}{\sim} N(\mu, 1).$$

From mathematical statistics,

$$\hat{\mu}_{MLE} = \bar{X}.$$

We already know the closed-form solution. However, we can also obtain the estimator computationally by minimizing the negative log-likelihood.


```

set.seed(8670)

x <- rnorm(
  n = 50,
  mean = 2,
  sd = 1
)

nll <- function(mu) {
  -sum(
    dnorm(
      x,
      mean = mu,
      sd = 1,
      log = TRUE
    )
  )
}

fit <- optimize(
  nll,
  interval = c(-1, 5)
)

c(
  numerical = fit$minimum,
  theoretical = mean(x)
)

```

```

numerical theoretical
1.89689      1.89689

```

Visualizing the Objective Function

Rather than treating an optimizer as a black box, we can evaluate the objective function over a grid.

```

mu_grid <- seq(
  -1,
  5,
  length.out = 400
)

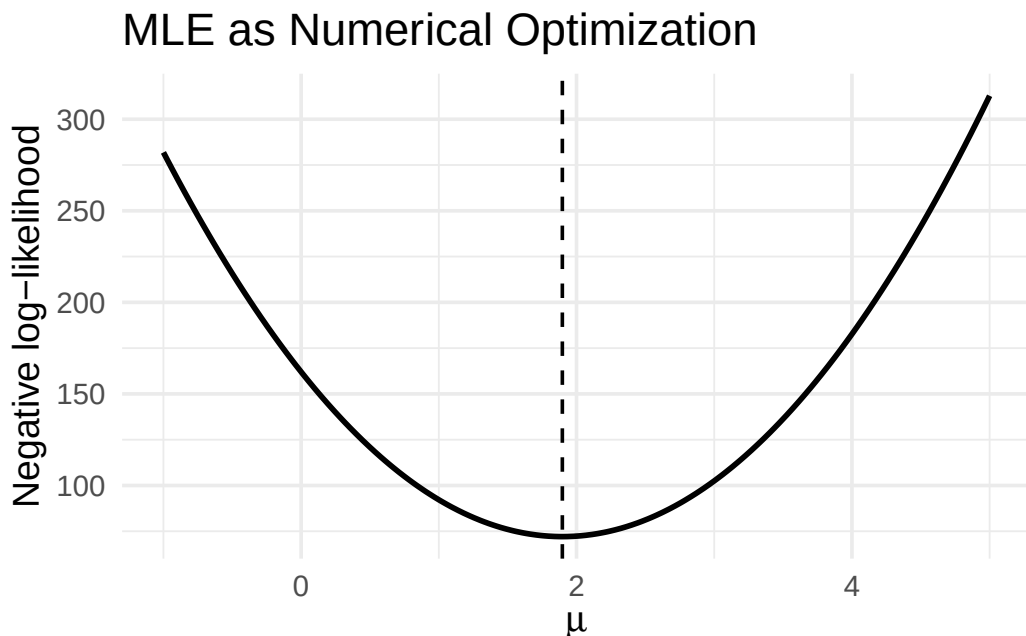
```

```

nll_data <- data.frame(
  mu = mu_grid,
  nll = vapply(
    mu_grid,
    nll,
    numeric(1)
  )
)

ggplot(
  nll_data,
  aes(
    x = mu,
    y = nll
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = mean(x),
    linetype = "dashed"
  ) +
  labs(
    x = expression(mu),
    y = "Negative log-likelihood",
    title = "MLE as Numerical Optimization"
  )

```



Why is numerical optimization useful here if we already know that

$$\hat{\mu}_{MLE} = \bar{X}?$$

The closed-form solution gives us a way to **validate the numerical algorithm**.

When a closed-form solution is unavailable, numerical optimization becomes essential.

3.2 When There Is No Closed-Form Solution

Consider the nonlinear regression model

$$Y_i = \exp(\theta X_i) + \varepsilon_i.$$

Suppose we estimate θ by minimizing

$$Q(\theta) = \sum_{i=1}^n [Y_i - \exp(\theta X_i)]^2.$$

Unlike ordinary linear regression, there is no convenient closed-form least-squares estimator.

```

set.seed(8670)

x <- seq(
  0,
  1,
  length.out = 50
)

y <- exp(1.5 * x) +
  rnorm(
    50,
    sd = 0.15
  )

sse <- function(theta) {
  sum(
    (y - exp(theta * x))^2
  )
}

fit_nonlinear <- optimize(
  sse,
  interval = c(0, 3)
)

fit_nonlinear$minimum

```

```
[1] 1.486825
```

```

theta_grid <- seq(
  0,
  3,
  length.out = 400
)

sse_data <- data.frame(
  theta = theta_grid,
  SSE = vapply(
    theta_grid,
    sse,
    numeric(1)
  )
)

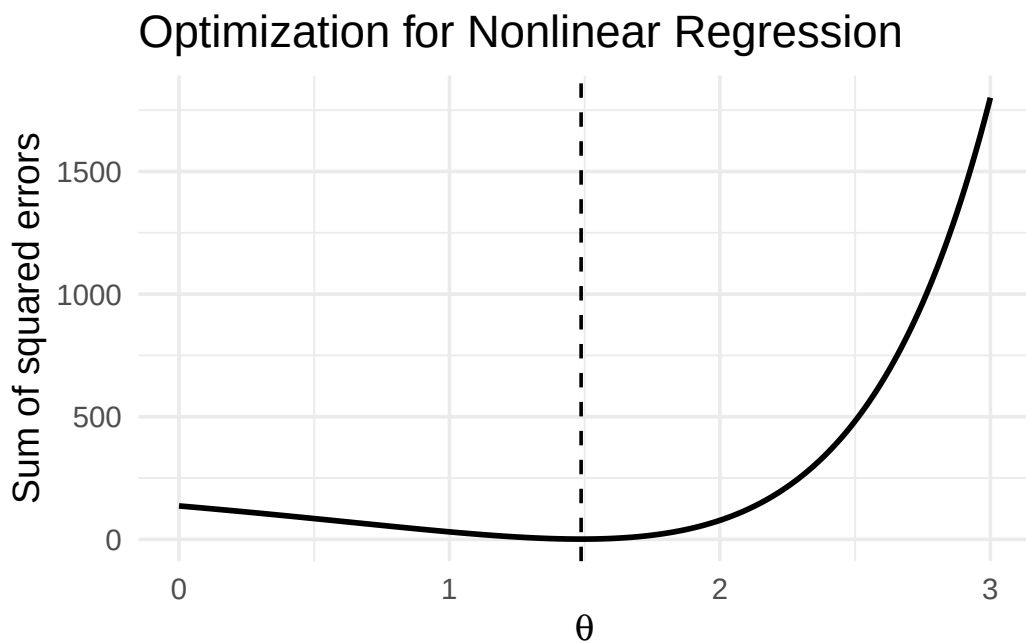
```

```

)
)

ggplot(
  sse_data,
  aes(
    x = theta,
    y = SSE
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = fit_nonlinear$minimum,
    linetype = "dashed"
  ) +
  labs(
    x = expression(theta),
    y = "Sum of squared errors",
    title = "Optimization for Nonlinear Regression"
  )
)

```



The important distinction is

| |
|---|
| Theory tells us what to optimize; computation tells us how to find it. |
|---|

3.3 Two Families of Optimization Methods

Computational optimization methods can be broadly divided into two families:

| |
|--|
| Deterministic Methods and Metaheuristic Methods. |
|--|

3.3.1 Deterministic Methods

A deterministic algorithm follows a predictable sequence of updates once the starting value is fixed.

Examples include:

- Newton's method;
- gradient descent;
- quasi-Newton methods such as BFGS;
- deterministic root-finding methods.

Advantages:

- usually efficient for smooth and well-behaved problems;
- mathematically well understood;
- convergence can often be studied theoretically;
- reproducible for a fixed starting value.

Limitations:

- may converge to a local rather than global optimum;
- can be sensitive to the starting value;
- some methods require derivatives;
- poor scaling can substantially slow convergence.

3.3.2 Metaheuristic Methods

Metaheuristic methods introduce randomness or population-based search to explore the parameter space more broadly.

Examples include:

- simulated annealing;
- genetic algorithms;
- particle swarm optimization.

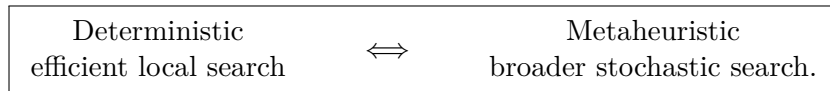
Advantages:

- can escape local optima;
- usually do not require derivatives;
- useful for multimodal or irregular objectives;
- can be applied to complicated search spaces.

Limitations:

- often computationally expensive;
- results can vary between runs;
- tuning parameters can strongly affect performance;
- global optimality is generally not guaranteed.

The basic trade-off is



3.4 Local and Global Optima

Consider

$$Q(x) = 0.05x^2 + \sin^2(x) + 0.2 \sin(5x).$$

```
objective <- function(x) {  
  0.05 * x^2 +  
  sin(x)^2 +  
  0.2 * sin(5 * x)  
}
```

```

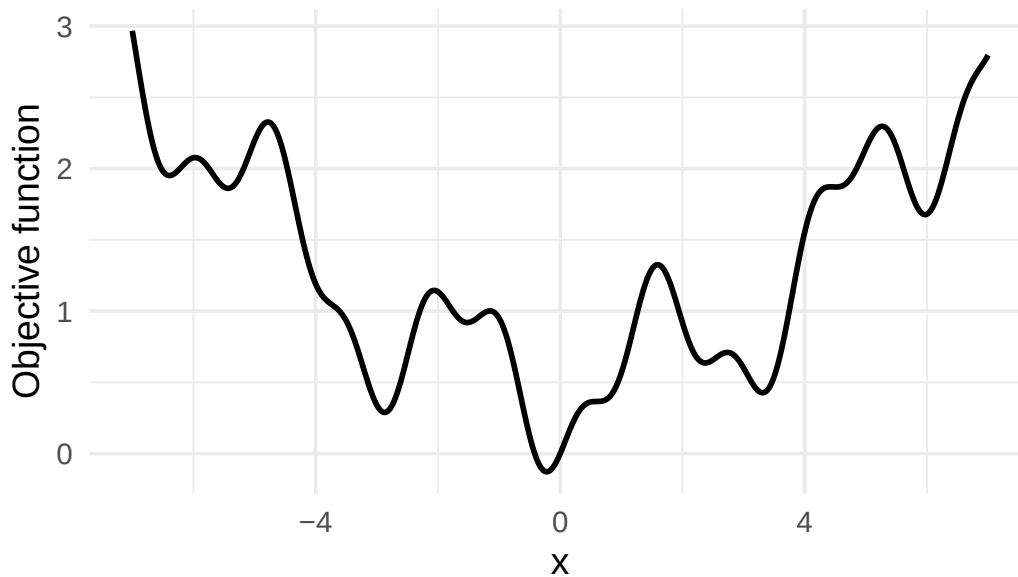
x_grid <- seq(
  -7,
  7,
  length.out = 1000
)

objective_data <- data.frame(
  x = x_grid,
  objective = objective(x_grid)
)

ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = "x",
    y = "Objective function",
    title = "Local and Global Optima"
  )

```


Local and Global Optima



The objective contains several local minima.

A deterministic local optimizer may converge to different solutions depending on its starting value.

```
starts <- c(
  -6,
  -2,
  2,
  6
)

solutions <- vapply(
  starts,
  function(x0) {
    optim(
      par = x0,
      fn = objective,
      method = "BFGS"
    )$par
  },
  numeric(1)
)
```

```
local_results <- data.frame(
  start = starts,
  solution = solutions,
  objective = objective(solutions)
)
```

```
local_results
```

```

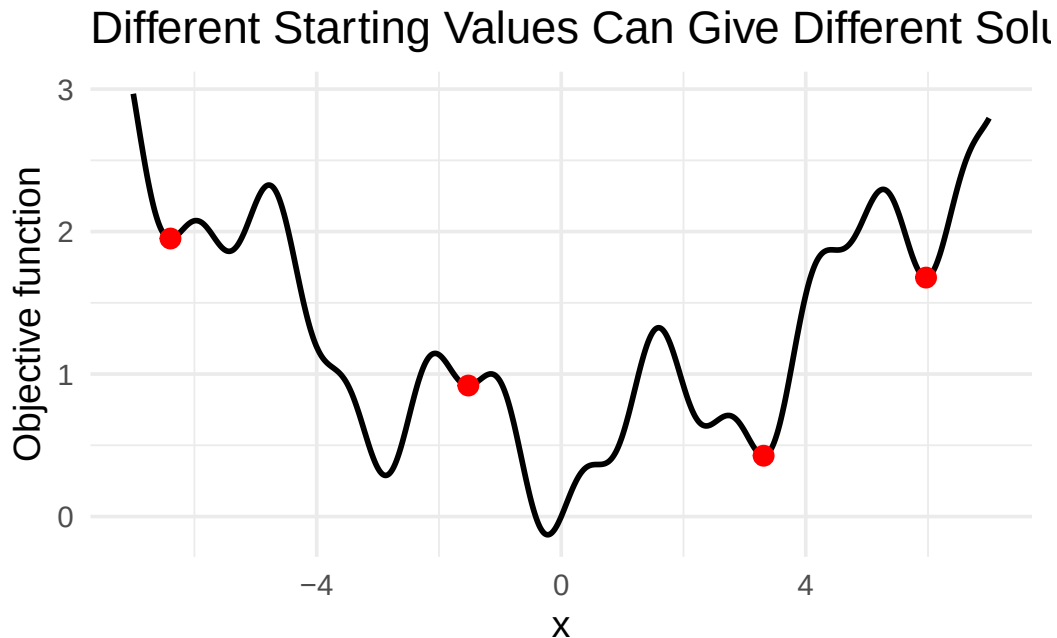
      start solution objective
1      -6 -6.392105  1.9511520
2      -2 -1.519268  0.9193572
3       2  3.310780  0.4266970
4       6  5.967669  1.6769489

```

```

ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  geom_point(
    data = local_results,
    col = "red",
    aes(
      x = solution,
      y = objective
    ),
    size = 3
  ) +
  labs(
    x = "x",
    y = "Objective function",
    title = "Different Starting Values Can Give Different Solutions"
  )

```



If the algorithm is deterministic, why can different starting values still produce different answers?

The important distinction is

Local Optimum $\neq$ Global Optimum.

3.5 Deterministic Numerical Methods

We first study several deterministic numerical methods.

3.5.1 Root Finding

Root finding solves

$$f(x) = 0.$$

Why is root finding relevant to optimization?

If $Q(x)$ is differentiable and x^* is an interior optimum, then typically

$$Q'(x^*) = 0.$$

Therefore, optimization can sometimes be converted into

$$f(x) = Q'(x) = 0.$$

A root of $Q'(x)$ is only a candidate optimum. It may correspond to a minimum, maximum, or another stationary point.

Suppose we want the 97.5th percentile of a standard normal distribution.

We seek z such that

$$P(Z \leq z) = 0.975.$$

Equivalently,

$$\Phi(z) - 0.975 = 0.$$

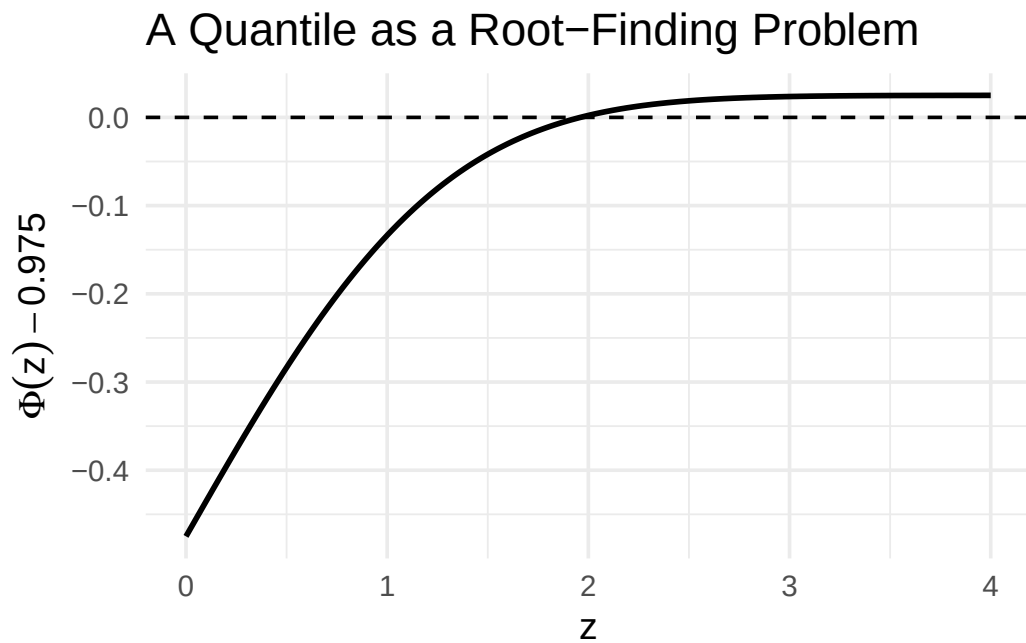
```
f_root <- function(z) {
  pnorm(z) - 0.975
}

z_grid <- seq(
  0,
  4,
  length.out = 400
)

root_data <- data.frame(
  z = z_grid,
  f = f_root(z_grid)
)

ggplot(
  root_data,
  aes(
    x = z,
    y = f
  )
) +
```

```
geom_line(linewidth = 1) +
geom_hline(
  yintercept = 0,
  linetype = "dashed"
) +
labs(
  x = "z",
  y = expression(Phi(z) - 0.975),
  title = "A Quantile as a Root-Finding Problem"
)
```



R can solve the root directly:

```
root_R <- uniroot(
  f_root,
  interval = c(0, 4)
)$root

c(
  root_finding = root_R,
  qnorm = qnorm(0.975)
)
```

| | |
|---------------------------|--------------------|
| <code>root_finding</code> | <code>qnorm</code> |
| 1.959963 | 1.959964 |

Thus,

Quantile Calculation $\Leftrightarrow$ Root Finding.

Why do `uniroot()` and `qnorm()` give essentially the same answer?

3.5.2 Bisection Method

The bisection method is similar to a binary search.

Suppose f is continuous and

$$f(a)f(b) < 0.$$

Then there must be at least one root between a and b .

1. Choose a and b such that $f(a)f(b) < 0$.
2. Compute

$$c = \frac{a + b}{2}.$$

3. Keep the half containing the sign change.
4. Repeat until the interval is sufficiently small.

The main intuition is

Each iteration cuts the search interval approximately in half.

```
bisect_trace <- function(
  f,
  a,
  b,
  tol = 1e-6,
  maxit = 100
) {
```

```

if (f(a) * f(b) > 0) {
  stop("f(a) and f(b) must have opposite signs.")
}

history <- data.frame()

for (k in 0:maxit) {

  midpoint <- (a + b) / 2

  history <- rbind(
    history,
    data.frame(
      iteration = k,
      a = a,
      b = b,
      midpoint = midpoint,
      f_midpoint = f(midpoint)
    )
  )

  if (
    abs(f(midpoint)) < tol ||
    (b - a) / 2 < tol
  ) {
    break
  }

  if (f(a) * f(midpoint) <= 0) {
    b <- midpoint
  } else {
    a <- midpoint
  }
}

history

bisect_history <- bisect_trace(
  f_root,
  a = 0,
  b = 4

```

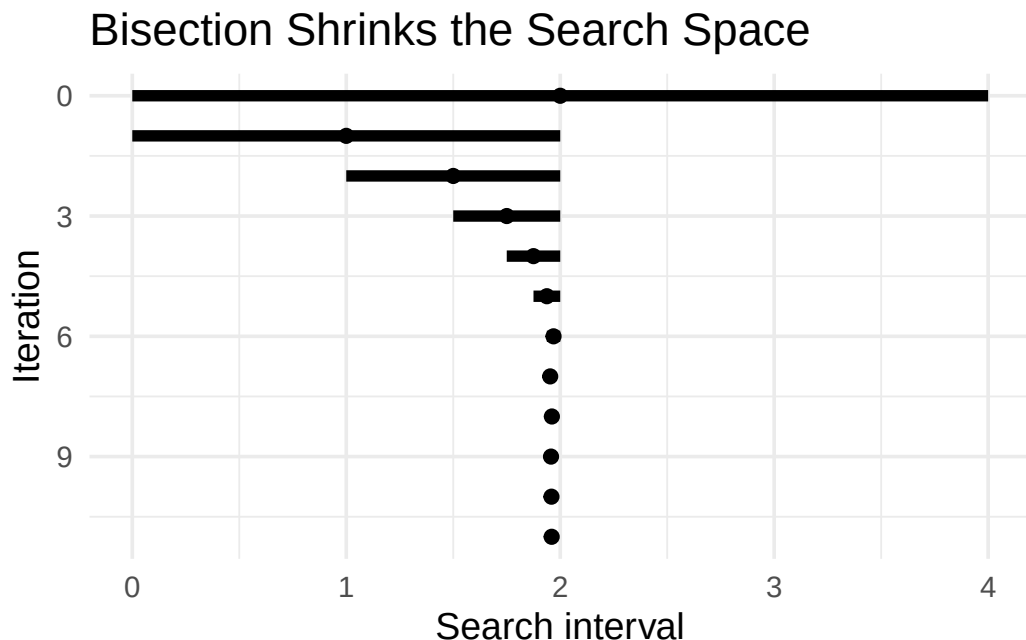
```
)
```

```
tail(bisect_history)
```

| | iteration | a | b | midpoint | f_midpoint |
|----|-----------|----------|----------|----------|---------------|
| 7 | 6 | 1.937500 | 2.000000 | 1.968750 | 5.090967e-04 |
| 8 | 7 | 1.937500 | 1.968750 | 1.953125 | -4.023926e-04 |
| 9 | 8 | 1.953125 | 1.968750 | 1.960938 | 5.684292e-05 |
| 10 | 9 | 1.953125 | 1.960938 | 1.957031 | -1.718972e-04 |
| 11 | 10 | 1.957031 | 1.960938 | 1.958984 | -5.730834e-05 |
| 12 | 11 | 1.958984 | 1.960938 | 1.959961 | -1.780850e-07 |

Visualizing

```
ggplot(  
  bisect_history,  
  aes(y = iteration)  
) +  
  geom_segment(  
    aes(  
      x = a,  
      xend = b,  
      yend = iteration  
    ),  
    linewidth = 2  
  ) +  
  geom_point(  
    aes(x = midpoint),  
    size = 2  
  ) +  
  scale_y_reverse() +  
  labs(  
    x = "Search interval",  
    y = "Iteration",  
    title = "Bisection Shrinks the Search Space"  
  )
```

Bisection requires continuity and a sign change, but does not require derivatives.

What information does bisection use about the function?

What information does it ignore?

3.5.3 Newton-Raphson Method

Newton's method uses local derivative information.

Around the current estimate x_k ,

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k).$$

Set the approximation equal to zero:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k).$$

Therefore,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

The intuition is:

Approximate the function locally by a straight line and use the root of that line as the next guess.

For

$$f(z) = \Phi(z) - 0.975,$$

we have

$$f'(z) = \phi(z).$$

```
f <- function(z) {  
  pnorm(z) - 0.975  
}
```

```
df <- function(z) {  
  dnorm(z)  
}
```

```
newton_trace <- function(  
  f,  
  df,  
  x0,  
  tol = 1e-8,  
  maxit = 100  
) {  
  
  x <- x0  
  
  history <- data.frame(  
    iteration = 0,  
    x = x,  
    fx = f(x)  
  )  
  
  for (k in seq_len(maxit)) {  
  
    dfx <- df(x)  
  
    if (abs(dfx) < 1e-12) {
```

```

    stop("Derivative is too close to zero.")
  }

  x_new <- x - f(x) / dfx

  history <- rbind(
    history,
    data.frame(
      iteration = k,
      x = x_new,
      fx = f(x_new)
    )
  )

  if (abs(x_new - x) < tol) {
    break
  }

  x <- x_new
}

history
}

newton_history <- newton_trace(
  f,
  df,
  x0 = 1
)

newton_history

```

| | iteration | x | fx |
|---|-----------|----------|---------------|
| 1 | 0 | 1.000000 | -1.336553e-01 |
| 2 | 1 | 1.552361 | -3.528790e-02 |
| 3 | 2 | 1.847484 | -7.338509e-03 |
| 4 | 3 | 1.948843 | -6.570601e-04 |
| 5 | 4 | 1.959844 | -7.006182e-06 |
| 6 | 5 | 1.959964 | -8.227717e-10 |
| 7 | 6 | 1.959964 | 0.000000e+00 |
| 8 | 7 | 1.959964 | 0.000000e+00 |

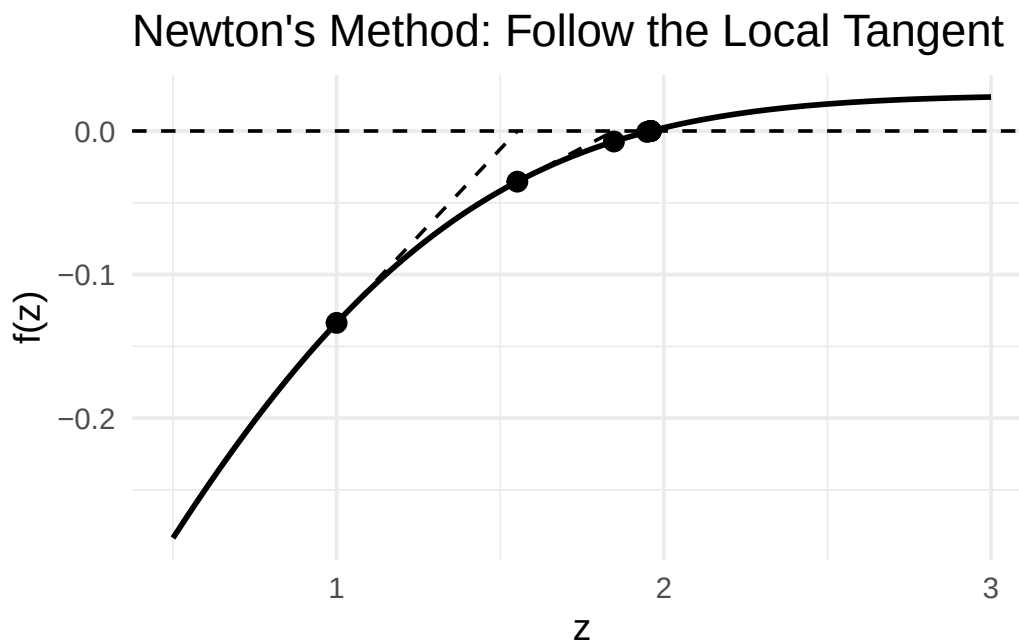
3.5.4 Visualizing Newton's Method

```
tangent_data <- data.frame(  
  x = head(newton_history$x, -1),  
  y = head(newton_history$fx, -1),  
  xend = tail(newton_history$x, -1),  
  yend = 0  
)  
  
z_grid <- seq(  
  0.5,  
  3,  
  length.out = 400  
)  
  
newton_function_data <- data.frame(  
  z = z_grid,  
  f = f(z_grid)  
)  
  
ggplot(  
  newton_function_data,  
  aes(  
    x = z,  
    y = f  
  )  
) +  
  geom_line(linewidth = 1) +  
  geom_hline(  
    yintercept = 0,  
    linetype = "dashed"  
  ) +  
  geom_segment(  
    data = tangent_data,  
    aes(  
      x = x,  
      y = y,  
      xend = xend,  
      yend = yend  
    ),  
    inherit.aes = FALSE,  
    linetype = "dashed"
```

```

) +
geom_point(
  data = newton_history,
  aes(
    x = x,
    y = fx
  ),
  inherit.aes = FALSE,
  size = 3
) +
labs(
  x = "z",
  y = "f(z)",
  title = "Newton's Method: Follow the Local Tangent"
)

```



Newton's method can converge very quickly near the root.

However, if

$$f'(x_k) \approx 0,$$

then the update may become very large and unstable.

What happens to Newton's update when $f'(x_k)$ is close to zero?

3.5.5 Secant Method

Newton's method requires the derivative $f'(x)$.

If the derivative is unavailable or expensive, we can approximate it using two previous points:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this into Newton's update gives

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

```
secant_trace <- function(  
  f,  
  x0,  
  x1,  
  tol = 1e-8,  
  maxit = 100  
) {  
  
  history <- data.frame(  
    iteration = c(0, 1),  
    x = c(x0, x1),  
    fx = c(  
      f(x0),  
      f(x1)  
    )  
  )  
  
  for (k in 2:maxit) {  
  
    f0 <- f(x0)  
    f1 <- f(x1)  
  
    denominator <- f1 - f0  
  
    if (abs(denominator) < 1e-12) {  
      stop("Secant denominator is too close to zero.")  
    }  
  }  
}
```

```

    }

    x2 <- x1 -
      f1 * (x1 - x0) /
      denominator

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        x = x2,
        fx = f(x2)
      )
    )

    if (abs(x2 - x1) < tol) {
      break
    }

    x0 <- x1
    x1 <- x2
  }

  history
}

secant_history <- secant_trace(
  f,
  x0 = 1,
  x1 = 2.5
)

secant_history

```

| | iteration | x | fx |
|---|-----------|----------|---------------|
| 1 | 0 | 1.000000 | -1.336553e-01 |
| 2 | 1 | 2.500000 | 1.879033e-02 |
| 3 | 2 | 2.315111 | 1.469657e-02 |
| 4 | 3 | 1.651361 | -2.433241e-02 |
| 5 | 4 | 2.065173 | 5.546662e-03 |
| 6 | 5 | 1.988354 | 1.613717e-03 |
| 7 | 6 | 1.956835 | -1.834592e-04 |

| | | | |
|----|----|----------|---------------|
| 8 | 7 | 1.960052 | 5.151979e-06 |
| 9 | 8 | 1.959964 | 1.577731e-08 |
| 10 | 9 | 1.959964 | -1.363021e-12 |
| 11 | 10 | 1.959964 | 0.000000e+00 |

The three methods illustrate different computational trade-offs.

| Method | Derivative? | Starting information | Main advantage | Main limitation |
|-----------|-------------|----------------------|-------------------------|-----------------|
| Bisection | No | $[a, b]$ | Very reliable | Slower |
| Newton | Yes | x_0 | Very fast near root | Can fail |
| Secant | No | x_0, x_1 | Fast without derivative | Less reliable |

3.6 Convergence

Suppose an iterative algorithm produces

$$x_0, x_1, x_2, \dots$$

and converges to a solution x^* .

Define

$$e_k = |x_k - x^*|.$$

A common description of convergence is

$$e_{k+1} \approx C e_k^q.$$

The value q describes how quickly the error decreases.

- **Linear convergence:** $q = 1$.
- **Superlinear convergence:** faster than linear.
- **Quadratic convergence:** $q = 2$.

Under suitable conditions:

- bisection converges linearly;
- the secant method converges superlinearly;
- Newton's method converges quadratically near the root.

3.6.1 Comparing Convergence in R

```
true_root <- qnorm(0.975)

bisection_error <- data.frame(
  iteration = bisect_history$iteration,
  error = abs(
    bisect_history$midpoint -
    true_root
  ),
  method = "Bisection"
)

newton_error <- data.frame(
  iteration = newton_history$iteration,
  error = abs(
    newton_history$x -
    true_root
  ),
  method = "Newton"
)

secant_error <- data.frame(
  iteration = secant_history$iteration,
  error = abs(
    secant_history$x -
    true_root
  ),
  method = "Secant"
)

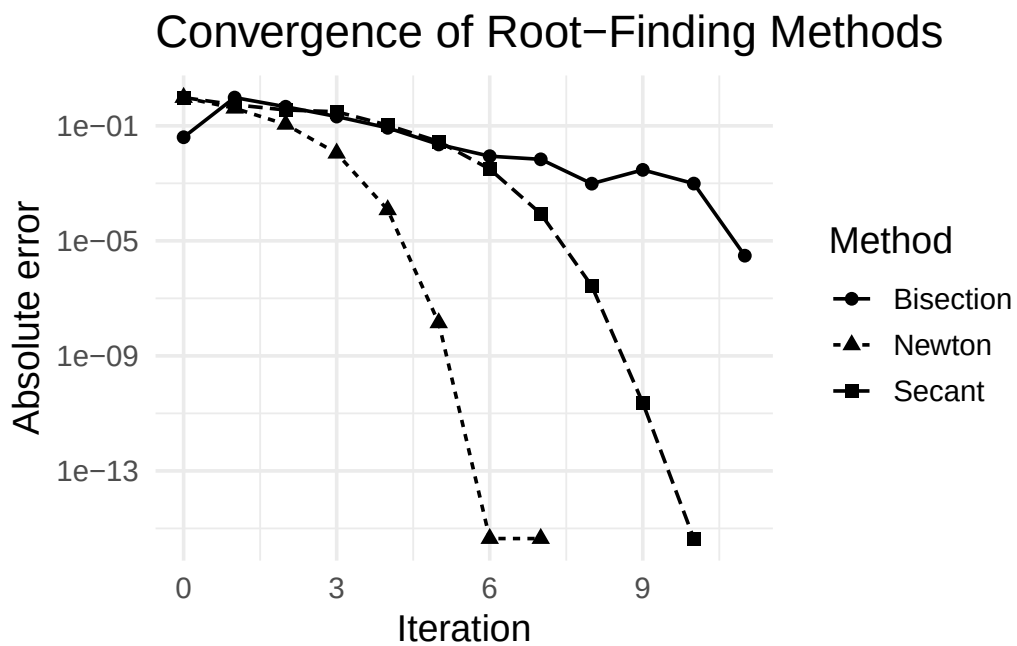
convergence_data <- rbind(
  bisection_error,
  newton_error,
  secant_error
)

convergence_data <- subset(
  convergence_data,
  error > .Machine$double.eps
)
```

```

ggplot(
  convergence_data,
  aes(
    x = iteration,
    y = error,
    linetype = method,
    shape = method
  )
) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_log10() +
  labs(
    x = "Iteration",
    y = "Absolute error",
    title = "Convergence of Root-Finding Methods",
    linetype = "Method",
    shape = "Method"
  )

```



Which method appears to converge fastest?

Does fewer iterations necessarily mean less computation?

A useful computational perspective is

| |
|--|
| Total Computational Cost $\approx$ Cost per Iteration $\times$ Number of Iterations. |
|--|

3.7 From One Dimension to Multiple Dimensions

Most statistical problems involve more than one parameter.

Instead of

$$x \in \mathbb{R},$$

we now consider

$$\theta = (\theta_1, \dots, \theta_p)^\top \in \mathbb{R}^p.$$

The one-dimensional concepts generalize naturally:

$$Q'(x) \quad \Rightarrow \quad \nabla Q(\theta),$$

and

$$Q''(x) \quad \Rightarrow \quad H(\theta).$$

The **gradient** is

$$\nabla Q(\theta) = \begin{pmatrix} \partial Q / \partial \theta_1 \\ \vdots \\ \partial Q / \partial \theta_p \end{pmatrix},$$

and the **Hessian** is the matrix of second derivatives,

$$H(\theta) = \left[\frac{\partial^2 Q}{\partial \theta_j \partial \theta_k} \right].$$

Consider

$$Q(\theta_1, \theta_2) = (\theta_1 - 2)^2 + 4(\theta_2 + 1)^2.$$

```

objective_2d <- function(theta1, theta2) {
  (theta1 - 2)^2 +
    4 * (theta2 + 1)^2
}

theta1_grid <- seq(
  -2,
  6,
  length.out = 150
)

theta2_grid <- seq(
  -4,
  2,
  length.out = 150
)

objective_grid <- expand.grid(
  theta1 = theta1_grid,
  theta2 = theta2_grid
)

objective_grid$value <- with(
  objective_grid,
  objective_2d(theta1, theta2)
)

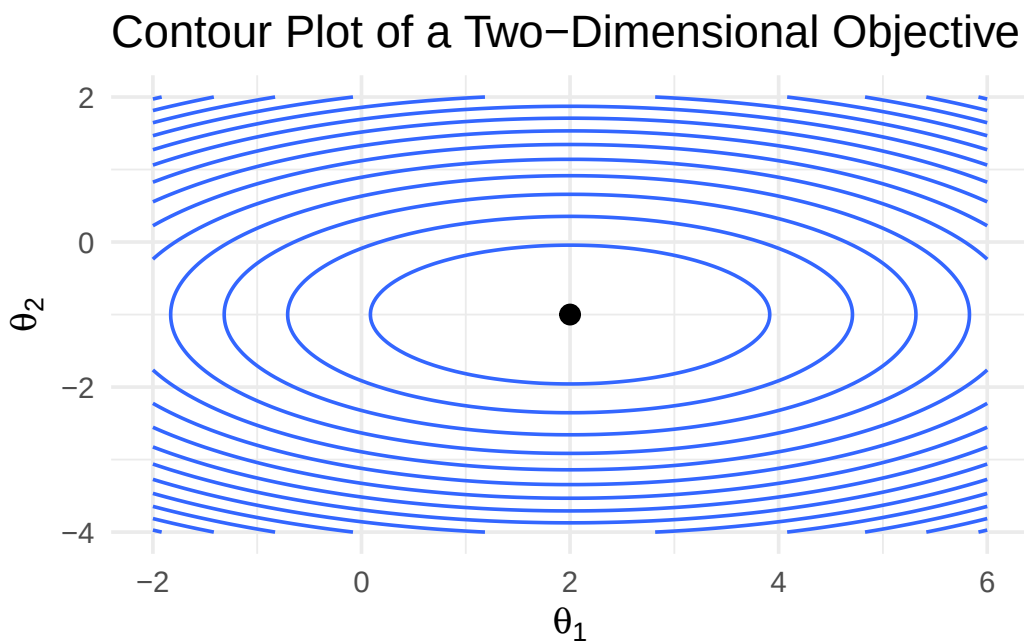
ggplot(
  objective_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_point(
    data = data.frame(
      theta1 = 2,
      theta2 = -1
    )
  )

```

```

),
aes(
  x = theta1,
  y = theta2
),
inherit.aes = FALSE,
size = 3
) +
labs(
  x = expression(theta[1]),
  y = expression(theta[2]),
  title = "Contour Plot of a Two-Dimensional Objective"
)

```



The minimum occurs at

$$\theta^* = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

3.8 Gradient Descent

The gradient points in the direction of greatest increase.

Therefore,

$$-\nabla Q(\theta)$$

is a natural direction for decreasing the objective.

Gradient descent updates

$$\theta_{k+1} = \theta_k - \alpha_k \nabla Q(\theta_k),$$

where $\alpha_k > 0$ is the **step size** or **learning rate**.

For the quadratic example,

$$\nabla Q(\theta_1, \theta_2) = \begin{pmatrix} 2(\theta_1 - 2) \\ 8(\theta_2 + 1) \end{pmatrix}.$$

```
gradient_2d <- function(theta) {  
  c(  
    2 * (theta[1] - 2),  
    8 * (theta[2] + 1)  
  )  
}  
  
gradient_descent <- function(  
  start,  
  alpha = 0.1,  
  maxit = 30  
) {  
  
  theta <- start  
  
  history <- data.frame(  
    iteration = 0,  
    theta1 = theta[1],  
    theta2 = theta[2]  
  )  
  
  for (k in seq_len(maxit)) {  
  
    theta <-  
      theta -
```

```

    alpha * gradient_2d(theta)

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        theta1 = theta[1],
        theta2 = theta[2]
      )
    )
  }

  history
}

gd_path <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.1,
  maxit = 20
)

```

3.8.1 Visualizing the Optimization Path

```

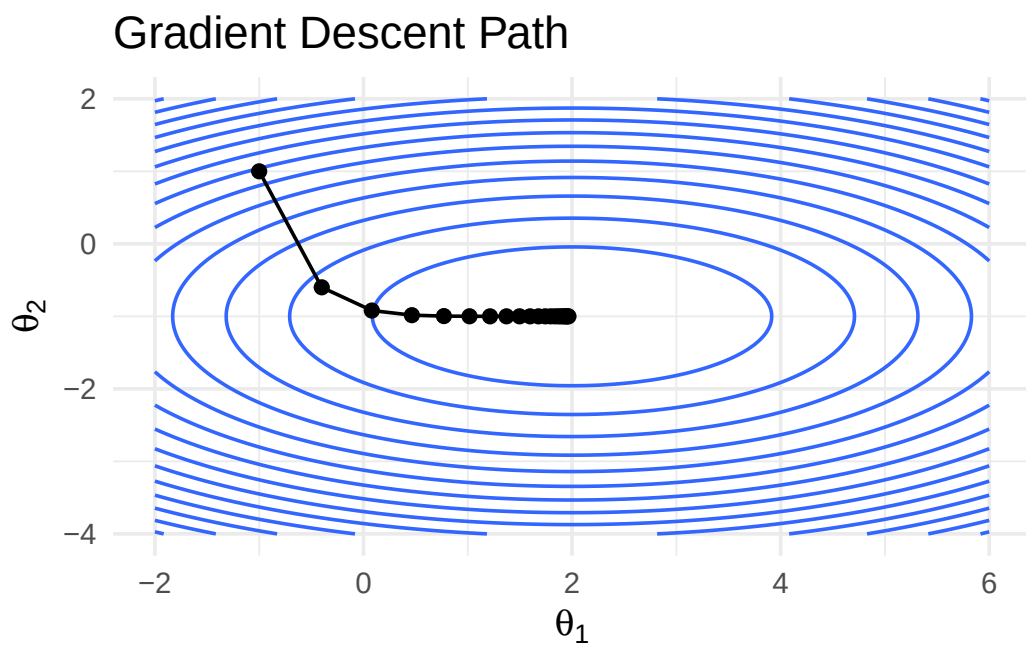
ggplot(
  objective_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_path(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
  ),

```

```

    inherit.aes = FALSE
  ) +
  geom_point(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE,
    size = 2
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),
    title = "Gradient Descent Path"
  )
)

```



What happens if the step size α is too small?

What happens if it is too large?

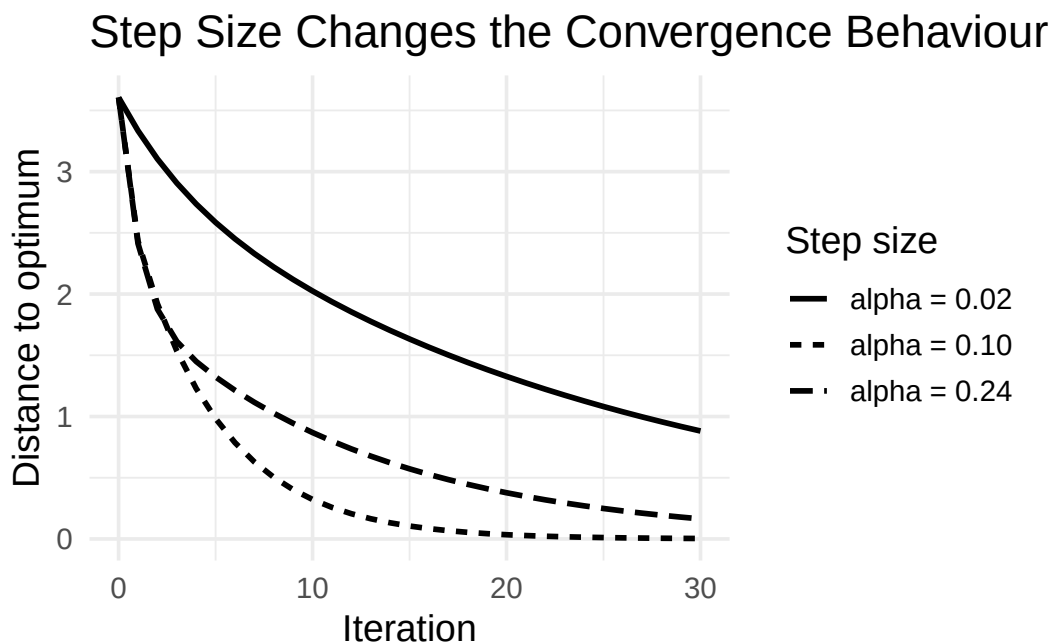
3.8.2 Step Size Matters

```
gd_small <- gradient_descent(  
  start = c(-1, 1),  
  alpha = 0.02,  
  maxit = 30  
)  
  
gd_medium <- gradient_descent(  
  start = c(-1, 1),  
  alpha = 0.1,  
  maxit = 30  
)  
  
gd_large <- gradient_descent(  
  start = c(-1, 1),  
  alpha = 0.24,  
  maxit = 30  
)  
  
gd_compare <- rbind(  
  transform(  
    gd_small,  
    step = "alpha = 0.02"  
  ),  
  transform(  
    gd_medium,  
    step = "alpha = 0.10"  
  ),  
  transform(  
    gd_large,  
    step = "alpha = 0.24"  
  )  
)  
  
gd_compare$error <-  
  sqrt(  
    (gd_compare$theta1 - 2)^2 +  
    (gd_compare$theta2 + 1)^2  
  )  
  
ggplot(  
  data = gd_compare,  
  aes(x = step, y = error)
```

```

gd_compare,
aes(
  x = iteration,
  y = error,
  linetype = step
)
) +
geom_line(linewidth = 1) +
labs(
  x = "Iteration",
  y = "Distance to optimum",
  linetype = "Step size",
  title = "Step Size Changes the Convergence Behaviour"
)

```



3.9 Multivariate Newton's Method

In one dimension,

$$x_{k+1} = x_k - \frac{Q'(x_k)}{Q''(x_k)}.$$

In multiple dimensions, this becomes

$$\theta_{k+1} = \theta_k - H(\theta_k)^{-1} \nabla Q(\theta_k).$$

However, computationally we usually do **not** calculate the inverse explicitly.

Instead, solve

$$H(\theta_k) d_k = -\nabla Q(\theta_k),$$

and update

$$\theta_{k+1} = \theta_k + d_k.$$

Why do we solve a linear system instead of explicitly computing

$$H^{-1}?$$

This connects directly to the earlier discussion of numerical stability and matrix inversion.

3.10 Quasi-Newton Methods

Newton's method requires the Hessian.

For high-dimensional statistical models, computing the Hessian can be expensive.

Quasi-Newton methods approximate curvature information rather than calculating the Hessian directly.

One of the most widely used methods is **BFGS**.

In R:

```
optim(  
  par = start,  
  fn = objective,  
  method = "BFGS"  
)
```

The main computational idea is

Use curvature information without repeatedly computing the full Hessian.

Consider

$$Y_i \sim \text{Bernoulli}(p_i),$$

with

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_i.$$

The parameter is now a vector

$$\beta = (\beta_0, \beta_1)^\top.$$

There is no closed-form MLE, so logistic regression requires numerical optimization.

```
set.seed(8670)

n <- 200

x_logit <- rnorm(n)

beta_true <- c(
  -0.5,
  1.5
)

eta <-
  beta_true[1] +
  beta_true[2] * x_logit

p <- plogis(eta)

y_logit <- rbinom(
  n,
  size = 1,
  prob = p
)
```

Define the negative log-likelihood:

```
logistic_nll <- function(beta) {  
  
  eta <-  
    beta[1] +  
    beta[2] * x_logit  
  
  p <- plogis(eta)  
  
  -sum(  
    dbinom(  
      y_logit,  
      size = 1,  
      prob = p,  
      log = TRUE  
    )  
  )  
}
```

Estimate the coefficients using BFGS:

```
fit_optim <- optim(  
  par = c(0, 0),  
  fn = logistic_nll,  
  method = "BFGS"  
)  
  
fit_optim$par
```

```
[1] -0.381730  1.014505
```

Compare with glm():

```
fit_glm <- glm(  
  y_logit ~ x_logit,  
  family = binomial()  
)  
  
coef(fit_glm)
```

```
(Intercept)      x_logit  
-0.3817297      1.0145052
```

```
fit_optim$par
```

```
[1] -0.381730  1.014505
```

Why is comparing `optim()` with `glm()` useful?

This illustrates the complete computational-statistics workflow:

Statistical Model → Likelihood → Optimization → Algorithm → Validation.

3.11 Scaling and Conditioning

Optimization can behave very differently when parameters operate on different scales.

Consider

$$Q(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 100(\theta_2 - 1)^2.$$

```
scaled_objective <- function(theta1, theta2) {  
  (theta1 - 1)^2 +  
    100 * (theta2 - 1)^2  
}
```

```
scale_grid <- expand.grid(  
  theta1 = seq(  
    -2,  
    4,  
    length.out = 150  
  ),  
  theta2 = seq(  
    -1,  
    3,  
    length.out = 150  
  )  
)
```

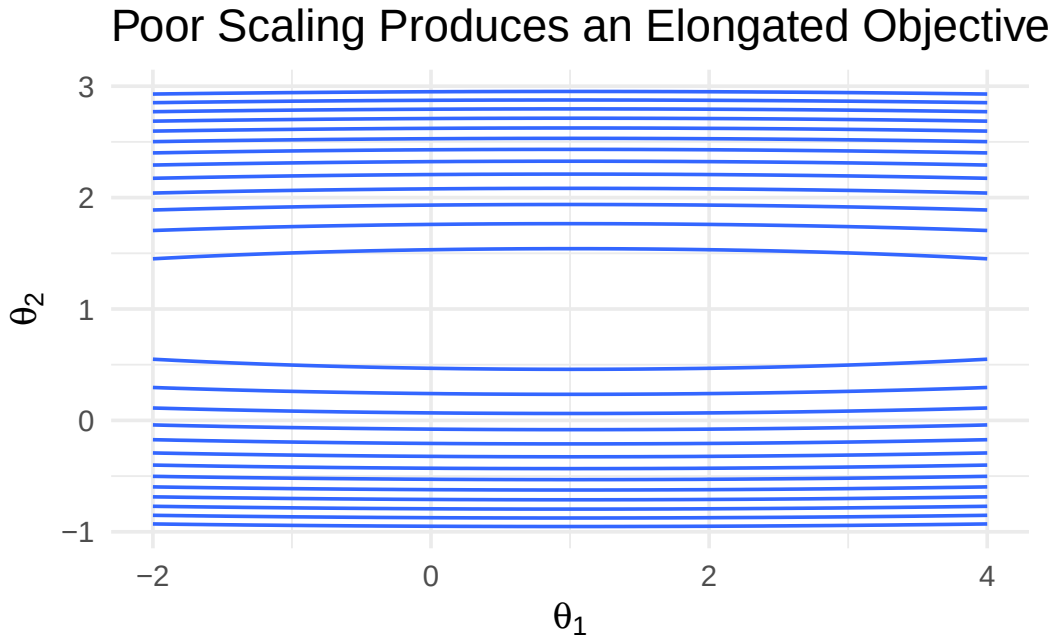
```
scale_grid$value <- with(  
  scale_grid, scaled_objective
```

```

scale_grid,
scaled_objective(
  theta1,
  theta2
)
)

ggplot(
  scale_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),
    title = "Poor Scaling Produces an Elongated Objective"
  )

```



A highly elongated objective can cause gradient-based algorithms to zig-zag and converge slowly.

This illustrates another important principle:

Same mathematical problem + different scaling $\implies$ different computational behaviour.

3.12 Metaheuristic Methods

Deterministic optimization methods can work extremely well for smooth and well-behaved objectives.

However, they may struggle when the objective:

- contains many local optima;
- is not differentiable;
- is discontinuous;
- has a complicated search space.

Metaheuristic methods use broader stochastic exploration.

The key idea is

Explore broadly $\longrightarrow$ search more locally later.

Why Accept a Worse Solution?

A local deterministic method usually attempts to improve the objective at every step.

A stochastic method may deliberately accept a worse solution.

Why would an optimization algorithm ever accept a worse solution?

Temporarily moving to a worse location can allow the algorithm to escape a local optimum and later reach a better region.

3.13 Simulated Annealing

Simulated annealing is a stochastic optimization method inspired by the physical process of heating and slowly cooling a material.

Suppose the current solution is x_k and a candidate solution x' is generated.

For a minimization problem, if

$$Q(x') < Q(x_k),$$

the new solution is accepted.

If

$$Q(x') > Q(x_k),$$

the worse solution may still be accepted with probability

$$P(\text{accept}) = \exp\left(-\frac{\Delta Q}{T}\right),$$

where

$$\Delta Q = Q(x') - Q(x_k) > 0.$$

The parameter T is called the **temperature**.

When

T is large,

the algorithm explores more broadly.

When

T is small,

the algorithm becomes more selective.

Visualizing the Acceptance Probability

```
delta_grid <- seq(
  0,
  5,
  length.out = 300
)

temperature <- c(
  0.1,
  0.5,
  2
)

sa_probability <- expand.grid(
  delta = delta_grid,
  temperature = temperature
)

sa_probability$probability <-
  exp(
    -sa_probability$delta /
    sa_probability$temperature
  )

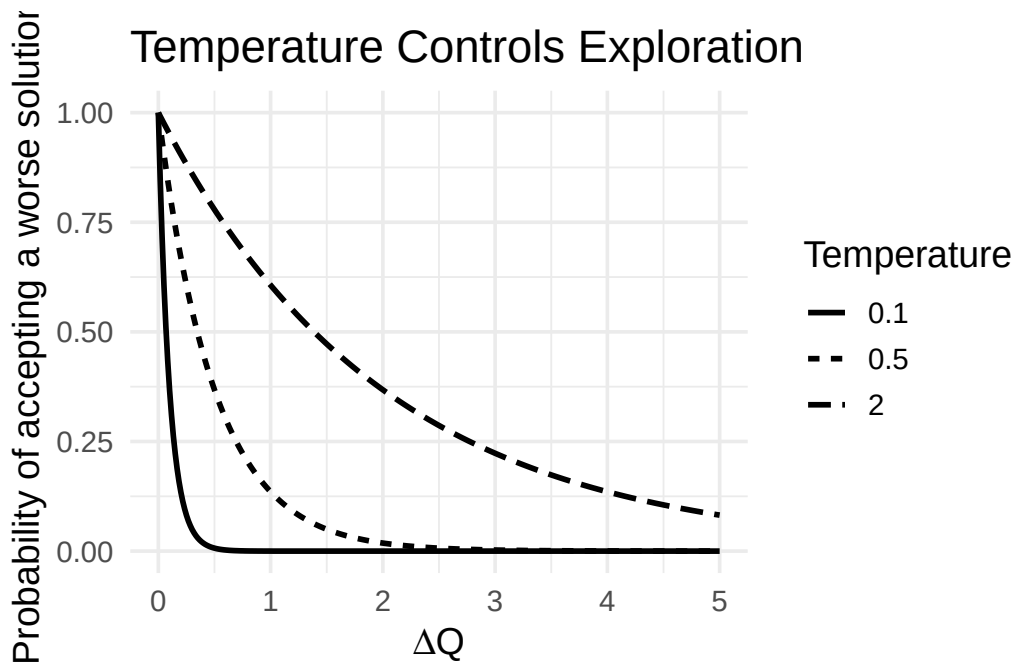
sa_probability$temperature <-
  factor(
    sa_probability$temperature
  )

ggplot(
  sa_probability,
```

```

aes(
  x = delta,
  y = probability,
  linetype = temperature
)
) +
geom_line(linewidth = 1) +
labs(
  x = expression(Delta * Q),
  y = "Probability of accepting a worse solution",
  linetype = "Temperature",
  title = "Temperature Controls Exploration"
)

```



At high temperature, why is the algorithm more willing to accept a poor move?

What happens as the temperature approaches zero?

3.13.1 Simulated Annealing in R

R provides simulated annealing through

```
optim(
  ...,
  method = "SANN"
)
```

Using the earlier multimodal objective:

```
set.seed(8670)

sa_fit <- optim(
  par = 5,
  fn = objective,
  method = "SANN",
  control = list(
    maxit = 3000
  )
)

sa_fit$par
```

```
[1] -0.1958383
```

```
sa_fit$value
```

```
[1] -0.1262269
```

Because simulated annealing is stochastic, different seeds may produce different results.

```
seeds <- 1:20

sa_solution <- vapply(
  seeds,
  function(seed) {

    set.seed(seed)

    optim(
      par = 5,
      fn = objective,
      method = "SANN",
```

```

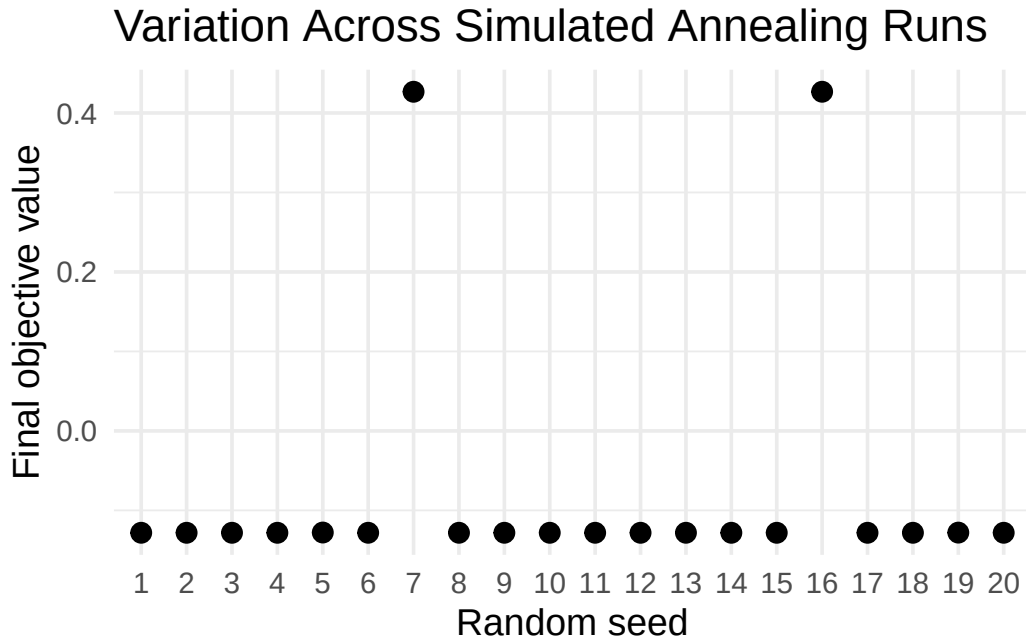
      control = list(
        maxit = 3000
      )
    )$par

  },
  numeric(1)
)

sa_data <- data.frame(
  seed = seeds,
  solution = sa_solution,
  objective = objective(sa_solution)
)

ggplot(
  sa_data,
  aes(
    x = factor(seed),
    y = objective
  )
) +
  geom_point(size = 3) +
  labs(
    x = "Random seed",
    y = "Final objective value",
    title = "Variation Across Simulated Annealing Runs"
  )

```



Advantages of Simulated Annealing

- Can escape local optima.
- Does not require derivatives.
- Easy to adapt to many objective functions.
- Useful for multimodal problems.

Limitations of Simulated Annealing

- Can require many function evaluations.
- Results vary across runs.
- Performance depends on the cooling schedule.
- Does not guarantee the global optimum.

3.14 Genetic Algorithm

A genetic algorithm uses a **population of candidate solutions** rather than one current solution.

The basic cycle is

Selection → Crossover → Mutation → New Generation.

3.14.1 Selection

Better candidate solutions are more likely to remain in the population.

3.14.2 Crossover

Information from two candidate solutions is combined to create new candidates.

3.14.3 Mutation

Random changes are introduced to preserve diversity and explore new regions.

The key difference from simulated annealing is:

SA: one evolving solution GA: a population of solutions.

Advantages

- Explores several regions simultaneously.
- Does not require gradients.
- Flexible for continuous and discrete problems.

Limitations

- Can require many objective evaluations.
- Requires tuning of population size and mutation rate.
- Can converge slowly.
- Does not guarantee a global optimum.

3.15 Particle Swarm Optimization

Particle swarm optimization also maintains a population of candidate solutions called **particles**.

Each particle uses information about:

- its current location;
- its previous velocity;
- its own best location;
- the best location found by the group.

A simplified velocity update is

$$v_{i,k+1} = wv_{i,k} + c_1r_1(p_i - x_{i,k}) + c_2r_2(g - x_{i,k}),$$

followed by

$$x_{i,k+1} = x_{i,k} + v_{i,k+1}.$$

The intuition is

Momentum + Individual Experience + Group Experience + Randomness.

3.15.1 Advantages

- Does not require derivatives.
- Explores multiple regions simultaneously.
- Relatively intuitive and flexible.

3.15.2 Limitations

- May require many function evaluations.
- Sensitive to tuning parameters.
- Can converge prematurely.
- Global optimality is not guaranteed.

3.16 Deterministic versus Metaheuristic Methods

The two families search very differently.

| Method | Type | Derivatives | Search | Main Advantage | Main Limitation |
|-----------|---------------|-------------|--------------|-------------------------|-----------------------------------|
| Bisection | Deterministic | No | Root finding | Very reliable | Slow |
| Newton | Deterministic | Yes | Local | Very fast near solution | Sensitive to derivative and start |

| Method | Type | Derivatives | Search | Main Advantage | Main Limitation |
|---------------------|---------------|------------------|--------------------|-------------------------------|---------------------------|
| Secant | Deterministic | No | Local | Fast without derivative | Less reliable |
| Gradient Descent | Deterministic | Gradient | Local | Simple and scalable | Step size matters |
| BFGS | Deterministic | Usually gradient | Local | Efficient for smooth problems | Can reach local optimum |
| Simulated Annealing | Metaheuristic | No | Global exploration | Can escape local optima | Slow and stochastic |
| Genetic Algorithm | Metaheuristic | No | Population | Flexible search | Computationally expensive |
| Particle Swarm | Metaheuristic | No | Population | Cooperative exploration | Requires tuning |

The important question is not

Which algorithm is the best?

Instead, ask

Which algorithm is appropriate for the structure of the statistical problem?

For a smooth and well-behaved objective,

deterministic methods

are usually preferable.

For a highly multimodal, irregular, or derivative-free objective,

metaheuristic methods

may become useful.

Suppose the objective is smooth, convex, and easy to differentiate.

Would you choose BFGS or simulated annealing?

What changes if the objective has many local minima and no useful derivatives?

3.17 Computational Perspective

The goal is not simply to memorize optimization algorithms.

For a statistical problem, ask:

1. What is the statistical quantity we want?
2. Can it be written as an optimization or root-finding problem?
3. Is there a closed-form solution?
4. Is the parameter scalar or multivariate?
5. Is the objective smooth?
6. Are derivatives available?
7. Are there many local optima?
8. How sensitive is the method to the starting value?
9. Is the computation numerically stable?
10. How can we validate the result?

This brings us back to the central idea of statistical computing:

$$\boxed{\text{Theory} \iff \text{Computation} \iff \text{Practice.}}$$

3.18 Questions to Think at Home

1. Why can a mathematically correct optimization formula still lead to a poor computational implementation?
2. Why is a closed-form solution useful even when numerical optimization will eventually be used?
3. Why is bisection more reliable than Newton's method, and what is the computational cost of that reliability?
4. What additional information does Newton's method use compared with bisection?
5. Why can different starting values lead to different solutions for the same objective function?
6. What role does the step size play in gradient descent?
7. Why might poor scaling make optimization difficult?
8. Why is solving

$$Hd = -\nabla Q$$

usually preferable to explicitly computing H^{-1} ?

9. Why can BFGS be useful when the Hessian is expensive to compute?
10. Why might a metaheuristic algorithm deliberately accept a worse solution?
11. When would you prefer simulated annealing over a deterministic local optimizer?
12. What is the main difference between simulated annealing, genetic algorithms, and particle swarm optimization?
13. Why does fewer iterations not necessarily imply lower computational cost?
14. How are optimization, likelihood estimation, and statistical inference connected?

3.19 Takeaways

- Many statistical estimators are solutions to optimization problems.
- Closed-form solutions are convenient but often unavailable.
- Numerical optimization connects statistical theory with practical computation.
- Root finding is closely related to optimization.
- Bisection is reliable but relatively slow.
- Newton's method is fast near a solution but uses derivative information.
- The secant method approximates derivative information numerically.
- Multivariate optimization replaces derivatives with gradients and Hessians.
- Gradient descent uses local first-order information.
- Newton's method uses curvature information through the Hessian.
- BFGS approximates curvature without repeatedly computing the full Hessian.
- Scaling can strongly affect computational performance.
- Deterministic methods are usually efficient for smooth, well-behaved objectives.
- Metaheuristic methods trade speed and predictability for broader exploration.
- Simulated annealing can escape local optima by occasionally accepting worse solutions.
- Genetic algorithms and particle swarm optimization use populations of candidate solutions.
- There is no universally best optimization algorithm.
- A good computational method balances accuracy, stability, speed, and exploration.

3.20 Optimal topic: Real-World Applications

The applications in this chapter illustrate two uses of optimization: **estimating unknown parameters from data** and **choosing an action under practical constraints**. We will connect these ideas to drug development, investment portfolios, factory scheduling, and advertising budgets. The examples use simplified models of these real applications.

Biomedical: Finding a Curve That Fits the Measurements

The situation. A research team measures drug concentration and the improvement in a biomarker. At low concentrations, more drug may produce a much larger response. At high concentrations, the response may level off. The team wants a curve that describes this pattern.

The computer's job: adjust the curve until its predictions are close to the observed dots. The following measurements are invented for illustration.

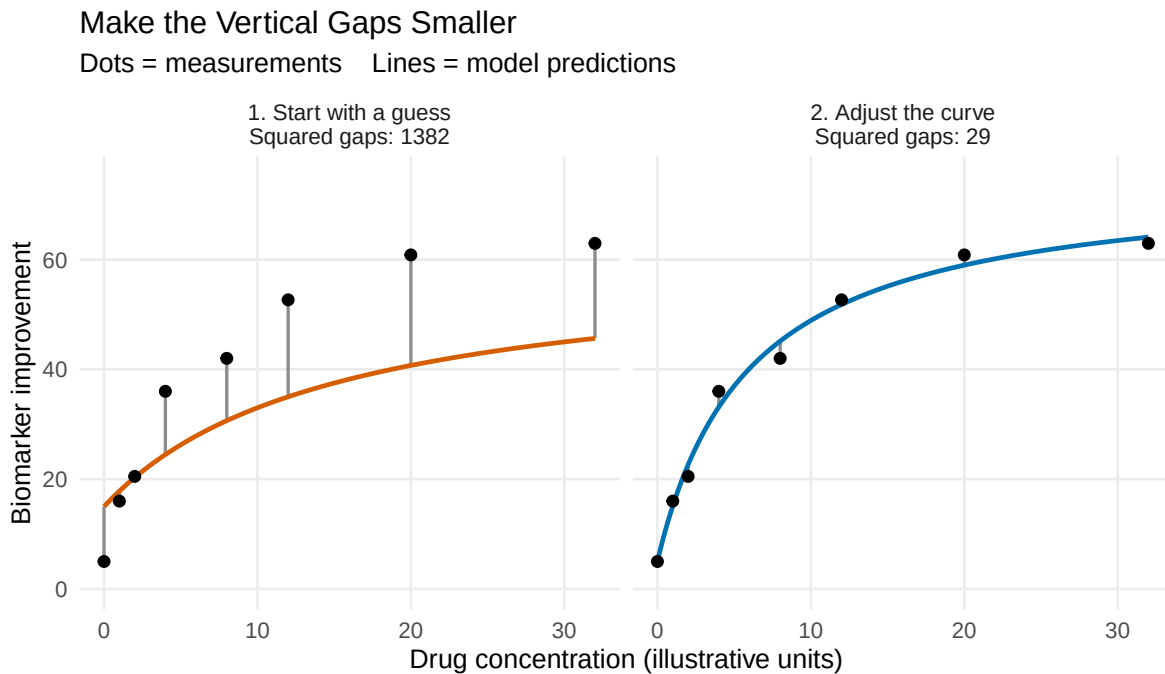


Figure 3.1: The same illustrative measurements, before and after fitting. Vertical gaps show prediction errors. Optimization reduces the sum of their squares from about 1382 to 29.

Read the picture. The dots stay fixed. The optimizer changes the curve's starting height, upper level, and how quickly it bends. Shorter vertical gaps mean better predictions of these measurements.

Decision variables. Choose the three parameters of the curve

$$m(c; E_0, E_{\max}, EC_{50}) = E_0 + \frac{E_{\max}c}{EC_{50} + c}.$$

- E_0 is the starting height (response at concentration zero).
- $E_{\max}$ is the largest increase above that starting height.
- EC_{50} is the concentration at which half of that increase is reached.

The eight concentrations c_i and measured responses y_i in the figure are **fixed data**, not quantities the optimizer can change.

Objective function. Minimize the sum of squared prediction errors:

$$\min_{E_0, E_{\max}, EC_{50}} Q(E_0, E_{\max}, EC_{50}) = \sum_{i=1}^8 \left[y_i - E_0 - \frac{E_{\max}c_i}{EC_{50} + c_i} \right]^2.$$

Constraints. For this increasing, saturating response curve, require

$$E_0 \in \mathbb{R}, \quad E_{\max} > 0, \quad EC_{50} > 0.$$

We impose no sign restriction on the baseline E_0 . In the R code, setting $E_{\max} = e^{\theta_2}$ and $EC_{50} = e^{\theta_3}$ enforces positivity automatically, while $E_0 = \theta_1$ remains unrestricted.

Optimal solution (numerical). For the illustrative measurements, the fitted parameters are approximately

$$(\widehat{E}_0, \widehat{E}_{\max}, \widehat{EC}_{50}) = (5.0815, 70.0297, 5.9845).$$

Thus, the fitted curve and its objective value are

$$\widehat{m}(c) = 5.0815 + \frac{70.0297c}{5.9845 + c}, \quad Q(\widehat{E}_0, \widehat{E}_{\max}, \widehat{EC}_{50}) \approx 28.8719.$$

The minimum found numerically corresponds to the small squared gaps in the right panel. Because this is a nonlinear fitting problem, BFGS convergence alone does not certify a global minimum; different starting values can lead to different results.

Why an algorithm? Moving one parameter changes many predictions at once. The parameters enter nonlinearly, so ordinary linear regression cannot estimate all three together. BFGS, used above, repeatedly proposes changes that reduce the total squared gap.

Exposure-response models help researchers plan further drug studies; dose selection also requires safety information and uncertainty estimates. [Application context: FDA exposure-response guidance](#).

Try explaining it: In the right panel, what has changed: the measurements, the model parameters, or both?

Finance: Splitting Money Between Two Funds

The situation. Suppose a fund manager has \$10,000 to invest. Begin with just two funds and the following illustrative annual estimates:

| Choice | Expected return | Volatility (standard deviation) |
|--------|-----------------|---------------------------------|
| Fund A | 8% | 20% |
| Fund B | 4% | 6% |

Fund A offers more expected return, but its returns vary more. Assume a return correlation of 0.10. The manager wants **the highest expected return while keeping portfolio volatility at or below 10%**. Volatility measures variation; a 10% limit does not cap possible losses at 10%.

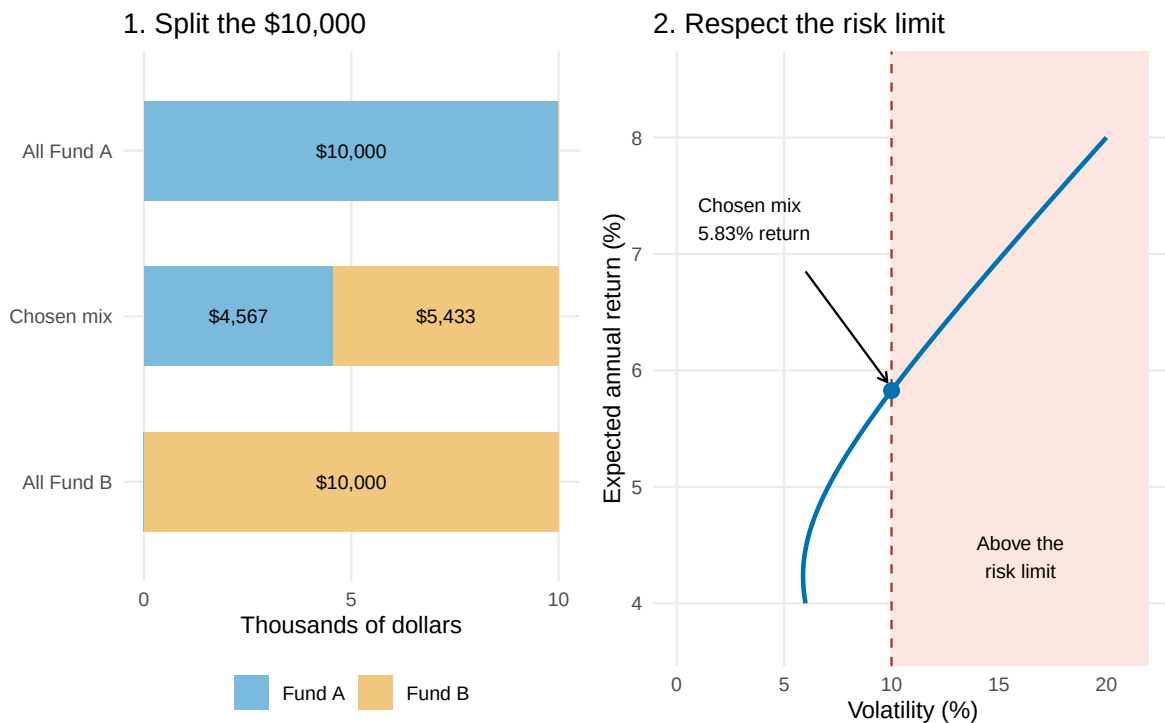


Figure 3.2: Left: three ways to split the same \$10,000. Right: each point on the curve is a possible split. The selected mix has the highest expected return among points at or left of the 10% volatility limit. All inputs are illustrative.

Read the picture. Moving more money to Fund A moves us toward higher expected return, but eventually crosses the dashed risk limit. The chosen mix puts about **\$4,567 in A** and **\$5,433 in B**, for an expected return of **5.83%** under these assumptions. All-A has a higher expected return, but is outside the allowed region.

Decision variable. Let w be the fraction of the \$10,000 invested in Fund A. The remaining fraction, $1 - w$, is invested in Fund B.

Objective function. Maximize the expected annual return rate:

$$\max_w R(w) = 0.08w + 0.04(1 - w) = 0.04 + 0.04w.$$

Constraints. All money is invested, with no borrowing or short selling, and portfolio volatility must not exceed 10%:

$$0 \leq w \leq 1, \quad \sigma^2(w) \leq 0.10^2 = 0.01.$$

The weights w and $1 - w$ already sum to one, so they enforce the \$10,000 budget. Using the two standard deviations and the correlation of 0.10, the variance is explicitly

$$\begin{aligned}\sigma^2(w) &= (0.20w)^2 + [0.06(1 - w)]^2 \\ &\quad + 2(0.10)(0.20)(0.06)w(1 - w) \\ &= 0.0412w^2 - 0.0048w + 0.0036.\end{aligned}$$

Optimal solution. Expected return increases with w , so choose the largest feasible weight. It lies on the risk boundary:

$$0.0412w^2 - 0.0048w + 0.0036 = 0.01.$$

The positive root gives

$$w^* \approx 0.4566657, \quad 1 - w^* \approx 0.5433343.$$

Invest approximately **\$4,566.66 in Fund A** and **\$5,433.34 in Fund B**. At the optimum,

$$R(w^*) \approx 0.0582666 = 5.82666\%, \quad \sigma(w^*) = 10\%.$$

Equivalently, this return corresponds to an expected annual gain of about **\$582.67** on the \$10,000 investment. This is the unique global maximum: within $0 \leq w \leq 1$, every feasible weight lies between zero and w^* , and $R(w)$ is strictly increasing.

Why an algorithm? In this two-fund example, `uniroot()` locates the boundary $\sigma(w) = 0.10$. With 20 funds, sector limits, and holding limits, many weights must be chosen together. Constrained portfolio solvers handle those interacting choices. [Application context: MOSEK portfolio optimization cookbook](#).

Try explaining it: If the dashed risk limit moved to the right, would the best allowed mix contain more or less of Fund A?

Industry: Finish the Same Orders Earlier

The situation. A small factory has three orders, A, B, and C. Every order must go through **Cutting, then Finishing**. Each machine handles one order at a time. These illustrative processing times stay the same in both schedules:

| Order | Cutting | Finishing |
|-------|---------|-----------|
| A | 4 hours | 1 hour |
| B | 1 hour | 4 hours |
| C | 2 hours | 2 hours |

The computer's job: choose the order of work so that all three orders finish as early as possible.

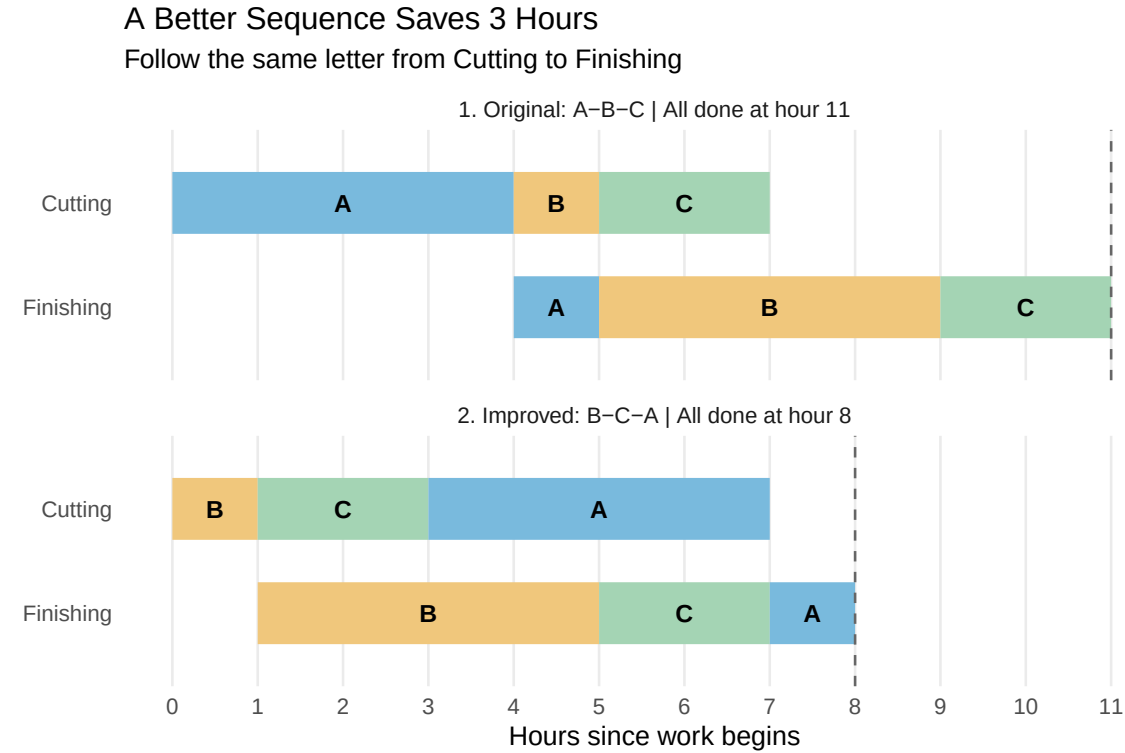


Figure 3.3: Same orders, same machines, same processing times. Changing the sequence reduces the last completion time from 11 to 8 hours. Empty horizontal space is idle time.

Read the picture. In the top schedule, Finishing waits four hours for A. In the bottom schedule, the short cutting step for B lets Finishing start after just one hour. The machines then overlap useful work more effectively. **No machine runs faster; the sequence is better.**

Decision variables. Choose a start time S_{jm} for each order $j \in \{A, B, C\}$ on each machine $m \in \{C, F\}$, where C means Cutting and F means Finishing. Also choose $C_{\max}$, the time by which all orders must be complete. Let p_{jm} denote the fixed processing times in the table. All orders are available at time zero, and each operation runs without interruption once it starts.

Objective function. Minimize the last completion time (the makespan):

$$\min_{\{S_{jm}\}, C_{\max}} C_{\max}.$$

Constraints. The start times must satisfy four rules:

1. **Work cannot start before time zero:** for every order j and machine m ,

$$S_{jm} \geq 0.$$

2. **Cutting precedes Finishing:** for each order j ,

$$S_{jF} \geq S_{jC} + p_{jC}.$$

3. **A machine cannot process two orders at once:** for every pair of different orders j, k and each machine m ,

$$S_{jm} + p_{jm} \leq S_{km} \quad \text{or} \quad S_{km} + p_{km} \leq S_{jm}.$$

In words, either j finishes before k starts, or k finishes before j starts. Choosing between these alternatives determines the job sequence.

4. **Every order finishes by $C_{\max}$:** for each order j ,

$$S_{jF} + p_{jF} \leq C_{\max}.$$

Optimal solution. One optimal sequence is **B, then C, then A** on both machines, with the following start and end times:

| Order | Cutting: start to end | Finishing: start to end |
|-------|-----------------------|-------------------------|
| B | 0 to 1 | 1 to 5 |
| C | 1 to 3 | 5 to 7 |
| A | 3 to 7 | 7 to 8 |

All times are in hours. This feasible schedule achieves

$$C_{\max}^* = 8 \text{ hours.}$$

Why is 8 hours globally optimal? Finishing requires $1 + 4 + 2 = 7$ hours of work on a single machine. It cannot begin before hour 1, because even the shortest cutting operation takes 1 hour. Therefore every feasible schedule satisfies $C_{\max} \geq 1 + 7 = 8$. The schedule above reaches that lower bound, so no schedule can finish earlier. The original A-B-C schedule takes 11 hours.

For three orders, all six common orderings can also be checked. Larger factories require more systematic search; 20 jobs already give $20! \approx 2.43 \times 10^{18}$ possible orderings on one machine.

Why an algorithm? Constraint programming and mixed-integer optimization search subject to the rules. Simulated annealing and genetic algorithms can also explore valid schedules, though they do not certify the best possible result. This is a discrete choice problem: swapping two jobs is not a gradient step. [Application context: OR-Tools job-shop scheduling](#).

Try explaining it: Find B in both panels. Why does doing its short cutting step early help the whole factory?

An online retailer has **\$100,000** for search advertising and social media. Start with a familiar decision: should it spend \$50,000 on each, or split the money differently?

Business Analytics: Same Budget, More Orders

The situation. The first dollars spent on search work well, but that channel soon reaches many of the customers it can reach. Social media responds more gradually. We use two invented response curves to make the allocation problem visible; these are not measured campaign results.

First, look at how each channel responds to spending. A steep curve means that another dollar produces a relatively large gain. A flat curve means that another dollar adds little.

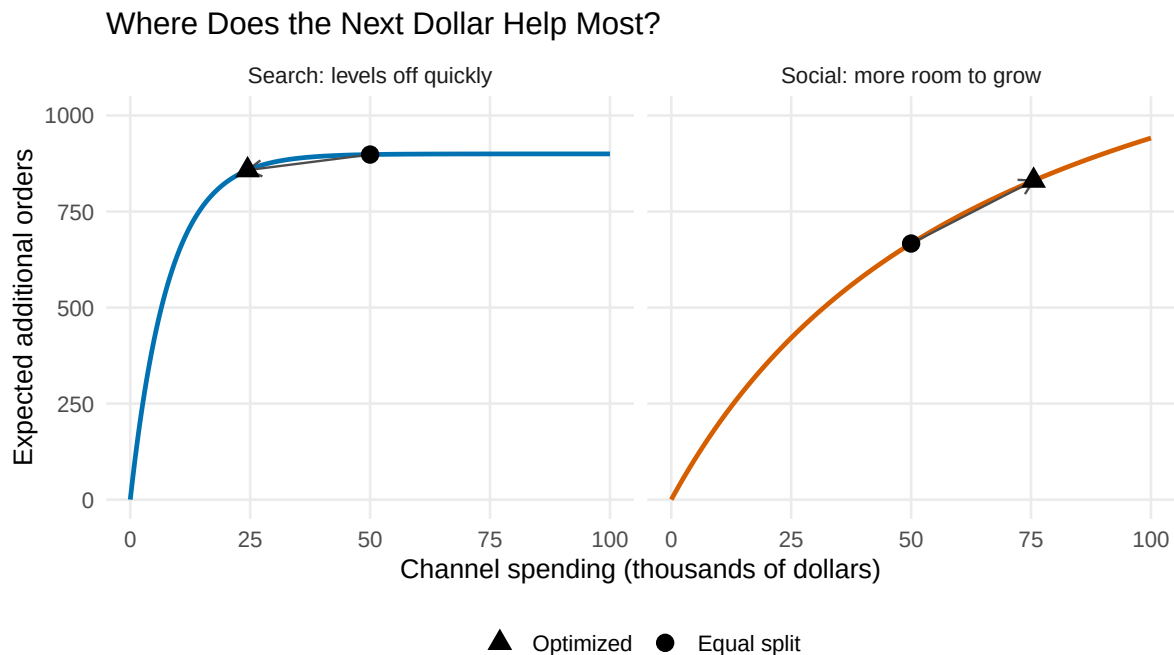


Figure 3.4: Illustrative response curves. Circles mark equal spending; triangles mark the optimized allocation. Arrows show the change: reduce search spending where the curve is flat and increase social spending where more gains remain.

Read the picture. At the equal split, search is already nearly flat. Moving some of that money to social loses relatively few search orders and gains more social orders. The optimizer keeps reallocating until the next dollar has the same marginal benefit in both channels.

Now compare the decisions. The total budget stays at \$100,000 in both rows. Only its allocation changes.

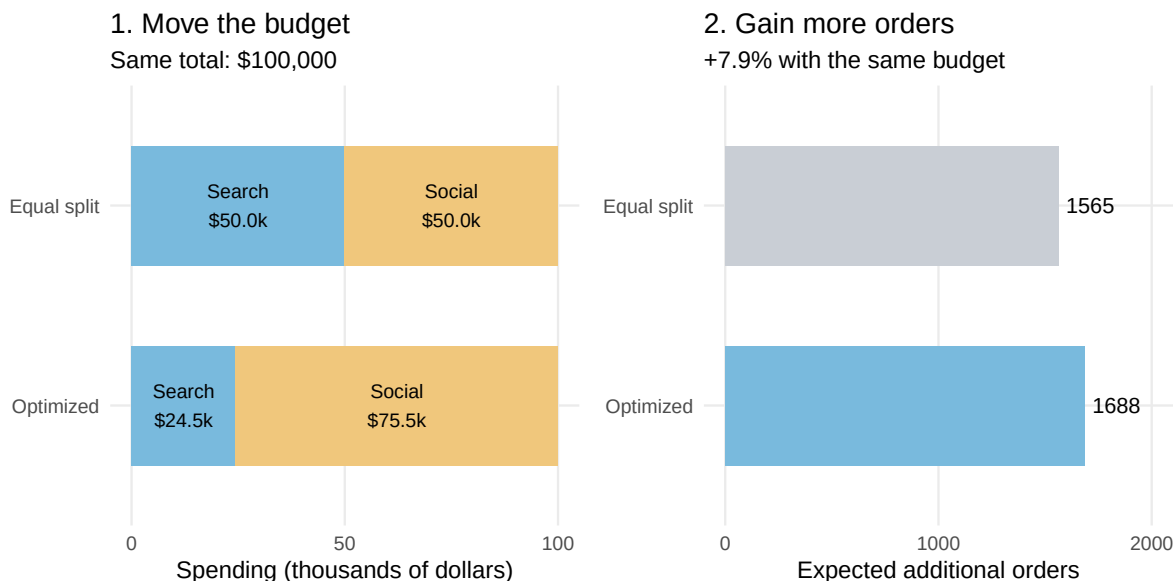


Figure 3.5: A visual before-and-after comparison. The optimized split spends about \$24,461 on search and \$75,539 on social, producing about 123 more expected orders with the same budget under the illustrative model.

The change is from about **1,565 to 1,688 expected additional orders**: approximately **123 extra orders, or 7.9%**, without increasing spending.

Decision variables. Let s and u be spending on search and social advertising, respectively, in thousands of dollars. For example, $s = 25$ means \$25,000 spent on search.

Objective function. Maximize the expected number of additional orders:

$$\max_{s,u} F(s,u) = 900(1 - e^{-s/8}) + 1600 \frac{u}{70 + u}.$$

The first term is the search contribution and the second is the social contribution. We assume these contributions add without overlap and that all orders have the same value.

Constraints. Spend the full \$100,000 budget, with neither channel receiving a negative amount:

$$s \geq 0, \quad u \geq 0, \quad s + u = 100.$$

There are no additional channel spending limits in this example. Substituting $u = 100 - s$ gives the equivalent problem used by `optimize()`:

$$\max_{0 \leq s \leq 100} f(s) = 900(1 - e^{-s/8}) + 1600 \frac{100 - s}{170 - s}.$$

Optimal solution. The best allocation is approximately

$$s^* = 24.46073, \quad u^* = 75.53927,$$

or **\$24,460.73 on search and \$75,539.27 on social**. Its objective value is

$$F(s^*, u^*) \approx 1688.1476 \text{ expected additional orders.}$$

For comparison, equal spending gives $F(50, 50) \approx 1564.9293$. The optimal allocation therefore adds about **123.2183 expected orders**, a **7.87%** increase with the same budget.

Why is this allocation optimal? Both response curves are strictly concave, so the feasible problem has a unique global maximum. The optimum lies inside $0 < s < 100$ and solves

$$f'(s) = 112.5e^{-s/8} - \frac{112000}{(170 - s)^2} = 0.$$

At s^* , both channels have the same marginal gain: approximately 5.2876 additional orders per \$1,000 of spending. Shifting a small amount of money from one channel to the other can no longer improve the total.

Why an algorithm? Setting $u = 100 - s$ leaves one adjustable number. `optimize()` searches for its best value. Here the total-response curve is strictly concave, so there is a unique maximum. With many channels and spending limits, this becomes a multivariate constrained problem.

Marketing tools such as Google's Meridian apply this response-curve idea to budget allocation. In practice, the curves must be estimated and validated; the optimizer's result depends on their accuracy. [Application context: Meridian budget optimization.](#)

Try explaining it: Point to the two equal-length budget bars. Where do the extra orders come from if no extra money is spent?

Examples are adapted from the following sources:

- Peng, R.D. [Advanced Statistical Computing](#).

4 Generating Random Variable

In the previous chapter, we used computation to estimate parameters from data. Here we go in the other direction: **given a probability model, how do we generate observations from it?**

We start with uniform random numbers, use them to construct one-dimensional distributions, and then build mixtures and random vectors. At each step, we ask the same three questions: **What is the construction? Why does it have the desired distribution? How can we check it?**

4.1 Overview

Generating random variables is the first step in a simulation study. Once we can generate data with a known truth, we can study an estimator's bias, a confidence interval's coverage, or a test's power.

The main methods in this chapter follow a common pattern:

| Available building block | Construction | Target |
|--------------------------------|-------------------------------------|---|
| Uniform probability levels | Apply a quantile function | A specified univariate distribution |
| An easy proposal distribution | Accept or reject proposals | A distribution with a known density |
| Familiar independent variables | Transform or combine them | Related distributions, sums, and mixtures |
| Independent standard normals | Apply a matrix or normalize lengths | Correlated vectors and random directions |

We develop these constructions first. The final **optional applications** use them in complete statistical experiments.

A pseudorandom number generator produces a deterministic sequence from an initial state. A well-designed generator makes that sequence useful as an approximation to independent random draws. In the mathematical arguments below, we work with ideal independent uniforms; software approximates them with finite precision.

Reproducibility

In R, use `set.seed()` to reproduce an example. The same seed, generator settings, software environment, and sequence of calls reproduce the same simulation. The seed identifies a computation; it does not certify the probability model.

```
set.seed(8670)
u_first <- runif(5)

set.seed(8670)
u_repeated <- runif(5)

rbind(first = u_first, repeated = u_repeated)
#>           [,1]      [,2]      [,3]      [,4]      [,5]
#> first      0.3114911 0.7752026 0.874258 0.7551533 0.9447904
#> repeated 0.3114911 0.7752026 0.874258 0.7551533 0.9447904
identical(u_first, u_repeated)
#> [1] TRUE
```

Set the seed **before a simulation experiment**, then let the generator advance. Resetting the same seed inside every replication creates repeated copies of the same random input. For a reproducible analysis, also record `RNGkind()` and `sessionInfo()`. For parallel work, use a framework that manages separate random-number streams, such as the streams supported by R's `parallel` package. See the [R documentation on random-number generation](#).

Example 1. Sampling from a Finite Population

Sampling with replacement allows an observation to appear more than once. Sampling without replacement makes later draws depend on which observations have already been selected.

```
set.seed(8671)
observed_waits <- c(2, 3, 3, 5, 7, 12) # illustrative waiting times

sample(observed_waits, size = 6, replace = TRUE)
#> [1] 3 3 7 5 2 3
sample(observed_waits, size = 6, replace = FALSE)
#> [1] 7 3 12 5 3 2
```


Connection to the Bootstrap

The empirical distribution places probability $1/n$ on each of the n observed records, counting repeated values separately. Drawing n times independently from this distribution is the basic **nonparametric bootstrap** resampling step. Sampling all n records without replacement only permutes them, so the sample mean cannot change.

This is our first example of an important principle: **the sampling rule is part of the statistical model.**

R Distribution Functions: A Reference

R uses four related prefixes. For example, `dnorm()` evaluates a normal density, `pnorm()` evaluates its CDF, `qnorm()` evaluates its quantile function, and `rnorm()` generates draws.

| Distribution | Density or PMF | CDF | Quantile | Generator | Parameters to check |
|-------------------|----------------------|----------------------|----------------------|----------------------|--|
| Uniform | <code>dunif</code> | <code>punif</code> | <code>qunif</code> | <code>runif</code> | <code>min</code> , <code>max</code> |
| Normal | <code>dnorm</code> | <code>pnorm</code> | <code>qnorm</code> | <code>rnorm</code> | <code>mean</code> , <code>sd</code> |
| Exponential | <code>dexp</code> | <code>pexp</code> | <code>qexp</code> | <code>rexp</code> | <code>rate</code> |
| Gamma | <code>dgamma</code> | <code>pgamma</code> | <code>qgamma</code> | <code>rgamma</code> | <code>shape</code> , <code>rate</code> or <code>scale</code> |
| Beta | <code>dbeta</code> | <code>pbeta</code> | <code>qbeta</code> | <code>rbeta</code> | <code>shape1</code> , <code>shape2</code> |
| Binomial | <code>dbinom</code> | <code>pbinom</code> | <code>qbinom</code> | <code>rbinom</code> | <code>size</code> , <code>prob</code> |
| Geometric | <code>dgeom</code> | <code>pgeom</code> | <code>qgeom</code> | <code>rgeom</code> | <code>prob</code> ; counts failures |
| Poisson | <code>dpois</code> | <code>ppois</code> | <code>qpois</code> | <code>rpois</code> | <code>lambda</code> |
| Negative binomial | <code>dnbinom</code> | <code>pnbinom</code> | <code>qnbinom</code> | <code>rnbinom</code> | <code>size</code> , <code>prob</code> or <code>mu</code> |
| Chi-square | <code>dchisq</code> | <code>pchisq</code> | <code>qchisq</code> | <code>rchisq</code> | <code>df</code> |
| Student's t | <code>dt</code> | <code>pt</code> | <code>qt</code> | <code>rt</code> | <code>df</code> |
| F | <code>df</code> | <code>pf</code> | <code>qf</code> | <code>rf</code> | <code>df1</code> , <code>df2</code> |
| Lognormal | <code>dlnorm</code> | <code>plnorm</code> | <code>qlnorm</code> | <code>rlnorm</code> | <code>meanlog</code> , <code>sdlog</code> |

See [R's distribution reference](#) for the available families.

Check the Parameterization

`rnorm(n, sd = 4)` has variance 16. An exponential rate of 4 gives mean $1/4$. In the gamma family, `scale = 1 / rate`. A lognormal `meanlog` is the mean of $\log X$, not the

mean of X .

A generator can run without errors while representing the wrong model if its parameterization is misunderstood.

4.2 Inverse Transformation Method

Imagine dividing the interval $(0, 1)$ into probability levels. A uniform draw selects a level without favoring one part of that interval. The quantile function converts that probability level into a value on the scale of the target distribution.

Definition 1. Generalized Inverse CDF

For any CDF F , define

$$F^{-1}(u) = \inf\{x \in \mathbb{R} : F(x) \geq u\}, \quad 0 < u < 1.$$

It is the first value of x at which accumulated probability reaches the level u .

Theorem 1. Inverse Transformation

If $U \sim \text{Unif}(0, 1)$, then $X = F^{-1}(U)$ has CDF F . Independent uniforms transformed separately give independent observations from F .

Proof. The generalized inverse satisfies

$$\begin{aligned} P\{F^{-1}(U) \leq x\} &= P\{U \leq F(x)\} \\ &= F(x). \end{aligned}$$

Thus the generated values have the required CDF.

Use an Inequality in the Definition

The condition is $F(x) \geq u$, not $F(x) = u$. A discrete CDF jumps over some probability levels. The generalized inverse still assigns every $0 < u < 1$ to an outcome.

Theorem 2. Probability Integral Transform

If X has a **continuous** CDF F , then $F(X) \sim \text{Unif}(0, 1)$. This is the reverse direction of the inverse-transform construction.

i A Connection to Model Checking

Under a correctly specified continuous predictive distribution, transformed observations have a uniform marginal distribution. Dependence must be checked separately. If F is estimated from the same observations, formal tests need to account for that estimation. For discrete X , $F(X)$ is not uniform. An optional randomized version spreads each probability mass over its corresponding interval:

$$U = F(X^-) + V\{F(X) - F(X^-)\},$$

where V is an independent uniform and $F(X^-)$ is the CDF just below X .

Algorithm 1. Inverse Transformation

1. Obtain the quantile function F^{-1} analytically or numerically.
2. Generate independent $U_1, \dots, U_n \sim \text{Unif}(0, 1)$.
3. Return $X_i = F^{-1}(U_i)$ for each i .

4.2.1 Continuous Distributions

Example 2. An Increasing Density on the Unit Interval

Consider

$$f(x) = 3x^2, \quad 0 < x < 1.$$

Within the support, $F(x) = x^3$, so $F^{-1}(u) = u^{1/3}$. Therefore,

$$X = U^{1/3}.$$

Why do the observations concentrate near 1? Equal-width intervals near 1 contain more probability because the CDF rises faster there. The inverse maps more uniform probability levels into those intervals.

The arrows in Figure 4.1 show what the inverse does: choose a height u , move across to the CDF, then read the corresponding x on the horizontal axis.

```
set.seed(8672)
n <- 5000
u <- runif(n)
x_power <- u^(1 / 3)
```

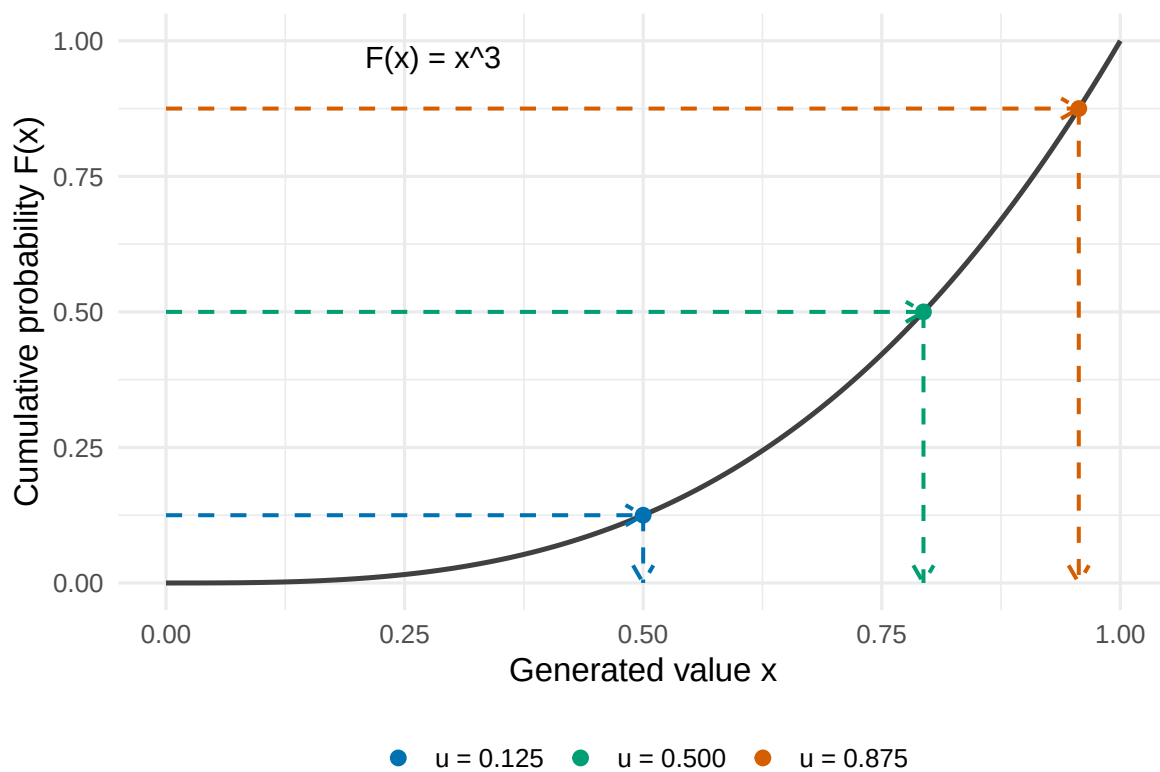


Figure 4.1: The inverse CDF converts a probability level into an observation. Each horizontal arrow starts at u ; the downward arrow lands at $x = u^{1/3}$. The dashed guides are a construction, not additional random draws.

```

plot_data <- rbind(
  data.frame(value = u, distribution = "1. Uniform input"),
  data.frame(value = x_power, distribution = "2. Transformed output")
)
grid <- seq(0, 1, length.out = 301)
curve_data <- rbind(
  data.frame(value = grid, density = 1,
             distribution = "1. Uniform input"),
  data.frame(value = grid, density = 3 * grid^2,
             distribution = "2. Transformed output")
)

ggplot(plot_data, aes(value)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                boundary = 0, fill = "lightblue", color = "white") +
  geom_line(data = curve_data, aes(y = density),
            color = "firebrick", linewidth = 1) +
  facet_wrap(~ distribution) +
  labs(x = "Value", y = "Density")

```

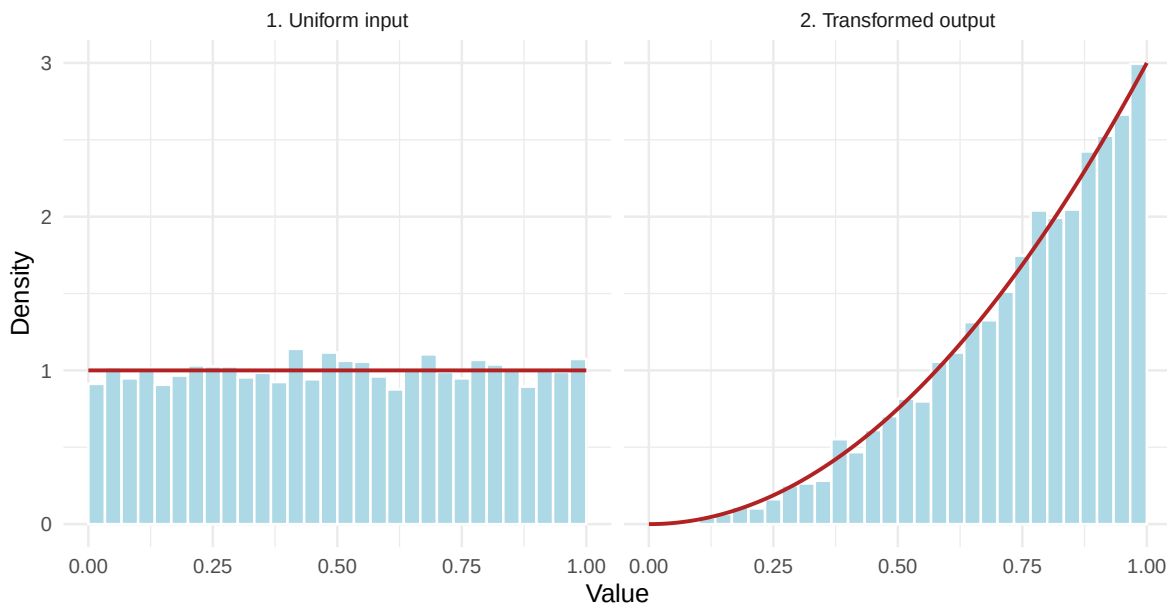


Figure 4.2: Left: uniform probability levels. Right: mapping them through the quantile function produces values with the increasing target density.

Here $\mathbb{E}(X) = 3/4$ and $\text{Var}(X) = 3/80$. A simulation provides an independent numerical check.

```
knitr::kable(
  data.frame(
    quantity = c("Mean", "Variance"),
    simulated = c(mean(x_power), var(x_power)),
    theoretical = c(3 / 4, 3 / 80)
  ),
  digits = 4
)
```

| quantity | simulated | theoretical |
|----------|-----------|-------------|
| Mean | 0.7532 | 0.7500 |
| Variance | 0.0365 | 0.0375 |

Agreement should be approximate. Random samples are not required to reproduce population moments exactly.

Example 3. Exponential Random Variables

For $T \sim \text{Exp}(\lambda)$ with **rate** $\lambda > 0$,

$$F(t) = 1 - e^{-\lambda t}, \quad F^{-1}(u) = -\frac{\log(1 - u)}{\lambda}.$$

Hence we can generate waiting times using

$$T = -\frac{\log(1 - U)}{\lambda}.$$

Since U and $1 - U$ have the same distribution, $-\log(U)/\lambda$ also works. Both formulas have the same marginal distribution.

```
set.seed(8673)
lambda <- 0.7
u <- runif(10000)
waiting_time <- -log1p(-u) / lambda

c(
  simulated_mean = mean(waiting_time),
  theoretical_mean = 1 / lambda,
```

```

simulated_prob_above_3 = mean(waiting_time > 3),
theoretical_prob_above_3 = exp(-lambda * 3)
)
#>           simulated_mean           theoretical_mean      simulated_prob_above_3
#>           1.4335222           1.4285714           0.1241000
#> theoretical_prob_above_3
#>           0.1224564

```

$\log 1p(-u)$ computes $\log(1-u)$ accurately when u is close to zero. This connects random-number generation to the numerical stability issues discussed earlier.

Equal Distributions Do Not Mean Independent Draws

The formulas $-\log(U)/\lambda$ and $-\log(1-U)/\lambda$ have the same marginal distribution. Applying both to the **same** U produces dependent outputs. Use independent uniforms when independence is required.

Optional Connection: Survival and Waiting Times

Exponential waiting times model the time between events in a homogeneous Poisson process. The constant rate is a substantive assumption: it implies a memoryless waiting time. Equipment aging or time-varying arrival rates may require a different distribution. More generally, if a nonnegative event time has continuous cumulative hazard $H(t)$ and survival function $S(t) = e^{-H(t)}$ tending to zero, then

$$T = H^{-1}\{-\log(U)\}.$$

This turns survival-model simulation into an inversion problem. For example, the Weibull cumulative hazard $H(t) = (t/b)^a$ gives $T = b\{-\log(U)\}^{1/a}$.

Exercise 1. In-Class Question

If the exponential rate doubles, what happens to the average waiting time? Why would multiplying the simulated waiting times by two represent the opposite change?

Example 4. Numerical Inversion: Using Root Finding

An inverse may be inconvenient algebraically even when the CDF is easy to evaluate. For each u , solve

$$F(x) - u = 0.$$

For example, consider

$$F(x) = \frac{x + x^3}{2}, \quad 0 \leq x \leq 1, \quad f(x) = \frac{1 + 3x^2}{2}.$$

The CDF is strictly increasing, so each $u \in (0, 1)$ has exactly one root in $(0, 1)$. We can use the bracketing methods from the optimization chapter.

```
custom_cdf <- function(x) (x + x^3) / 2 # used only on [0, 1]

custom_quantile <- function(u) {
  vapply(u, function(ui) {
    uniroot(
      function(x) custom_cdf(x) - ui,
      interval = c(0, 1), tol = 1e-10
    )$root
  }, numeric(1))
}

set.seed(8676)
u <- runif(2000)
x_numeric <- custom_quantile(u)

c(
  largest_cdf_residual = max(abs(custom_cdf(x_numeric) - u)),
  simulated_mean = mean(x_numeric),
  theoretical_mean = 5 / 8
)
#> largest_cdf_residual      simulated_mean      theoretical_mean
#>      4.819278e-11      6.213665e-01      6.250000e-01
```

There are two distinct errors: the root solver introduces **numerical error**, while finite simulation introduces **Monte Carlo error**. Tightening `tol` reduces the former; increasing the number of independent draws reduces the latter. Neither repairs a wrong CDF.

Example 5. Optional: A Truncated Normal Distribution

Suppose only values in (a, b) are eligible for a study. If F is continuous and $F(b) > F(a)$, then the conditional CDF within this interval is

$$P(X \leq x \mid a < X < b) = \frac{F(x) - F(a)}{F(b) - F(a)}.$$

Consequently,

$$X = F^{-1} [F(a) + U\{F(b) - F(a)\}]$$

generates from the truncated distribution.

```
set.seed(8677)
a <- 1
b <- 2
u <- runif(5000)
truncated_normal <- qnorm(pnorm(a) + u * (pnorm(b) - pnorm(a)))

c(
  simulated_mean = mean(truncated_normal),
  theoretical_mean = (dnorm(a) - dnorm(b)) / (pnorm(b) - pnorm(a)),
  fraction_in_interval = mean(truncated_normal > a & truncated_normal < b)
)
#>      simulated_mean      theoretical_mean fraction_in_interval
#>           1.381749           1.383169           1.000000
```

Truncation, Censoring, and Extreme Tails

Truncation changes which individuals enter the dataset. Censoring instead records incomplete information about some individuals who remain in the dataset, such as knowing that a failure time exceeds the study duration. These require different simulation rules. The direct CDF subtraction above is suitable for this moderate interval; extreme tails require numerically stable tail probabilities or a specialized truncated-distribution generator.

4.2.2 Discrete Distributions

For ordered support values $x_1 < x_2 < \dots$, the quantile rule selects the first value whose cumulative probability reaches u :

$$X = x_j \quad \text{when} \quad F(x_{j-1}) < U \leq F(x_j).$$

Each outcome receives a subinterval of $(0, 1)$ whose length equals its probability.

Example 6. Bernoulli and Categorical Outcomes

For $X \sim \text{Bernoulli}(p)$, the generalized inverse gives

$$X = \mathbf{1}\{U > 1 - p\}.$$

The threshold divides the unit interval into pieces of lengths $1 - p$ and p .

The same idea applies to three categories with probabilities 0.2, 0.5, and 0.3. In Figure 4.3, interval **length** represents probability. A uniform draw at $u = 0.62$ selects the middle category because $0.2 < 0.62 \leq 0.7$.

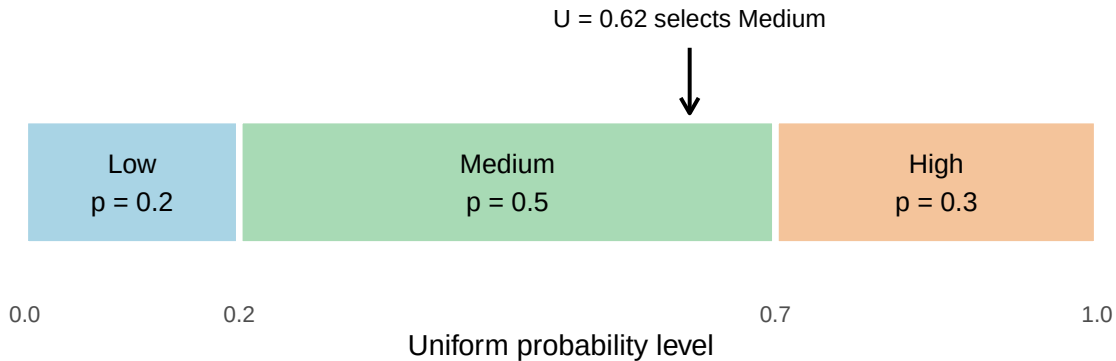


Figure 4.3: Discrete generation allocates the unit interval to outcomes. The middle category receives half of the interval, so it is selected with probability 0.5. Boundary points follow the generalized-inverse convention and have probability zero for an ideal continuous uniform.

```
set.seed(8674)
p <- 0.4
bernoulli <- as.integer(runif(10000) > 1 - p)

labels <- c("Low", "Medium", "High")
probabilities <- c(0.2, 0.5, 0.3)
cutoffs <- cumsum(probabilities)
u <- runif(10000)

# First cutoff that reaches u: this also handles u at a cutoff.
index <- vapply(u, function(ui) which(ui <= cutoffs)[1], integer(1))
category <- factor(labels[index], levels = labels)

mean(bernoulli)
#> [1] 0.4008
prop.table(table(category))
#> category
```

```
#>      Low Medium   High
#> 0.1915 0.5056 0.3029

# Built-in alternative for the same categorical distribution.
head(sample(labels, size = 10, replace = TRUE, prob = probabilities))
#> [1] "High"   "High"   "High"   "High"   "Medium" "Low"
```

Example 7. Geometric Random Variables

For the geometric distribution, use R's convention: X counts **failures before the first success**. Then

$$P(X = x) = p(1 - p)^x, \quad x = 0, 1, 2, \dots,$$

and $F(x) = 1 - (1 - p)^{x+1}$. Solving $F(x - 1) < u \leq F(x)$ gives

$$F^{-1}(u) = \left\lceil \frac{\log(1 - u)}{\log(1 - p)} \right\rceil - 1.$$

```
set.seed(8675)
p <- 0.25
u <- runif(10000)
failures <- ceiling(log1p(-u) / log1p(-p)) - 1

c(
  simulated_mean = mean(failures),
  theoretical_mean = (1 - p) / p,
  largest_difference_from_qgeom = max(abs(failures - qgeom(u, prob = p)))
)
#>              simulated_mean              theoretical_mean
#>                2.9599                3.0000
#> largest_difference_from_qgeom
#>                0.0000
```

The number of **trials including the success** is $X + 1$, with mean $1/p$. Mixing up these conventions creates an off-by-one modeling error. See [R's geometric distribution documentation](#).

i Optional: Poisson Inversion

For a Poisson distribution, cumulative probabilities can be constructed from

$$p_0 = e^{-\lambda}, \quad p_{k+1} = \frac{\lambda}{k+1} p_k, \quad F(k+1) = F(k) + p_{k+1}.$$

Search for the first k satisfying $F(k) \geq u$. This explains an inversion algorithm, but the recurrence can underflow for large λ . In practice, use `qpois(runif(n), lambda)` or the specialized generator `rpois(n, lambda)`. An `r` function need not implement the simple algorithm used in our derivation.

The inverse-transform method is especially convenient when a quantile is available. When a density is easier to work with than its CDF, the next method gives another route.

4.3 Acceptance-Rejection Method

Inverse transformation requires a usable quantile function. Acceptance-rejection requires a different resource: an easy **proposal distribution** that can cover the target.

Let f be the normalized target density and g a proposal density. We need a finite constant M such that

$$f(y) \leq M g(y)$$

everywhere on the target support. In particular, $g(y)$ must be positive wherever $f(y) > 0$.

Algorithm 2. Acceptance-Rejection Sampling

Repeat until the desired number of observations has been accepted:

1. Generate $Y \sim g$.
2. Independently generate $U \sim \text{Unif}(0, 1)$.
3. Accept Y if $U \leq f(Y)/\{Mg(Y)\}$; otherwise discard it.

Theorem 3. Distribution and Cost of Accepted Proposals

Let f and g be normalized densities with $f(y) \leq M g(y)$ throughout the target support. In the algorithm above:

- The probability of accepting each proposal is $1/M$.
- Each accepted value has density f .
- Independent proposal/decision pairs produce independent accepted observations.
- The expected number of proposals for n accepted observations is nM .

Proof. For continuous proposals,

$$P(\text{accept}) = \int g(y) \frac{f(y)}{Mg(y)} dy = \frac{1}{M}.$$

The density of an accepted proposal is therefore

$$f_{Y|\text{accept}}(y) = \frac{g(y)P(\text{accept} | Y = y)}{P(\text{accept})} = f(y).$$

i Why Thinning Works

The proposal visits some regions too frequently. The acceptance rule thins those visits by a location-specific amount. After thinning, the relative frequencies match the target. For discrete distributions, use sums instead of integrals in the proof. The number of proposals through one acceptance is geometric on $\{1, 2, \dots\}$ with mean M . The realized cost varies from run to run.

This produces independent accepted draws, unlike a Markov chain method whose successive states are generally dependent.

Example 8. Beta(2,2): Choosing an Efficient Envelope

For the Beta(2, 2) target,

$$f(y) = 6y(1 - y), \quad 0 < y < 1.$$

With a uniform proposal, $g(y) = 1$. Since the maximum occurs at $y = 1/2$,

$$M_{\min} = \max_{0 < y < 1} 6y(1 - y) = \frac{3}{2}.$$

The resulting acceptance probability is $2/3$. Choosing $M = 6$ is also valid, but accepts only $1/6$ of proposals on average. A bound can be correct and unnecessarily expensive.

```
set.seed(8678)
proposal <- runif(1500)
height <- 1.5 * runif(1500)
accepted <- height <= dbeta(proposal, 2, 2)
proposal_data <- data.frame(proposal, height, accepted)

ggplot(proposal_data, aes(proposal, height, color = accepted)) +
  geom_point(alpha = 0.6, size = 1.2) +
```

```

stat_function(fun = function(x) dbeta(x, 2, 2),
              color = "black", linewidth = 1) +
geom_hline(yintercept = 1.5, linetype = "dashed") +
scale_color_manual(values = c("grey70", "steelblue"),
                  labels = c("Rejected", "Accepted")) +
labs(x = "Proposal y", y = "Height", color = NULL)

```

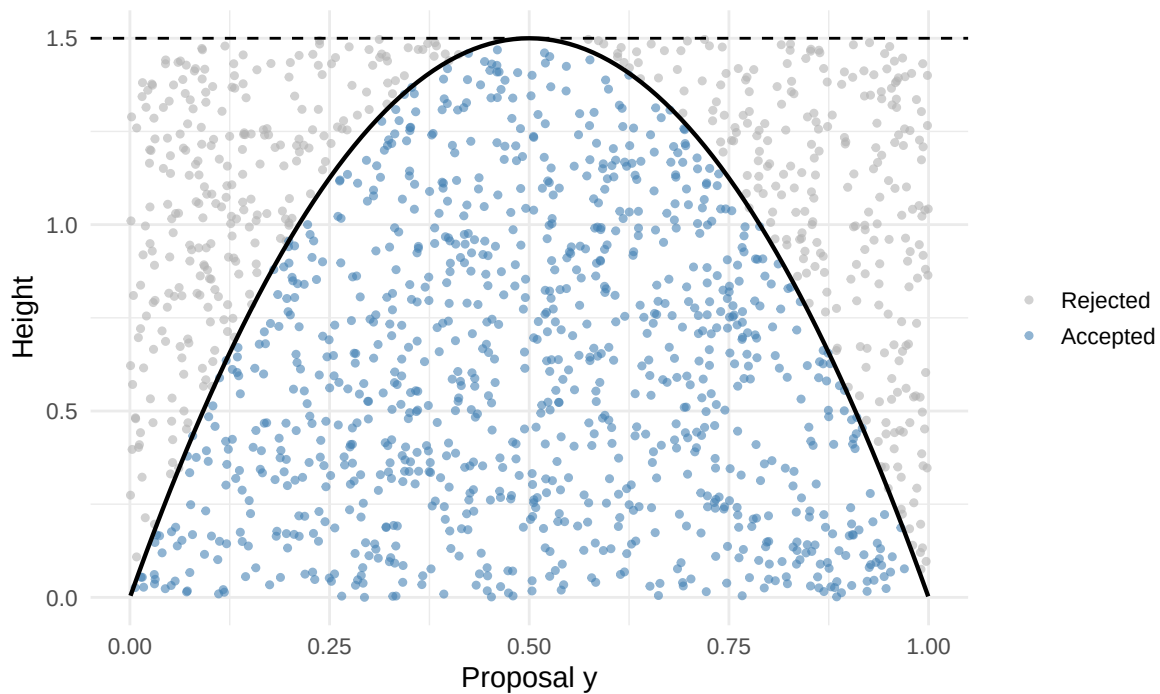


Figure 4.4: Uniform proposals form points under the envelope of height 1.5. Retaining only points below the target density produces Beta(2,2) draws along the horizontal axis.

The following implementation stops as soon as it has exactly n accepted observations. Counting all proposals makes its efficiency visible.

```

rbeta22_rejection <- function(n, M = 1.5) {
  stopifnot(
    length(n) == 1L, is.finite(n), n >= 0, n == floor(n),
    length(M) == 1L, is.finite(M), M >= 1.5
  )
  draws <- numeric(n)
  n_accepted <- 0L
  proposals <- 0L

```

```

while (n_accepted < n) {
  y <- runif(1)
  u <- runif(1)
  proposals <- proposals + 1L
  if (u <= dbeta(y, 2, 2) / M) {
    n_accepted <- n_accepted + 1L
    draws[n_accepted] <- y
  }
}
list(draws = draws, proposals = proposals)
}

set.seed(8679)
beta_tight <- rbeta22_rejection(2000, M = 1.5)
beta_loose <- rbeta22_rejection(2000, M = 6)

knitr::kable(data.frame(
  M = c(1.5, 6),
  proposals = c(beta_tight$proposals, beta_loose$proposals),
  expected_proposals = 2000 * c(1.5, 6),
  observed_acceptance = 2000 / c(beta_tight$proposals, beta_loose$proposals),
  theoretical_acceptance = 1 / c(1.5, 6)
), digits = 3, col.names = c(
  "M", "Proposals", "Expected proposals", "Observed rate", "Target rate"
))

```

| M | Proposals | Expected proposals | Observed rate | Target rate |
|-----|-----------|--------------------|---------------|-------------|
| 1.5 | 2910 | 3000 | 0.687 | 0.667 |
| 6.0 | 12628 | 12000 | 0.158 | 0.167 |

Each proposal here uses two uniforms, one for Y and one for the decision. Thus $n = 2000$ with $M = 1.5$ uses 3000 proposals and 6000 uniform draws **in expectation**. For another proposal generator, its internal random-number cost could differ.

Exercise 2. In-Class Question

What goes wrong if we choose $M = 1$ and simply replace acceptance probabilities above one by one? Why does a histogram that looks plausible fail to justify this shortcut?

A Bound Must Hold Everywhere

The envelope condition would fail near the mode. Capping the ratio changes the accepted density to one proportional to $\min\{g(y), f(y)/M\}$, which generally is not f .

Unnormalized Targets

Sometimes we know only $h(x)$ where

$$f(x) = \frac{h(x)}{Z}, \quad Z = \int h(x) dx.$$

If we can establish $h(x) \leq Cg(x)$ everywhere, we may accept with probability $h(Y)/\{Cg(Y)\}$ without knowing Z . The acceptance rate is then Z/C , not $1/C$.

This is useful for some posterior distributions, where the normalizing constant is difficult to compute. The challenge remains finding an easy proposal and a valid global bound.

Connections and Limitations

Connection to optimization. Finding the smallest bound involves maximizing f/g , or its logarithm. A numerical optimizer or a finite grid may help explore this ratio, but does not by itself prove a global bound, especially on unbounded support.

Connection to importance sampling. Both methods use a proposal g and the ratio f/g . Rejection sampling discards proposals to obtain draws from the target; importance sampling keeps proposals and weights their contributions when estimating an expectation.

Computational limitation. In high dimensions, a proposal that misses the target's concentration or dependence can yield a very small acceptance rate. This helps motivate more specialized methods, including the Markov chain Monte Carlo methods encountered in other statistics courses.

4.4 Using Probability Distribution Theory

Known distributional relationships are also simulation algorithms. They often reveal *why* a distribution appears in statistical inference.

Theorem 4. Box-Muller Transformation

If U_1, U_2 are independent uniforms, define

$$R = \sqrt{-2 \log U_1}, \quad \Theta = 2\pi U_2,$$

and

$$Z_1 = R \cos \Theta, \quad Z_2 = R \sin \Theta.$$

Then Z_1 and Z_2 are independent standard normal variables.

i Radius and Direction

A bivariate standard normal has circularly symmetric density. Its direction is uniform, and its squared radius has a χ_2^2 distribution, equivalently an exponential distribution with rate 1/2. Box-Muller generates that radius and direction, then converts polar coordinates to Cartesian coordinates. The polar Jacobian contributes a factor r , giving the radial density $re^{-r^2/2}$.

Example 9. Constructing a Bivariate Normal Cloud

```
set.seed(8680)
n <- 4000
radius <- sqrt(-2 * log(runif(n)))
angle <- 2 * pi * runif(n)
z1 <- radius * cos(angle)
z2 <- radius * sin(angle)

ggplot(data.frame(z1, z2), aes(z1, z2)) +
  geom_point(alpha = 0.2, color = "steelblue", size = 1) +
  coord_equal() +
  labs(x = expression(Z[1]), y = expression(Z[2]))
```

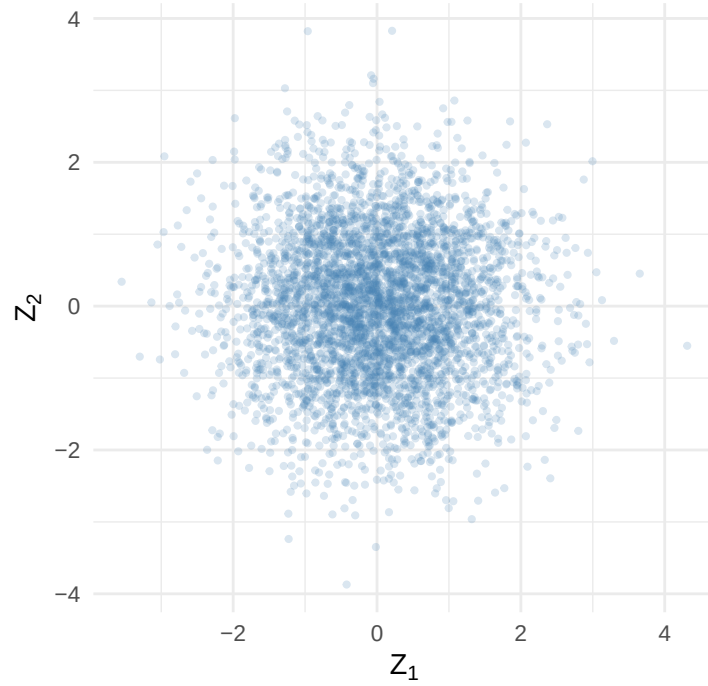


Figure 4.5: Box-Muller creates a circular normal cloud from two independent uniforms. The two normal coordinates are independent even though their formulas share a radius and angle.

🔥 Independence Comes from the Joint Distribution

Shared ingredients do not automatically imply dependence, and an approximately zero sample correlation does not prove independence. Here independence follows from the joint density. For routine work, use `rnorm()`; this construction explains one way a normal generator can work.

Theorem 5. Normal, Chi-Square, t, and F Relationships

If $Z_1, \dots, Z_k$ are independent $N(0, 1)$ variables,

$$V = \sum_{j=1}^k Z_j^2 \sim \chi_k^2.$$

If $Z \sim N(0, 1)$ is independent of $V \sim \chi_\nu^2$, then

$$T = \frac{Z}{\sqrt{V/\nu}} \sim t_\nu.$$

If $V_1 \sim \chi_{d_1}^2$ and $V_2 \sim \chi_{d_2}^2$ are independent, then

$$\frac{V_1/d_1}{V_2/d_2} \sim F_{d_1, d_2}.$$

i Why a t Statistic Has Heavier Tails

A t statistic divides a standardized normal estimation error by an estimated scale. When the scale estimate is unusually small, the ratio can be large. This explains the heavier tails of the t distribution and the larger critical values used when estimating an unknown normal variance.

Example 10. Generating a t Distribution with Five Degrees of Freedom

```
set.seed(8681)
n <- 10000
nu <- 5
normal_numerator <- rnorm(n)
chi_square <- rowSums(matrix(rnorm(n * nu), nrow = n)^2)
t_draws <- normal_numerator / sqrt(chi_square / nu)

ggplot(data.frame(value = t_draws), aes(value)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 0.2,
    fill = "lightblue", color = "white") +
  stat_function(fun = function(x) dt(x, df = nu),
    color = "firebrick", linewidth = 1) +
  stat_function(fun = dnorm, linetype = "dashed", linewidth = 0.8) +
  coord_cartesian(xlim = c(-6, 6)) +
  labs(x = "Ratio", y = "Density")
```

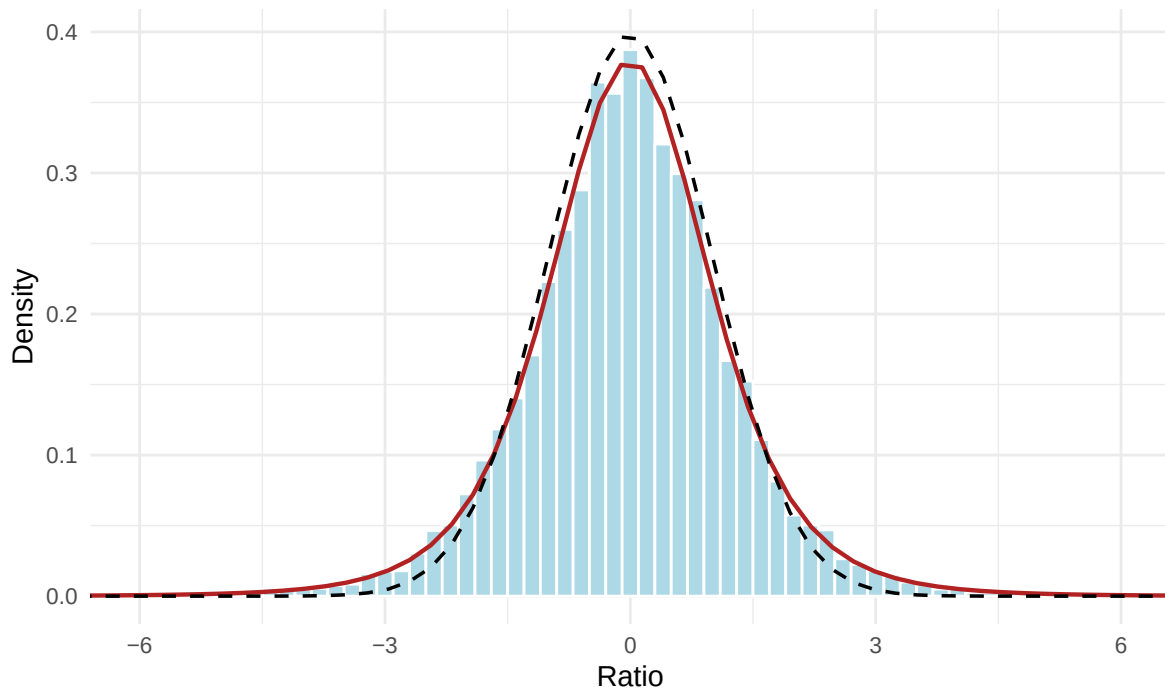


Figure 4.6: A random denominator creates heavier tails. The generated ratios agree with the t distribution with 5 degrees of freedom; the dashed normal density has lighter tails.

🔥 Generate the Numerator and Denominator Independently

The numerator and denominator must be generated independently. Reusing the same normals in both changes the distribution. In this example, $\text{Var}(T) = \nu/(\nu - 2) = 5/3$, so the raw t_5 distribution also has greater variance than $N(0, 1)$.

Theorem 6. Beta as a Ratio of Independent Gammas

If $G_1 \sim \text{Gamma}(a, \lambda)$ and $G_2 \sim \text{Gamma}(b, \lambda)$ are independent with the **same rate**, then

$$X = \frac{G_1}{G_1 + G_2} \sim \text{Beta}(a, b).$$

📌 A Proportion of a Random Total

The ratio is a random share of a positive total. This gives intuition for beta models of probabilities and proportions. The common rate cancels from the ratio.

Example 11. Generating and Checking Beta(3,2) Draws

```

set.seed(8682)
n <- 5000
g1 <- rgamma(n, shape = 3, rate = 1)
g2 <- rgamma(n, shape = 2, rate = 1)
beta_ratio <- g1 / (g1 + g2)

qq_data <- data.frame(
  theoretical = qbeta(ppoints(n), shape1 = 3, shape2 = 2),
  observed = sort(beta_ratio)
)

ggplot(qq_data, aes(theoretical, observed)) +
  geom_point(alpha = 0.3, size = 0.8, color = "steelblue") +
  geom_abline(intercept = 0, slope = 1, color = "firebrick") +
  coord_equal() +
  labs(x = "Theoretical Beta(3,2) quantile", y = "Sample quantile")

```

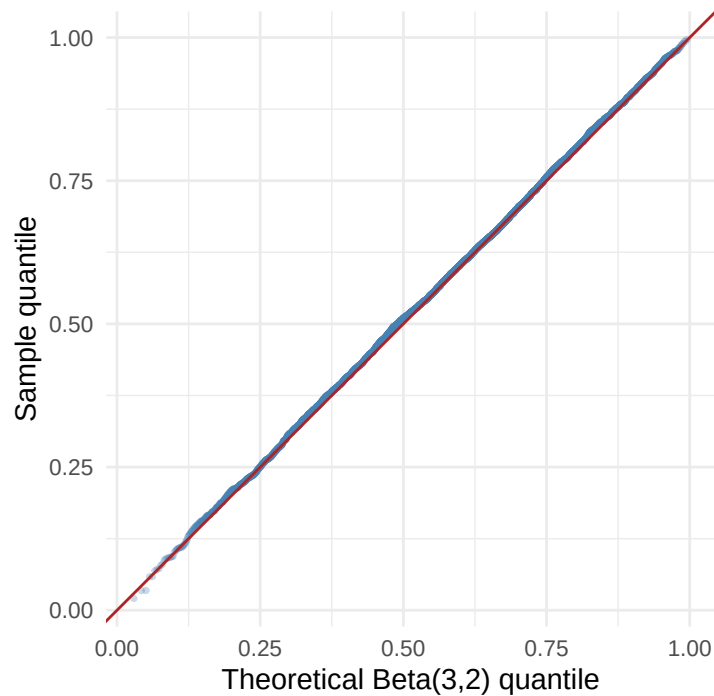


Figure 4.7: A quantile comparison checks the entire distribution of the gamma ratio against Beta(3,2), including its tails.

i Connection to Dirichlet Distributions

Normalizing several independent gammas with a common rate similarly produces a **Dirichlet** random vector. Its components sum to one, making it useful for uncertain category probabilities and mixture weights.

These identities construct new variables from familiar ones. We next distinguish two ways of combining distributions: adding contributions and selecting a population.

4.5 Sums and Mixtures

4.5.1 Sums and Convolution

A sum combines contributions from **all** its terms:

$$S = X_1 + \cdots + X_k.$$

For independent continuous variables, the density of $X_1 + X_2$ is

$$f_S(s) = \int f_{X_1}(x)f_{X_2}(s-x)dx.$$

The integral is a convolution. Simulation avoids computing this integral: generate the terms jointly according to the model and add them.

Theorem 7. Useful Convolution Identities

Under the independence conditions below:

- A sum of independent χ^2 variables is chi-square with the sum of the degrees of freedom.
- A sum of k independent $\text{Exp}(\lambda)$ variables is $\text{Gamma}(k, \lambda)$.
- A sum of k independent geometric failure counts with success probability p is negative binomial with **size** = **k** and **prob** = **p**.

i What a Sum Represents

A total processing time adds durations from all stages. A count of failures before the k th success adds the failures before each successive success. The sum corresponds to an actual mechanism, not just an algebraic identity.

4.5.2 Mixture Distributions

Definition 2. Finite Mixture

A finite mixture has CDF and density

$$F_X(x) = \sum_{j=1}^K w_j F_j(x), \quad f_X(x) = \sum_{j=1}^K w_j f_j(x),$$

where $w_j \geq 0$ and $\sum_j w_j = 1$.

Introduce an unobserved label C :

$$P(C = j) = w_j, \quad X \mid C = j \sim F_j.$$

The law of total probability gives the mixture distribution after we average over the unknown label.

Algorithm 3. Generating a Finite Mixture

For every observation:

1. Draw a component label C with probabilities $w_1, \dots, w_K$.
2. Draw one observation from F_C .

Mixture Weights Average Distributions

The weights average **distributions**. They do not instruct us to return the numerical weighted average $w_1 X_1 + \dots + w_K X_K$.

Example 12. Three Different Data-Generating Mechanisms

Let $X_1 \sim N(0, 1)$ and $X_2 \sim N(4, 1)$ be independent. Compare:

| | |
|-------------------|-------------------------------------|
| Sum: | $S = X_1 + X_2 \sim N(4, 2),$ |
| Weighted average: | $A = (X_1 + X_2)/2 \sim N(2, 1/2),$ |
| Mixture: | $X \sim 0.5N(0, 1) + 0.5N(4, 1).$ |

The mixture has mean 2 and variance 5. It describes observations from two subpopulations; each observation belongs to one subpopulation.

```

set.seed(8683)
n <- 10000
x1 <- rnorm(n, mean = 0, sd = 1)
x2 <- rnorm(n, mean = 4, sd = 1)
component <- sample.int(2, n, replace = TRUE, prob = c(0.5, 0.5))
x_mixture <- rnorm(n, mean = c(0, 4)[component], sd = 1)

mechanisms <- c("1. Sum", "2. Weighted average", "3. Mixture")
mixture_plot <- data.frame(
  value = c(x1 + x2, (x1 + x2) / 2, x_mixture),
  mechanism = rep(mechanisms, each = n)
)
grid <- seq(-4, 10, length.out = 500)
mixture_curves <- data.frame(
  value = rep(grid, 3),
  density = c(
    dnorm(grid, 4, sqrt(2)),
    dnorm(grid, 2, sqrt(0.5)),
    0.5 * dnorm(grid, 0, 1) + 0.5 * dnorm(grid, 4, 1)
  ),
  mechanism = rep(mechanisms, each = length(grid))
)
ggplot(mixture_plot, aes(value)) +
  geom_histogram(aes(y = after_stat(density)), bins = 45,
    fill = "lightblue", color = "white") +
  geom_line(data = mixture_curves, aes(y = density),
    color = "firebrick", linewidth = 0.9) +
  facet_wrap(~ mechanism, nrow = 1) +
  labs(x = "Value", y = "Density")

```

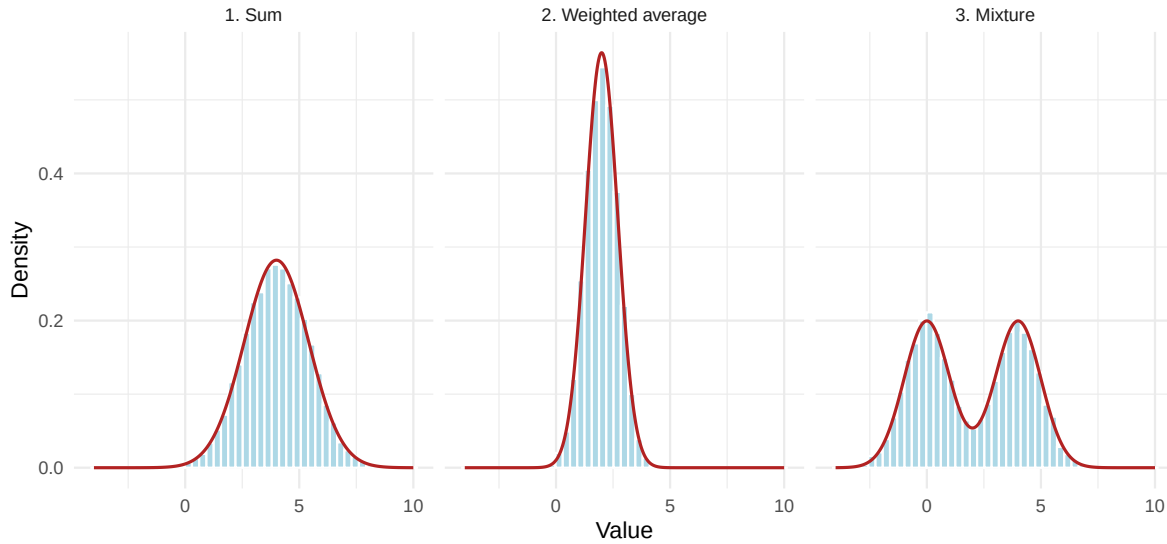



Figure 4.8: Adding two observations, averaging them, and selecting one population produce different distributions. The solid curves show their respective theoretical densities.

Theorem 8. Mean and Variance of a Mixture

If component j has mean μ_j and variance σ_j^2 , then

$$\mathbb{E}(X) = \sum_j w_j \mu_j = \mu,$$

and the law of total variance gives

$$\begin{aligned} \text{Var}(X) &= \mathbb{E}\{\text{Var}(X | C)\} + \text{Var}\{\mathbb{E}(X | C)\} \\ &= \underbrace{\sum_j w_j \sigma_j^2}_{\text{within-component variation}} + \underbrace{\sum_j w_j (\mu_j - \mu)^2}_{\text{between-component variation}}. \end{aligned}$$

i Heterogeneity and Latent Labels

This decomposition explains why unobserved heterogeneity can inflate variability. A mixture of distinct normal components is generally nonnormal, but it need not be bimodal. Identical components yield the original normal distribution.

Connection to clustering and EM. Simulation starts with the label and then generates the observation. In mixture-model estimation, we see the observation and infer uncertainty about its label. This reversal is the basis for the responsibility calculations in the EM

algorithm.

Exercise 3. In-Class Question

In the example above, the weighted average and the mixture both have mean 2. Why would checking only the sample mean fail to detect the wrong generator?

Example 13. Poisson-Gamma Mixing and Overdispersion

A mixture label need not take only finitely many values. Consider

$$\Lambda \sim \text{Gamma}(a, b), \quad Y \mid \Lambda \sim \text{Poisson}(\Lambda),$$

where b is a rate. Then

$$\mathbb{E}(Y) = \frac{a}{b}, \quad \text{Var}(Y) = \frac{a}{b} + \frac{a}{b^2}.$$

The second term is variability in the underlying rate. Marginally, Y is negative binomial with $\text{size} = a$ and $\text{prob} = b / (b + 1)$.

```
set.seed(8684)
n <- 20000
a <- 2
b <- 0.5
latent_rate <- rgamma(n, shape = a, rate = b)
counts <- rpois(n, lambda = latent_rate)

knitr::kable(data.frame(
  quantity = c("Mean", "Variance"),
  simulated = c(mean(counts), var(counts)),
  theoretical = c(a / b, a / b + a / b^2),
), digits = 3)
```

| quantity | simulated | theoretical |
|----------|-----------|-------------|
| Mean | 3.968 | 4 |
| Variance | 11.766 | 12 |

Optional Connection: Heterogeneous Event Rates

Customers, neighborhoods, or machines can have different event rates. Even when each unit follows a Poisson model conditional on its rate, the pooled counts may have variance greater than their mean. This is one reason negative binomial regression is useful.

Shared Latent Variables Create Dependence

We drew one independent rate for each count. If several observations share a single group-specific rate, they become dependent after that rate is integrated out. **Where we place a draw in the simulation loop determines which observations share uncertainty.**

The same generative logic extends to random vectors. The additional task is to reproduce how their coordinates vary together.

4.6 Generating Multivariate Random Variables

4.6.1 Multivariate Normal Distribution

Suppose two measurements each have a standard normal marginal distribution. We still need to specify how they move together. Independent `rnorm()` calls impose independence; they cannot represent a positive correlation merely because the marginal histograms look correct.

For a normal vector, the covariance matrix supplies this information:

$$X \sim N_d(\mu, \Sigma).$$

Theorem 9. Linear Transformation of a Normal Vector

If $Z \sim N_d(0, I_d)$ and $AA^\top = \Sigma$, then

$$X = \mu + AZ.$$

Proof.

$$\mathbb{E}(X) = \mu, \quad \text{Cov}(X) = AI_dA^\top = \Sigma.$$

The normality follows because every linear combination of X is a linear combination of independent normals.

🔥 Match the Matrix Factor to the Row Convention

R's `chol(Sigma)` returns an **upper triangular** matrix R satisfying

$$R^\top R = \Sigma.$$

If simulated observations are **rows** of an $n \times d$ matrix Z , use

$$X = ZR + 1_n \mu^\top.$$

For a column vector, the corresponding factor is $A = R^\top$. Mixing up these orientations generally produces the wrong covariance. See the [R documentation for `chol\(\)`](#).

The construction has a geometric interpretation. In Figure 4.9, the same points start in a circular cloud, become an ellipse after multiplication by the matrix factor, and then move to the desired mean. Color tracks each point's original first coordinate.

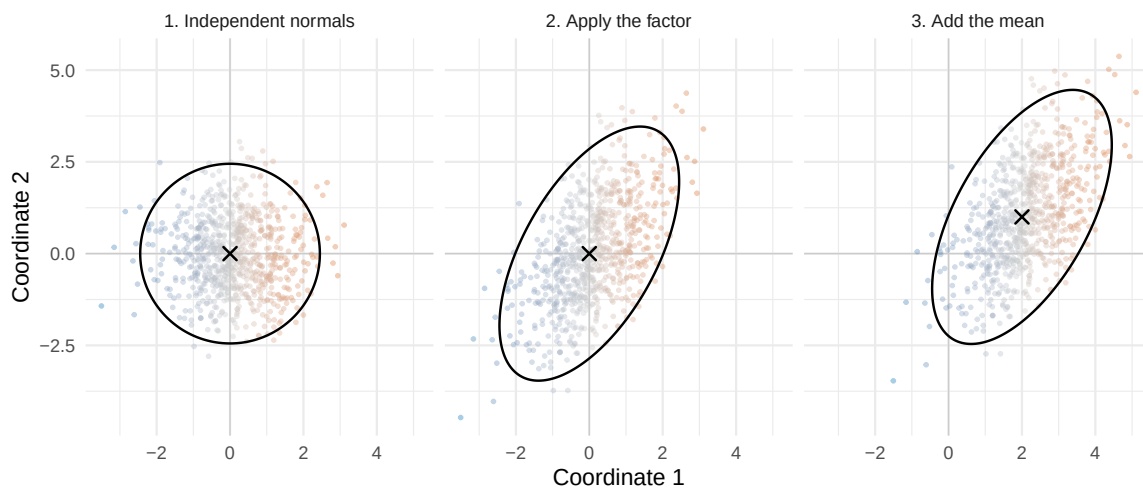


Figure 4.9: A matrix factor introduces scale and dependence; adding the mean shifts location. The black curves are corresponding 95% normal probability contours. All three panels use the same underlying draws and the same axis scale.

Example 14. A Generator with a Specified Mean and Covariance

```
rmvn_chol <- function(n, mu, Sigma) {  
  d <- length(mu)  
  stopifnot(  
    length(n) == 1L, is.finite(n), n >= 0, n == floor(n),  
    d > 0L, all(is.finite(mu)),  
    is.matrix(Sigma), all(dim(Sigma) == c(d, d)),  
  )  
}
```

```

    all(is.finite(Sigma)), isSymmetric(Sigma)
  )
  R <- chol(Sigma) # requires positive definiteness here
  Z <- matrix(rnorm(n * d), nrow = n, ncol = d)
  sweep(Z %*% R, MARGIN = 2, STATS = mu, FUN = "+")
}

set.seed(8685)
mu <- c(1, 2)
Sigma <- matrix(c(1, 0.8, 0.8, 2), nrow = 2)
mvn_draws <- rmvn_chol(10000, mu, Sigma)

colMeans(mvn_draws)
#> [1] 1.005665 2.013197
cov(mvn_draws)
#>           [,1]      [,2]
#> [1,] 1.0025949 0.7703105
#> [2,] 0.7703105 1.9799767
Sigma
#>      [,1] [,2]
#> [1,]  1.0  0.8
#> [2,]  0.8  2.0

```

The function also returns an $n \times d$ matrix for $n = 0$ or $n = 1$. This matters when a component in a simulated mixture receives few or no observations.

i Eigenvalues, PCA, and Reusing the Factorization

The spectral decomposition is

$$\Sigma = P\Lambda P^\top, \quad \Sigma^{1/2} = P\Lambda^{1/2}P^\top.$$

The eigenvectors specify directions, and the square roots of the eigenvalues specify how much to stretch a standard normal cloud in those directions.

This connects generation to **principal component analysis**. PCA identifies directions of variation in observed data; simulation constructs that variation from independent coordinates.

A covariance matrix can be positive semidefinite with some eigenvalues equal to zero. The normal vector then lies in a lower-dimensional subspace. Ordinary unpivoted Cholesky may fail, whereas an eigenvalue construction can handle that case. Tiny negative eigenvalues caused by roundoff may be set to zero after a tolerance check; substantial negative eigenvalues indicate an invalid covariance matrix and should not be silently removed.

If Σ stays fixed across many simulation batches, compute its factor once and reuse it. Factorization costs roughly order d^3 , while multiplying n observations by a dense factor costs order nd^2 . Avoid repeating work that does not change the model.

Example 15. The Same Marginals, Different Joint Tail Probabilities

Consider two standardized laboratory measurements with unit marginal variances. We compare independence with correlation $\rho = 0.8$ while keeping the marginals the same.

```
set.seed(8686)
n <- 30000
independent_pair <- rmvn_chol(n, c(0, 0), diag(2))
correlated_pair <- rmvn_chol(
  n, c(0, 0), matrix(c(1, 0.8, 0.8, 1), 2)
)

joint_probability <- c(
  mean(independent_pair[, 1] > 1 & independent_pair[, 2] > 1),
  mean(correlated_pair[, 1] > 1 & correlated_pair[, 2] > 1)
)

joint_results <- data.frame(
  model = c("Independent", "Correlation 0.8"),
  probability = joint_probability,
  MCSE = sqrt(joint_probability * (1 - joint_probability) / n)
)

# Plot a subset for legibility; estimates use all n observations.
dependence_plot <- rbind(
  data.frame(x = independent_pair[1:2000, 1],
             y = independent_pair[1:2000, 2], model = "Independent"),
  data.frame(x = correlated_pair[1:2000, 1],
             y = correlated_pair[1:2000, 2], model = "Correlation 0.8")
)

ggplot(dependence_plot, aes(x, y)) +
  annotate("rect", xmin = 1, xmax = Inf, ymin = 1, ymax = Inf,
         fill = "gold", alpha = 0.15) +
  geom_point(alpha = 0.2, color = "steelblue", size = 0.8) +
  geom_vline(xintercept = 1, linetype = "dashed") +
  geom_hline(yintercept = 1, linetype = "dashed") +
  facet_wrap(~ model) +
  coord_equal() +
  labs(x = "Measurement 1", y = "Measurement 2")
```

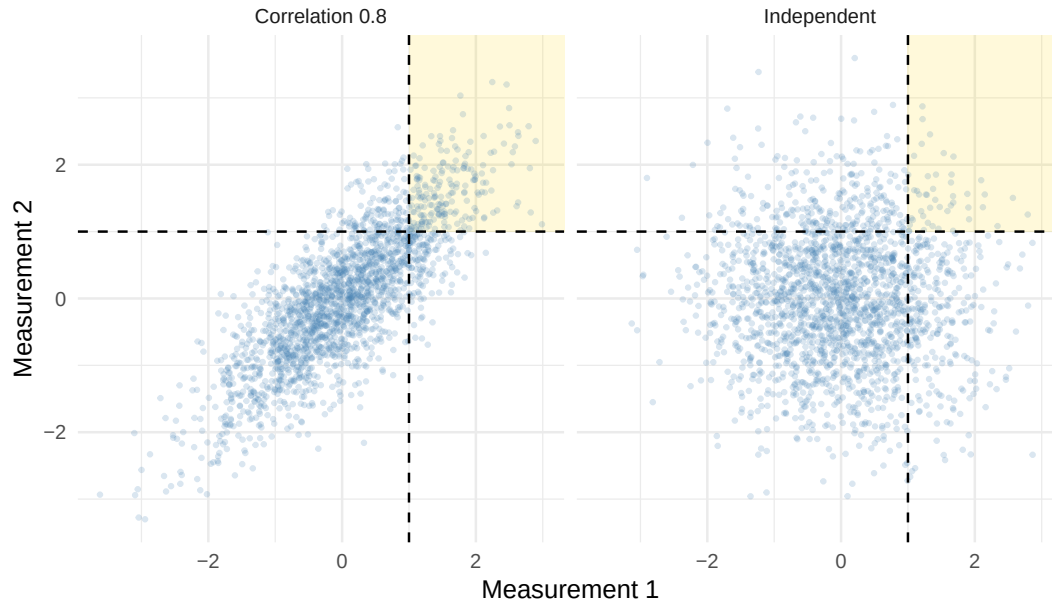


Figure 4.10: The marginal distributions are the same in both panels. Positive correlation puts more probability in the upper-right region where both measurements exceed 1.

```
knitr::kable(joint_results, digits = 4,
              col.names = c("Model", "Joint probability", "MCSE"))
```

| Model | Joint probability | MCSE |
|-----------------|-------------------|--------|
| Independent | 0.0249 | 0.0009 |
| Correlation 0.8 | 0.0987 | 0.0017 |

```
(1 - pnorm(1))^2 # exact probability under independence
#> [1] 0.02517149
```

The simulated joint probability rises from approximately 0.025 to 0.099 under these two models. The **Monte Carlo standard error (MCSE)** measures simulation noise; we derive it below.

The same issue arises with simultaneous equipment failures, correlated regression errors, and repeated measurements. A study of a joint event must preserve the relevant dependence.

Theorem 10. Optional: Wishart Construction

If $X_1, \dots, X_m$ are independent $N_d(0, \Sigma)$ vectors, then

$$W = \sum_{i=1}^m X_i X_i^\top \sim \text{Wishart}_d(m, \Sigma).$$

Optional Connection: Uncertainty in a Sample Covariance

With observations stored in rows, this is simply `crossprod(X)`. For a normal sample of size n with an estimated mean, its usual sample covariance satisfies

$$(n-1)S \sim \text{Wishart}_d(n-1, \Sigma).$$

This explains why Wishart simulation is relevant to uncertainty in covariance estimates and multivariate inference.

```
set.seed(8688)
Sigma_w <- matrix(c(1, 0.4, 0.4, 2), 2)
wishart_df <- 12
W <- crossprod(rmvn_chol(wishart_df, c(0, 0), Sigma_w))
W / wishart_df # expectation equals Sigma_w, but one draw is noisy
#>           [,1]      [,2]
#> [1,] 0.4730056 0.0600369
#> [2,] 0.0600369 1.1392812
```

4.6.2 Multivariate Normal Mixtures

One Component Label per Vector

Each observation needs **one component label for the entire vector**. Drawing a different label for each coordinate changes the joint model.

Example 16. A Mixture of Two Bivariate Normals

```
set.seed(8687)
n <- 3000
component <- sample.int(2, n, replace = TRUE, prob = c(0.65, 0.35))
means <- list(c(0, 0), c(3, 2))
covariances <- list(
  matrix(c(1, 0.6, 0.6, 1), 2),
  matrix(c(0.6, -0.3, -0.3, 1), 2)
)
mixed_vectors <- matrix(NA_real_, nrow = n, ncol = 2)
```



```

for (j in seq_len(2)) {
  rows <- which(component == j)
  if (length(rows) > 0) {
    mixed_vectors[rows, ] <- rmvn_chol(
      length(rows), means[[j]], covariances[[j]]
    )
  }
}

ggplot(
  data.frame(x = mixed_vectors[, 1], y = mixed_vectors[, 2],
             component = factor(component)),
  aes(x, y, color = component)
) +
  geom_point(alpha = 0.25, size = 1) +
  coord_equal() +
  scale_color_manual(values = c("steelblue", "firebrick")) +
  labs(x = "Variable 1", y = "Variable 2", color = "Component")

```

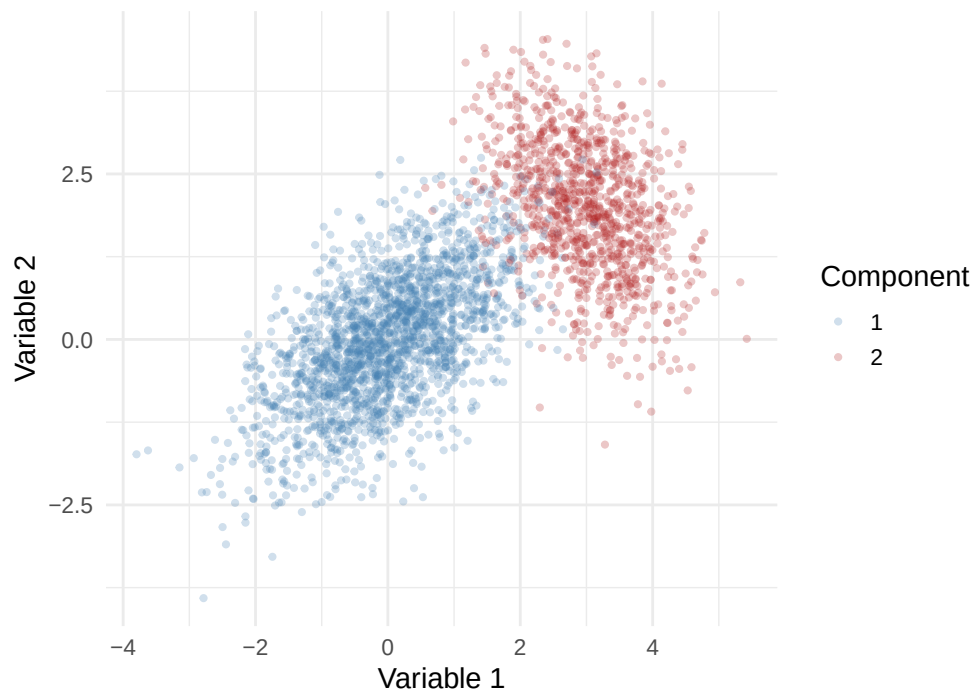


Figure 4.11: Each point is generated from one bivariate normal component. The label determines both its coordinates and its covariance structure.

i Within-Group and Between-Group Covariance

For vector means μ_j and covariances Σ_j ,

$$\text{Cov}(X) = \sum_j w_j \Sigma_j + \sum_j w_j (\mu_j - \mu)(\mu_j - \mu)^\top.$$

Between-group differences can create marginal association even if variables are independent *within* every group. This is useful when designing simulation studies for clustering or confounding.

4.6.3 Uniform Directions and Points in a Ball

Theorem 11. Uniform Direction from a Standard Normal Vector

Let $Z \sim N_d(0, I_d)$. Its distribution is unchanged by rotations, so it has no preferred direction. Consequently,

$$U = \frac{Z}{\|Z\|}$$

is uniform on the **surface** of the unit sphere in $\mathbb{R}^d$. The zero vector has probability zero under the ideal normal model.

Theorem 12. Uniform Volume Requires a Nonuniform Radius

Uniform direction alone puts every point at distance 1. To fill the ball uniformly in volume, independently generate a radius R satisfying

$$P(R \leq r) = r^d, \quad 0 \leq r \leq 1.$$

This follows because a ball of radius r has r^d times the volume of the unit ball. Therefore,

$$R = V^{1/d}, \quad X = RU,$$

where V is uniform and independent of the direction.

Example 17. Generating a Circle and a Disk

```

runif_sphere <- function(n, d) {
  stopifnot(
    length(n) == 1L, is.finite(n), n >= 0, n == floor(n),
    length(d) == 1L, is.finite(d), d >= 1, d == floor(d)
  )
  Z <- matrix(rnorm(n * d), nrow = n, ncol = d)
  lengths <- sqrt(rowSums(Z^2))
  sweep(Z, MARGIN = 1, STATS = lengths, FUN = "/")
}

```

We divide each row by its length directly. Constructing an $n \times n$ diagonal matrix of inverse lengths would waste memory.

```

set.seed(8689)
n <- 2500
directions <- runif_sphere(n, d = 2)
disk <- sweep(directions, 1, sqrt(runif(n)), FUN = "*")

sphere_plot <- rbind(
  data.frame(x = directions[, 1], y = directions[, 2],
             region = "Surface: unit circle"),
  data.frame(x = disk[, 1], y = disk[, 2],
             region = "Interior: unit disk")
)
ggplot(sphere_plot, aes(x, y)) +
  geom_point(alpha = 0.3, size = 0.7, color = "steelblue") +
  facet_wrap(~ region) +
  coord_equal() +
  labs(x = "Coordinate 1", y = "Coordinate 2")

```

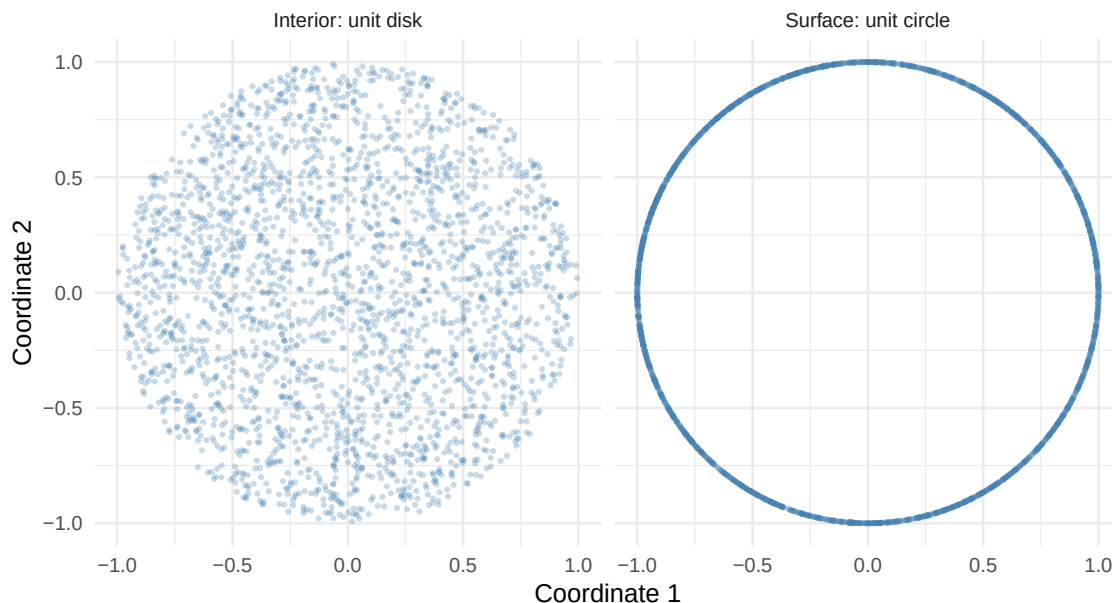


Figure 4.12: In two dimensions, normalized normals lie on the circle. Multiplying by an independent square-root-uniform radius fills the disk uniformly in area.

i Geometry in High Dimensions

Random directions appear in rotationally invariant perturbations, geometric Monte Carlo methods, and randomized algorithms. In high dimensions, a uniform point in a ball is typically close to its boundary: $P(R \leq 0.9) = 0.9^d$. At $d = 100$, this probability is about 0.000027.

Exercise 4. In-Class Question

Why does a uniform radius on $(0, 1)$ place too much mass near the center of a disk? Why does choosing a uniform latitude fail to generate uniform points on the surface of a three-dimensional sphere?

4.7 Checking a Generator

A mathematical construction tells us the target law. Numerical checks then help detect programming errors and quantify how much disagreement to expect from a finite sample.

A histogram is a useful first view, but its appearance depends on bin width and sample size. Different distributions can have the same mean, variance, or a similar-looking central region.

A convincing validation combines theory with checks that target different possible mistakes:

| Check | What it can reveal |
|---|---|
| Support and constraints | Negative waiting times, noninteger counts, proportions outside (0, 1) |
| Mean and variance | Incorrect location, scale, or parameterization |
| Quantiles, ECDF, and tail probabilities | Distributional errors hidden by matching moments |
| Correlations and joint events | Incorrect dependence or matrix orientation |
| Repeated runs and random-number use | Reused seeds or repeated observations |
| Acceptance rate and proposal count | A costly envelope or an implementation error |

For the rejection sampler, the ECDF should track the target CDF, and the transformed values should be approximately uniform.

```
ggplot(data.frame(value = beta_tight$draws), aes(value)) +  
  stat_ecdf(geom = "step", color = "steelblue", linewidth = 0.7) +  
  stat_function(fun = function(x) pbeta(x, 2, 2),  
               color = "firebrick", linewidth = 0.9, linetype = "dashed") +  
  labs(x = "Accepted value", y = "Cumulative probability")
```

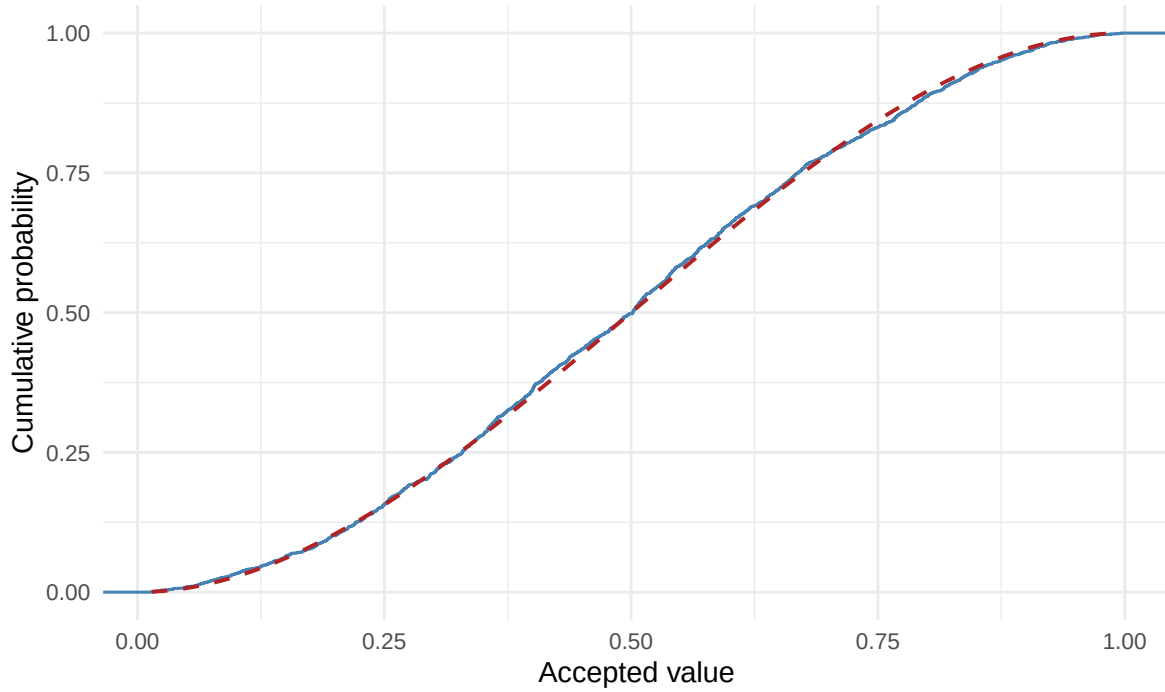


Figure 4.13: The empirical CDF of accepted proposals tracks the Beta(2,2) CDF. Sampling variation creates small discrepancies even for a correct generator.

Formal goodness-of-fit tests can supplement these checks, but passing a test is not a proof of correctness. A correct generator can occasionally be rejected, and a small sample may fail to detect a faulty one. The algorithm's mathematical justification remains essential.

i Monte Carlo Standard Error

Suppose we estimate $p = P(X > c)$ using B independent simulated observations:

$$\hat{p} = \frac{1}{B} \sum_{b=1}^B \mathbf{1}\{X_b > c\}.$$

Each indicator is Bernoulli, so

$$\text{Var}(\hat{p}) = \frac{p(1-p)}{B}, \quad \widehat{\text{MCSE}}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{B}}.$$

For an estimated mean of independent $h(X_b)$ with finite variance, the corresponding MCSE is

$$\widehat{\text{MCSE}} \left(\frac{1}{B} \sum_{b=1}^B h(X_b) \right) = \frac{s_h}{\sqrt{B}}.$$

Example 18. Checking an Exponential Tail Probability

```
set.seed(8690)
B <- 10000
event <- rexp(B, rate = 0.7) > 3
p_hat <- mean(event)
mcse <- sqrt(p_hat * (1 - p_hat) / B)

knitr::kable(data.frame(
  estimate = p_hat,
  MCSE = mcse,
  exact = exp(-0.7 * 3),
  lower_MC_limit = p_hat - 1.96 * mcse,
  upper_MC_limit = p_hat + 1.96 * mcse
), digits = 4,
col.names = c("Estimate", "MCSE", "Exact", "Lower MC limit", "Upper MC limit"))
```

| Estimate | MCSE | Exact | Lower MC limit | Upper MC limit |
|----------|--------|--------|----------------|----------------|
| 0.1215 | 0.0033 | 0.1225 | 0.1151 | 0.1279 |

What More Simulation Can and Cannot Fix

These limits quantify numerical uncertainty from the finite simulation. They do not quantify uncertainty about whether the exponential model or its rate describes a real population.

To halve MCSE, we generally need about **four times** as many independent replications. For rare events, zero simulated occurrences do not imply zero probability: the plug-in MCSE also becomes zero and is misleading. A binomial interval or a method designed for rare events is then more informative.

Three Different Sources of Uncertainty

Sampling uncertainty concerns what would happen with another real dataset. **Monte Carlo uncertainty** comes from using finitely many simulated replications to study that behavior. **Model uncertainty** concerns whether the assumed mechanism is an adequate description of the setting.

More simulation reduces Monte Carlo uncertainty. It does not automatically reduce the other two.

i Choosing a Generation Method

| Available structure | Natural approach | Main issue to check |
|--|--|--|
| A reliable built-in generator | Use the distribution's $\mathbf{r}$ function | Parameterization and joint assumptions |
| A tractable quantile | Inverse transformation | Correct generalized inverse and endpoints |
| A tractable CDF without a convenient inverse | Numerical root finding | Bracketing, monotonicity, tolerance, and cost |
| A target density and a suitable proposal | Acceptance-rejection | Global envelope and acceptance rate |
| A known distributional identity | Transform simpler independent variables | Conditions such as independence and common rates |
| A latent-class or hierarchical model | Generate latent variables, then observations | Which observations share each latent draw |
| A normal covariance structure | Matrix factorization | Positive definiteness and factor orientation |
| Rotational or volume symmetry | Geometric construction | Surface versus volume and the correct radius |

Methods are often combined. A hierarchical model may use inverse transformation for event times, normal matrix factors for random effects, and conditional Bernoulli draws for outcomes.

The goal is a generator that is statistically correct, numerically reliable, and efficient enough for the problem. A sophisticated algorithm cannot compensate for an incorrect data-generating mechanism.

i The Main Ideas to Keep

- A probability model becomes operational when we can generate data from it.
- Uniform draws can be transformed into observations through a generalized inverse CDF.
- Rejection sampling reshapes an easy proposal; a valid, tight envelope controls efficiency.
- Distributional identities connect simulation to the foundations of statistical inference.

- A mixture selects a component; a sum combines contributions from all components.
- Hierarchical draws specify both marginal variation and dependence.
- Multivariate generation must reproduce covariance and joint behavior, not only marginal histograms.
- Validation combines mathematical reasoning, numerical checks, and distributional diagnostics.
- More independent replications reduce Monte Carlo error; they do not correct a wrong model.

The next step is to use these generators to approximate expectations, integrals, and sampling distributions. That is the central task of **Monte Carlo simulation**.

4.8 Optional: Real-World Applications

The core chapter is complete. The following **optional examples** show how to combine the generators in full statistical analyses. Their assumptions and parameters are illustrative. They can be used for additional class discussion or independent study.

Example 19. Optional Application: Experimental Power

Suppose an experiment compares two independent groups, each with n observations:

$$X_i \sim N(0, \sigma^2), \quad Y_i \sim N(\delta, \sigma^2).$$

Here δ is a meaningful difference we want to detect. To keep the benchmark exact, assume $\sigma = 1$ is known. The two-sided level-0.05 test rejects when

$$|Z| > z_{0.975}, \quad Z = \frac{\bar{Y} - \bar{X}}{\sigma\sqrt{2/n}}.$$

For $\delta = 0$, the rejection probability is the Type I error rate. For $\delta \neq 0$, it is power.

We generate **whole datasets**, apply the proposed analysis to each dataset, and record the result. The data size n determines how informative each experiment is. The number of replications B determines how accurately we estimate its rejection probability.

```
set.seed(8691)
B <- 4000
sigma <- 1
design <- expand.grid(n = c(10, 20, 40, 80), delta = c(0, 0.5))
critical <- qnorm(0.975)
```

```

power_results <- do.call(rbind, lapply(seq_len(nrow(design)), function(i) {
  n <- design$n[i]
  delta <- design$delta[i]
  control <- matrix(rnorm(B * n, 0, sigma), nrow = B)
  treatment <- matrix(rnorm(B * n, delta, sigma), nrow = B)
  se <- sigma * sqrt(2 / n)
  statistic <- (rowMeans(treatment) - rowMeans(control)) / se
  estimated <- mean(abs(statistic) > critical)
  shift <- delta / se
  exact <- pnorm(-critical - shift) +
    pnorm(critical - shift, lower.tail = FALSE)

  data.frame(
    n = n, delta = delta, estimated = estimated, exact = exact,
    MCSE = sqrt(estimated * (1 - estimated) / B)
  )
}))
power_results$scenario <- ifelse(
  power_results$delta == 0, "Null: Type I error", "Effect 0.5: power"
)
knitr::kable(power_results[, c("n", "delta", "estimated", "exact", "MCSE")],
  digits = 4)

```

| n | delta | estimated | exact | MCSE |
|----|-------|-----------|--------|--------|
| 10 | 0.0 | 0.0520 | 0.0500 | 0.0035 |
| 20 | 0.0 | 0.0555 | 0.0500 | 0.0036 |
| 40 | 0.0 | 0.0568 | 0.0500 | 0.0037 |
| 80 | 0.0 | 0.0478 | 0.0500 | 0.0034 |
| 10 | 0.5 | 0.2010 | 0.2010 | 0.0063 |
| 20 | 0.5 | 0.3622 | 0.3526 | 0.0076 |
| 40 | 0.5 | 0.6082 | 0.6088 | 0.0077 |
| 80 | 0.5 | 0.8815 | 0.8854 | 0.0051 |

```

ggplot(power_results, aes(n, estimated)) +
  geom_line(aes(y = exact), color = "firebrick", linewidth = 0.9) +
  geom_errorbar(aes(
    ymin = pmax(0, estimated - 1.96 * MCSE),
    ymax = pmin(1, estimated + 1.96 * MCSE)
  ), width = 3, color = "steelblue") +
  geom_point(color = "steelblue", size = 2) +

```

```
facet_wrap(~ scenario) +
scale_x_continuous(breaks = c(10, 20, 40, 80)) +
coord_cartesian(ylim = c(0, 1)) +
labs(x = "Sample size per group", y = "Rejection probability")
```

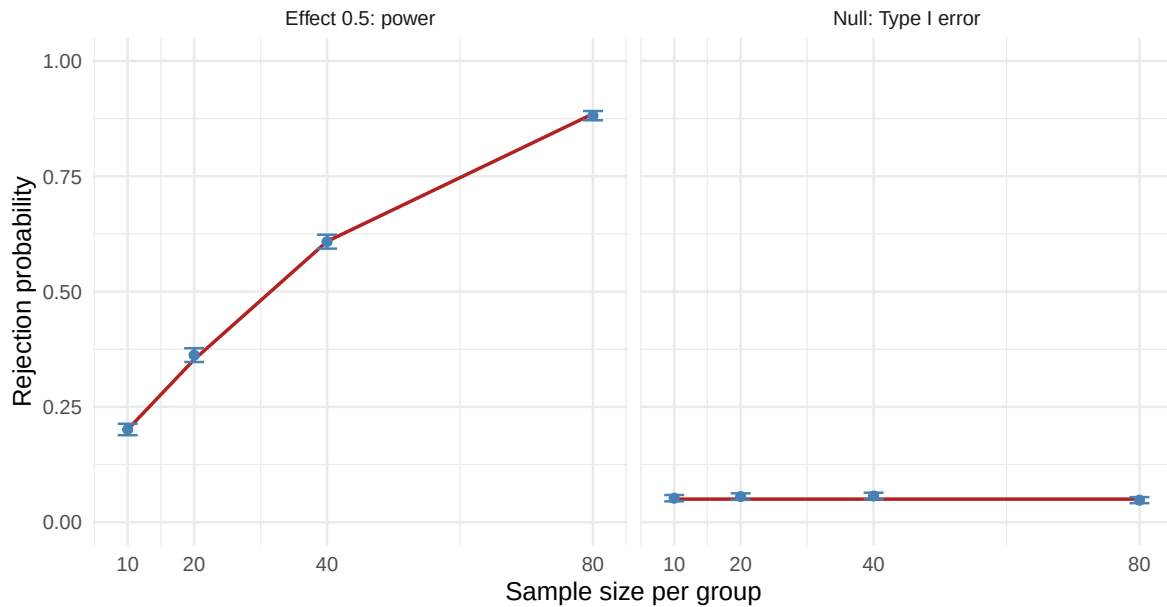


Figure 4.14: Points are simulated rejection rates with approximate 95% Monte Carlo error bars. Lines are exact normal-model probabilities. Increasing the experiment's sample size increases power, while the Type I error remains near 0.05.

i Interpretation and Assumptions

The exact formula gives a benchmark for our simulation. When the model becomes more realistic, such as unequal variances, clustered observations, or missing outcomes, simulation may remain feasible even when a simple power formula does not.

If σ is estimated, replace the known-variance test with the actual planned procedure, such as a two-sample t test. Simulating one analysis and reporting power for another answers the wrong question.

Example 20. Optional Application: Predicting Defect Counts

Suppose a process produces independent binary defect indicators conditional on an unknown defect probability p . Start with a Beta(1,1) prior and observe 3 defects among 18 items. Beta-binomial conjugacy gives

$$p \mid \text{data} \sim \text{Beta}(4, 16).$$

To predict the number of defects Y_{new} in a future batch of $m = 50$ items, generate

$$p^{(b)} \sim \text{Beta}(4, 16), \quad Y_{\text{new}}^{(b)} \mid p^{(b)} \sim \text{Binomial}(50, p^{(b)}).$$

Each simulated future batch uses one shared draw of its possible process rate. This integrates uncertainty about p into the prediction.

The plug-in alternative uses only $\bar{p} = 4/20 = 0.2$:

$$Y_{\text{plug-in}} \sim \text{Binomial}(50, 0.2).$$

Both have mean 10, but the posterior predictive variance is larger:

$$\begin{aligned} \text{Var}(Y_{\text{new}} \mid \text{data}) &= m\bar{p}(1-\bar{p})\frac{a+b+m}{a+b+1} \\ &= 50(0.2)(0.8)\frac{70}{21} \approx 26.67, \end{aligned}$$

compared with the plug-in variance $50(0.2)(0.8) = 8$.

```
set.seed(8692)
B <- 20000
a_post <- 4
b_post <- 16
batch_size <- 50
p_mean <- a_post / (a_post + b_post)

p_draw <- rbeta(B, shape1 = a_post, shape2 = b_post)
predictive_defects <- rbinom(B, size = batch_size, prob = p_draw)
plugin_defects <- rbinom(B, size = batch_size, prob = p_mean)

k <- 0:batch_size
# Exact beta-binomial probabilities, computed on the log scale.
predictive_pmf <- exp(
  lchoose(batch_size, k) +
  lbeta(k + a_post, batch_size - k + b_post) -
  lbeta(a_post, b_post)
)
predictive_comparison <- data.frame(
```

```

model = c("Posterior predictive", "Plug-in"),
simulated_variance = c(var(predictive_defects), var(plugin_defects)),
theoretical_variance = c(
  batch_size * p_mean * (1 - p_mean) *
    (a_post + b_post + batch_size) / (a_post + b_post + 1),
  batch_size * p_mean * (1 - p_mean)
),
simulated_prob_at_least_15 = c(
  mean(predictive_defects >= 15), mean(plugin_defects >= 15)
),
exact_prob_at_least_15 = c(
  sum(predictive_pmf[k >= 15]),
  pbinom(14, size = batch_size, prob = p_mean, lower.tail = FALSE)
)
)
knitr::kable(predictive_comparison, digits = 3, col.names = c(
  "Model", "Sim. variance", "Exact variance", "Sim. P(Y >= 15)", "Exact P(Y >= 15)"
))

```

| Model | Sim. variance | Exact variance | Sim. P(Y >= 15) | Exact P(Y >= 15) |
|----------------------|---------------|----------------|-----------------|------------------|
| Posterior predictive | 26.898 | 26.667 | 0.187 | 0.187 |
| Plug-in | 7.986 | 8.000 | 0.060 | 0.061 |

```

predictive_plot <- rbind(
  data.frame(
    defects = k,
    proportion = tabulate(predictive_defects + 1, nbins = batch_size + 1) / B,
    exact = predictive_pmf,
    model = "Posterior predictive"
  ),
  data.frame(
    defects = k,
    proportion = tabulate(plugin_defects + 1, nbins = batch_size + 1) / B,
    exact = dbinom(k, size = batch_size, prob = p_mean),
    model = "Plug-in"
  )
)
ggplot(predictive_plot, aes(defects, proportion)) +
  geom_col(fill = "lightblue", width = 0.85) +
  geom_line(aes(y = exact), color = "firebrick", linewidth = 0.8) +

```

```
geom_vline(xintercept = 14.5, linetype = "dashed") +
facet_wrap(~ model) +
coord_cartesian(xlim = c(0, 35)) +
labs(x = "Defects in the next 50 items", y = "Probability")
```

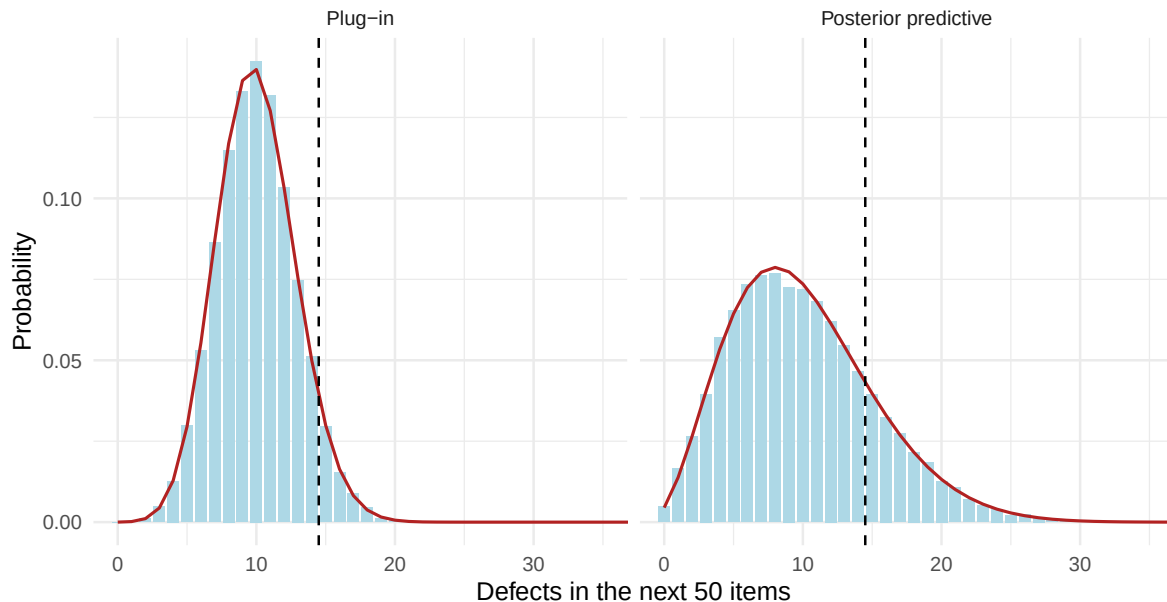


Figure 4.15: The posterior predictive distribution spreads probability over a wider range of defect counts. The plug-in model ignores uncertainty about the process rate. Lines show exact probabilities and bars show simulated proportions.

i Interpretation and Shared Uncertainty

A decision triggered by a large defect count depends on the predictive tail, not just the expected count. Hierarchical simulation carries parameter uncertainty through to that tail.

If we drew a fresh independent p for each individual item instead of one shared p for the batch, the marginal total would be binomial with probability $\mathbb{E}(p)$. That would represent a different mechanism and lose the shared uncertainty in this posterior prediction.

Example 21. Optional Application: Queue Delays

Consider a single server, first-come-first-served, initially idle. Let interarrival times be independent exponentials with rate λ , and service times be independent $\text{Gamma}(2, 1)$ variables, with mean 2 minutes. Assume the arrival and service processes are independent.

If A_i is the time between customers $i - 1$ and i , S_i is customer i 's service time, and W_i is the wait before service, then

$$W_1 = 0, \quad W_i = \max\{0, W_{i-1} + S_{i-1} - A_i\}.$$

This recursion turns independent primitive random inputs into dependent waiting times. A long service can delay several later customers.

We compare arrival rates 0.35 and 0.45 per minute. Their utilization values are $\lambda\mathbb{E}(S) = 0.7$ and 0.9. Both are below one, but the system closer to capacity has less room to absorb bursts.

```
simulate_queue <- function(arrival_rate, n_customers = 200) {
  interarrival <- rexp(n_customers, rate = arrival_rate)
  service <- rgamma(n_customers, shape = 2, rate = 1)
  wait <- numeric(n_customers)
  if (n_customers > 1) {
    for (i in 2:n_customers) {
      wait[i] <- max(0, wait[i - 1] + service[i - 1] - interarrival[i])
    }
  }
  wait
}

set.seed(8693)
B_days <- 800
rates <- c(0.35, 0.45)
queue_results <- do.call(rbind, lapply(rates, function(rate) {
  # Each replication is an independent run of 200 customers, starting empty.
  fraction_delayed <- replicate(
    B_days, mean(simulate_queue(rate) > 5)
  )
  data.frame(
    arrival_rate = rate,
    utilization = 2 * rate,
    fraction_waiting_over_5 = mean(fraction_delayed),
    MCSE = sd(fraction_delayed) / sqrt(B_days)
  )
}))
knitr::kable(queue_results, digits = 4, col.names = c(
  "Arrival rate", "Utilization", "Fraction waiting > 5", "MCSE"
))
```

| Arrival rate | Utilization | Fraction waiting > 5 | MCSE |
|--------------|-------------|----------------------|--------|
| 0.35 | 0.7 | 0.2438 | 0.0038 |
| 0.45 | 0.9 | 0.5586 | 0.0067 |

```
set.seed(8694)
queue_paths <- do.call(rbind, lapply(rates, function(rate) {
  data.frame(
    customer = 1:200, wait = simulate_queue(rate),
    scenario = paste("Arrival rate =", rate)
  )
}))
ggplot(queue_paths, aes(customer, wait)) +
  geom_line(color = "steelblue", linewidth = 0.6) +
  geom_hline(yintercept = 5, linetype = "dashed", color = "firebrick") +
  facet_wrap(~ scenario) +
  labs(x = "Customer index", y = "Wait before service (minutes)")
```

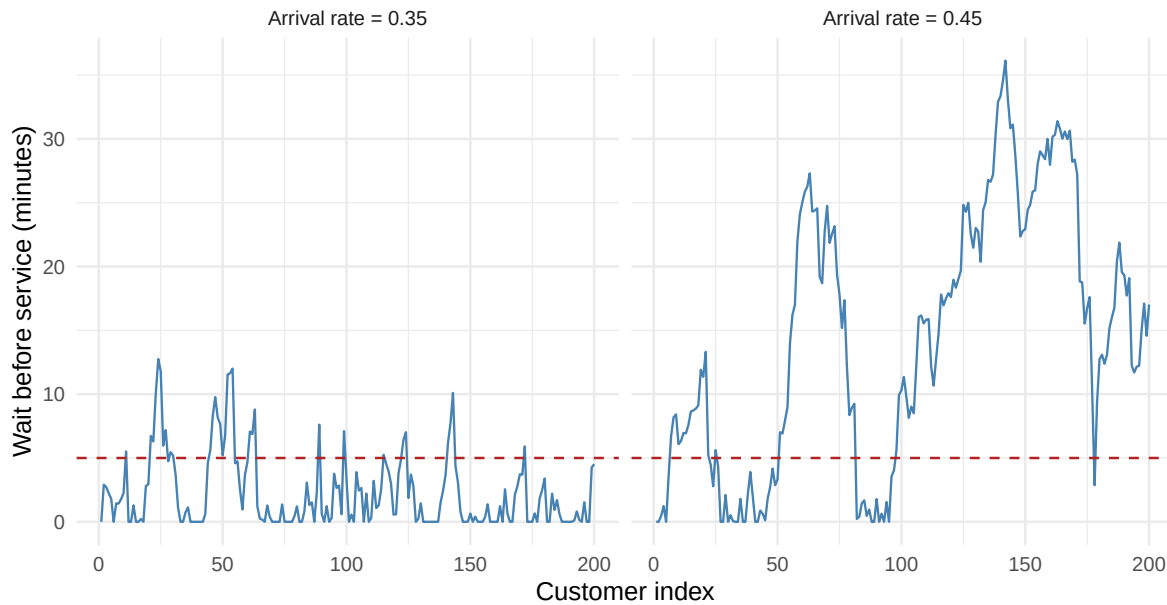


Figure 4.16: Two illustrative paths show how delays carry over between customers. Estimates in the table average 800 independent runs per scenario; one path alone can be misleading.

i Interpretation and Assumptions

The estimand is the expected fraction of the first 200 customers who wait more than five minutes in a system starting empty. It is not automatically the long-run stationary probability. Studying steady-state performance requires addressing the initial transient and dependence.

We compute MCSE from **independent run-level fractions**. Pooling all customers and treating their wait indicators as independent Bernoulli trials would ignore the dependence created by the queue.

This example combines distribution choice, recursive computation, and statistical reasoning about the unit of replication.

Exercises

Exercise 5. Questions to Think at Home

1. Derive an inverse-transform generator for $f(x) = 4x^3$ on $(0, 1)$. Check its theoretical mean and one tail probability.
2. Generate 500 observations from an exponential distribution with mean 5. Explain the difference between `rate = 5` and `rate = 1 / 5`.
3. Why does the generalized inverse use $F(x) \geq u$? Illustrate your answer with a Bernoulli distribution.
4. For a $\text{Beta}(3, 1)$ target and a uniform proposal, find the smallest valid rejection bound M , the acceptance probability, and the expected number of proposals for 1000 accepted draws.
5. Derive the acceptance probability when the target is known only through an unnormalized density. Which constant need not be computed?
6. Generate a t_4 variable from independent normal and chi-square variables. Explain why its variance is 2 rather than 1. How would you rescale it to unit variance?
7. Compare $0.3X_1 + 0.7X_2$ with a mixture that selects X_1 with probability 0.3. Derive both means and variances for independent $X_1 \sim N(0, 1)$ and $X_2 \sim N(4, 1)$.
8. For a fixed positive definite covariance matrix, compare `Z %*% chol(Sigma)` with `Z %*% t(chol(Sigma))`. Derive the covariance produced by each.
9. In a mixture of bivariate normals, what changes if you draw one component label per coordinate instead of one per row?

10. Derive the radius distribution for a uniform point inside a three-dimensional ball. Verify it using the empirical distribution of simulated radii.
11. Extend the power experiment to unknown variance and a two-sample t test. Compare Type I error and power under normal errors and under rescaled t_3 errors with the same variance.
12. Simulate repeated datasets and estimate the coverage of a 95% confidence interval for an exponential mean. State the true mean, interval procedure, number of datasets, and MCSE of the coverage estimate.
13. In the defect prediction example, vary the posterior concentration $a + b$ while holding $a/(a + b) = 0.2$. What happens to predictive variance, and why?
14. In the queue example, why is the number of independent runs relevant to the MCSE calculation? How would the target quantity change if you analyzed only later customers after a warm-up period?

References and Further Reading

This chapter adapts and expands the [2025 STAT 8670 chapter on generating random variables](#) and its [Quarto source in the master branch](#).

For software details, consult the official R documentation linked at the relevant examples. Useful local help pages include `?RNGkind`, `?Distributions`, `?qgeom`, `?chol`, and `?rWishart`. The course's original reading is Rizzo, M. L. (2007), *Statistical Computing with R*, Chapman & Hall/CRC.

Part I

Assignment

Assignment 1

Due date: Friday, September 11, 2026

Total: 30 points

Submission

For this assignment, please use **R Markdown** or **Quarto**.

Submit both:

1. your source file (`.Rmd` or `.qmd`), and
2. the corresponding rendered PDF.

For multiple-choice questions, clearly indicate your selected answer.

For computational questions, include the R code used to obtain your answer.

Question 1: R Programming and Statistical Computing [10]

Select the **best answer**. Each question is worth **1 point**.

1a)

Which of the following creates a numeric vector in R?

- A. `x <- list(1, 2, 3)`
- B. `x <- c(1, 2, 3)`
- C. `x <- data.frame(1, 2, 3)`
- D. `x <- matrix(1, 2, 3)`

1b)

Which expression correctly returns the mean of `x` after removing missing values?

- A. `mean(x, remove = TRUE)`
- B. `mean(x, missing = FALSE)`
- C. `mean(x, na.rm = TRUE)`
- D. `mean(na.omit = x)`

1c)

Suppose

$$x = (10, 20, 30, 40).$$

What does the following R expression return?

```
x[c(1, 3)]
```

- A. 10 20
- B. 10 30
- C. 20 40
- D. 30 40

1d)

Which statement about R indexing is correct?

- A. R indexing starts from 0.
- B. R indexing starts from 1.
- C. R indexing depends on the data type.
- D. R indexing starts from -1 .

1e)

Which function is most appropriate for applying a function to the **columns of a numeric matrix**?

- A. `apply()`
- B. `lm()`
- C. `filter()`
- D. `rbind()`

1f)

What is the main advantage of using `vapply()` instead of `sapply()`?

- A. It always runs in parallel.
- B. It requires a prespecified output type.
- C. It works only on matrices.
- D. It automatically removes missing values.

1g)

Which of the following is the native R pipe operator?

- A. `%>%`
- B. `->`
- C. `|>`
- D. `%*%`

1h)

Which `dplyr` function is used to retain rows satisfying a condition?

- A. `select()`
- B. `filter()`
- C. `mutate()`
- D. `summarise()`

1i)

Which statement concerning `apply()` and a `for` loop is most accurate?

- A. `apply()` is always faster than a `for` loop.
- B. A `for` loop is always faster than `apply()`.
- C. Neither is always faster; performance depends on the computation.
- D. `apply()` performs computations in parallel by default.

1j)

Why is reproducibility important in statistical computing?

- A. It guarantees that every statistical model is correct.
 - B. It allows an analysis to be rerun and verified.
 - C. It eliminates sampling variability.
 - D. It eliminates numerical error.
-

Question 2: Computational Results Using the Iris Data [10]

Use the built-in `iris` dataset in R.

For each computational question, include the R code used to obtain your answer.

2a) [2]

Report the sample mean of `Sepal.Length`, rounded to three decimal places.

2b) [2]

Report the sample standard deviation of `Sepal.Width`, rounded to three decimal places.

2c) [2]

How many observations in the `iris` dataset belong to the species `virginica`?

- A. 25
- B. 40
- C. 50
- D. 75

2d) [2]

Fit the linear regression model

$$\text{Sepal.Length} = \beta_0 + \beta_1 \text{Sepal.Width} + \varepsilon.$$

Report the estimated slope coefficient $\hat{\beta}_1$, rounded to three decimal places.

2e) [2]

Based on the fitted model in part (d), which statement is correct?

- A. Sepal length tends to increase as sepal width increases.
- B. Sepal length tends to decrease as sepal width increases.
- C. The estimated slope is exactly zero.
- D. The model cannot be fitted because both variables are continuous.

Question 3: R Coding and Visualization [6]

Use the `iris` dataset for both parts.

3a) [3]

Using `ggplot2`, construct a scatter plot with:

- `Sepal.Width` on the x -axis,
- `Sepal.Length` on the y -axis,
- points coloured according to `Species`.

Your figure must contain:

1. appropriate axis labels,
2. an informative title, and
3. a readable legend.

Include both your R code and the resulting figure.

3b) [3]

Add a linear regression line to the plot from part (a).

Use

```
geom_smooth(method = "lm", se = FALSE)
```

and include the final plot in your report.

Question 4: Numerical Linear Algebra [4]

4a) [2]

Suppose Σ is a positive-definite covariance matrix with Cholesky decomposition

$$\Sigma = R^{\top} R,$$

where R is upper triangular.

Which expression correctly computes $\log \det(\Sigma)$?

A.

$$\sum_i \log(R_{ii})$$

B.

$$2 \sum_i \log(R_{ii})$$

C.

$$\sum_i R_{ii}^2$$

D.

$$2 \log \left(\sum_i R_{ii} \right)$$

4b) [2]

Let

$$R = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}, \quad \Sigma = R^T R.$$

Compute $\log \det(\Sigma)$.

- A. $\log(6)$
 - B. $\log(12)$
 - C. $2 \log(6)$
 - D. 6
-

Assignment 2

Due date: October 2, 2026, 23:59.

Total: 30 points

Coverage: Chapter 2 (Computational Approaches) and Chapter 3 (Optimization).

Submission

Use **R Markdown** or **Quarto**. Submit both your **source file** (.Rmd or .qmd) and its **rendered PDF**.

Begin the PDF with an answer table for Questions 1-3, followed by Question 4. Use the supplied response template, or make your own table with the same part labels.

- **Questions 1-2:** Select one best answer, A, B, C, or D.
- **Question 3:** Write **True** or **False**.
- **Question 4:** Explain your reasoning in your own words, using one or two sentences per numbered prompt.

Only Question 4 requires explanations. R code and figures are optional. If you use R to check your work, include the code in your source file.

| Question | Format | Points |
|--------------|---|-----------|
| 1 | 8 multiple-choice questions on Chapter 2 | 8 |
| 2 | 8 multiple-choice questions on Chapter 3 | 8 |
| 3 | 6 true/false questions | 6 |
| 4 | 2 short explanations with four prompts each | 8 |
| Total | | 30 |

The questions use the main material in both chapters; the optional real-world applications at the end of Chapter 3 are not required.

Question 1: Computational Approaches [8]

Select the **best answer**. Each question is worth **1 point**.

1a)

Two formulas are mathematically equivalent in exact arithmetic. Which statement about their computer implementations is most accurate?

- A. They can produce different numerical results because computers use finite-precision arithmetic.
- B. They must produce exactly the same result in every decimal place.
- C. They must use the same amount of memory.
- D. They must take the same amount of time.

1b)

Let A be a nonsingular square matrix and b a compatible vector. Which R expression solves $Ax = b$ **without explicitly forming** A^{-1} ?

- A. `solve(A) %*% b`
- B. `solve(A, b)`
- C. `crossprod(A, b)`
- D. `A %*% b`

1c)

For a numeric matrix X , which expression is mathematically equivalent to `crossprod(X)`?

- A. `X %*% t(X)`
- B. `X * X`
- C. `sum(X * X)`
- D. `t(X) %*% X`

1d)

A regression design matrix has one column that is almost a copy of another. What is the main numerical concern?

- A. The number of observations must have changed.
- B. The response variable must contain missing values.
- C. The normal-equation system can be ill-conditioned, making coefficient estimates sensitive to small perturbations.
- D. The residual sum of squares must be exactly zero.

1e)

Suppose a full-column-rank design matrix has the thin QR decomposition $X = QR$, with $Q^\top Q = I$. Which triangular system gives the least-squares coefficients?

- A. $R\hat{\beta} = Qy$
- B. $R\hat{\beta} = Q^\top y$
- C. $Q\hat{\beta} = R^\top y$
- D. $R^\top \hat{\beta} = Q^\top y$

1f)

Suppose $\Sigma = R^\top R$ is positive definite, with R upper triangular, and $z = x - \mu$. Which computation gives $z^\top \Sigma^{-1} z$ without explicitly forming an inverse?

- A. Solve $R^\top v = z$, then compute $v^\top v$.
- B. Solve $Rv = z$, then compute $v^\top v$.
- C. Compute $v = Rz$, then compute $v^\top v$.
- D. Compute $z^\top z$ without using R .

1g)

Use the same Cholesky convention, with

$$R = \begin{pmatrix} 4 & 1 \\ 0 & 2 \end{pmatrix}, \quad z = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \quad \Sigma = R^\top R.$$

What is $z^\top \Sigma^{-1} z$?

- A. 29
- B. 25
- C. 5
- D. 3

1h)

You evaluate multivariate normal log-densities at many different observations, using the **same positive-definite covariance matrix** Σ . Which work can be reused?

- A. The centered vector $z = x - \mu$ for every observation.
- B. The complete log-density value for every observation.
- C. The quadratic form $z^\top \Sigma^{-1} z$ for every observation.
- D. The Cholesky factor of Σ and its log-determinant.

Question 2: Optimization [8]

Select the **best answer**. Each question is worth **1 point**.

2a)

You want to maximize a log-likelihood $\ell(\theta)$ using an optimizer that **minimizes** its objective. Which objective should you supply?

- A. $\ell(\theta)$
- B. $[\ell(\theta)]^2$
- C. $-\ell(\theta)$
- D. $|\ell(\theta)|$

2b)

Which conditions give the usual root-bracketing guarantee for bisection on an interval $[a, b]$?

- A. $f(a)$ and $f(b)$ are both positive.
- B. f is differentiable at a , regardless of its behavior elsewhere.
- C. The midpoint is smaller than both endpoints.
- D. f is continuous on $[a, b]$ and $f(a)f(b) < 0$.

2c)

Apply **one Newton root-finding update** to $f(x) = x^2 - 3$, starting at $x_0 = 2$. What is x_1 ?
Use $x_1 = x_0 - f(x_0)/f'(x_0)$.

- A. 1.75
- B. 1.50
- C. 2.25
- D. 3.00

2d)

Minimize $Q(x) = (x - 3)^2$ using gradient descent. Starting at $x_0 = 0$ with step size $\alpha = 0.25$, what is the next iterate?

Use $x_1 = x_0 - \alpha Q'(x_0)$.

- A. -1.50
- B. 1.50
- C. 0.75
- D. 3.00

2e)

What is the main computational idea behind BFGS?

- A. Enumerate every feasible parameter value.
- B. Use the exact Hessian inverse at every iteration.
- C. Guarantee a global optimum by adding random noise.
- D. Update an approximation to curvature information instead of repeatedly computing the exact Hessian.

2f)

Consider $Q(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 100(\theta_2 - 1)^2$. Why can a single fixed step size be difficult for gradient descent?

- A. The much larger curvature in one direction can cause zig-zagging or force a small step size.
- B. The function is not differentiable.
- C. The function has infinitely many local minima.
- D. The gradient is zero at every point.

2g)

In simulated annealing for a minimization problem, a proposed move increases the objective by $\Delta Q = 2$. At temperature $T = 1$, what is its acceptance probability?

Use $P(\text{accept}) = \exp(-\Delta Q/T)$.

- A. 0
- B. $e^{-2} \approx 0.1353$
- C. 0.5
- D. 1

2h)

A smooth, nonconvex objective is minimized with BFGS from several starting values. The returned objective values differ. What is the most appropriate next step?

- A. Always keep the run with the fewest iterations.
- B. Average all parameter vectors without evaluating the objective there.
- C. Compare feasible returned objective values, examine diagnostics, and consider additional starts.
- D. Declare the first converged result globally optimal.

Question 3: True or False [6]

Write **True** or **False** for each statement. Each statement is worth **1 point**. No explanation is required.

3a)

For a full-column-rank regression design matrix, a normal-equation implementation and a QR implementation must return coefficients that agree in every floating-point digit.

3b)

When optimizing a multivariate normal log-likelihood over μ and Σ with fixed dimension d and sample size, dropping an additive term that is constant in these parameters does not change the maximizer.

3c)

Replacing an explicit matrix inverse with a direct linear-system solve guarantees accurate coefficients even when the matrix is arbitrarily ill-conditioned.

3d)

A solution of $Q'(x) = 0$ can be a local maximum rather than a local minimum.

3e)

Under suitable smoothness conditions and with a starting point sufficiently close to a simple root, Newton's method has quadratic convergence.

3f)

A genetic algorithm uses one evolving candidate solution, whereas simulated annealing maintains a population of candidate solutions.

Question 4: Explain What the Results Mean [8]

Each part is worth **4 points: 1 point for each numbered prompt**. Use one or two sentences per prompt. We grade the statistical and computational reasoning, not the exact wording of your response.

4a) Rank deficiency is not missing data [4]

Suppose a regression design matrix contains an intercept and two predictors, with $x_2 = 2x_1$ exactly and x_1 nonconstant. There are no missing values. You try

```
solve(crossprod(X), crossprod(X, y))
```

and receive a computationally singular system error. A QR-based fit instead reports the numerical rank and returns NA for one redundant coefficient.

Explain:

1. What feature of the design matrix causes the normal-equation solve to fail?
2. What does the NA coefficient mean in this situation?
3. Can a QR-based approach still provide ordinary least-squares fitted values? Explain how.
4. What is one sensible way to remove the redundancy while retaining the same ordinary least-squares fitted values?

4b) A converged result still needs interpretation [4]

You are minimizing

$$Q(x) = (x^2 - 1)^2 + 0.2x + 1, \quad x \in \mathbb{R}.$$

Two BFGS runs give the following results, rounded to four decimal places:

| Run | Starting value | Returned value | Objective value | |
|-----|----------------|----------------|-----------------|------------------|
| | | | $Q(x)$ | Convergence code |
| A | 1 | 0.9740 | 1.1974 | 0 |
| B | -1 | -1.0241 | 0.7976 | 0 |

Here, code 0 means the optimizer reports successful completion according to its convergence criteria. You do not need to run the optimizer or prove global optimality.

Explain:

1. Which returned point would you retain as the **best found so far**, and why?
2. How can a deterministic method return different answers for the same objective?
3. Do these convergence codes alone establish that either point is a global minimum? Explain.
4. Describe one additional check or search you would perform, and explain what it would tell you.